Jaque al modelo

Afiné dos IAs para jugar ajedrez y descubrí que el árbitro jugaba por ellas

Bitácora real de un duelo entre Qwen y Gemma en una GPU de 6 GB

Denis Sánchez Leyva

Vertex Coders LLC · Miami, FL

Jaque al modelo: afiné dos IAs para jugar ajedrez y descubrí que el árbitro jugaba por ellas

Primera edición, 2026

© 2026 Denis Sánchez Leyva — Vertex Coders LLC. Todos los derechos reservados.

Ninguna parte de esta publicación puede ser reproducida, distribuida o transmitida en cualquier forma o por cualquier medio sin el permiso previo por escrito del autor, salvo en el caso de citas breves para reseñas y ciertos usos no comerciales permitidos por la ley de derechos de autor.

Los modelos descritos en este libro son obras derivadas de Qwen2.5-3B-Instruct (Qwen Team, Alibaba Cloud) y de Gemma 2 2B (Google), y están sujetos a las licencias y términos de uso de sus modelos base. Los datos de entrenamiento proceden de conjuntos públicos de Lichess y de Hugging Face. El código y los ejemplos de este libro se ofrecen con fines educativos, «tal cual», sin garantía de ningún tipo.

El tablero de la aplicación usa cm-chessboard (licencia MIT) y las piezas de ajedrez de Colin M. L. Burnett (licencia CC BY-SA 3.0). Las reglas del juego se validan con python-chess y chess.js.

Las marcas y nombres de productos mencionados pertenecen a sus respectivos dueños.

Publicado de forma independiente.

A los que entrenan modelos de noche, después del turno, con una GPU que no da para más.
Y a todos los que alguna vez creyeron que su modelo era bueno... hasta que lo midieron.

Prefacio

La idea parecía sencilla. Tomo dos modelos de lenguaje abiertos, pequeños, de los que caben en una tarjeta gráfica de casa. Los afino con el mismo conjunto de posiciones de ajedrez. Los pongo a jugar uno contra otro. Gana el mejor.

Ganó Qwen. Ganó siempre. Y eso que, según yo, había entrenado bastante menos que Gemma.

Ese "y eso que" es el motivo de este libro. Un resultado que contradice lo que esperas es la mejor noticia que te puede dar un experimento, porque te obliga a mirar debajo. Y lo que encontré debajo no fue un modelo prodigioso. Fue un árbitro que, sin que nadie se lo pidiera, jugaba por los modelos; un adaptador que no era el que yo creía; una función entera que no se ejecutaba nunca; un parser que aceptaba jugadas que no existían. Cada pieza, por separado, parecía razonable. Juntas, fabricaban un ganador.

Este no es un libro de ajedrez, aunque haya tableros en casi todas las páginas. Es un libro sobre **medir**. Sobre la diferencia entre evaluar un modelo y evaluar el sistema que lo rodea. Sobre por qué una cifra que te gusta merece más sospecha que una que no te gusta. Y sobre cómo se arregla todo eso cuando lo encuentras: con un entrenamiento espejo, un duelo justo y una arena con reloj, como la de cualquier torneo.

Todo lo que cuento pasó de verdad, en mi máquina, entre el 16 y el 20 de octubre de 2026, y está registrado en la bitácora del proyecto. Cuando digo que algo tardó 13 horas y 42 minutos, es porque tardó eso. Cuando digo que un modelo acertó 4 de 40, es porque lo medí con las mismas 40 posiciones que al otro.

Si estás afinando tu propio modelo, ojalá este libro te ahorre la semana que a mí me costó descubrir que el árbitro jugaba por ellas.

— Denis Sánchez Leyva
Miami, 2026

Para quién es este libro

Este libro es para ti si:

- Has afinado (o quieres afinar) un modelo de lenguaje abierto con QLoRA y quieres saber si de verdad aprendió algo.
- Comparas modelos entre sí y te preguntas si la comparación es justa.
- Construyes sistemas que combinan un LLM con reglas, motores o herramientas, y necesitas separar lo que aporta cada parte.
- Trabajas con hardware modesto (una GPU de 6 GB, Windows) y estás cansado de tutoriales que dan por hecho una A100.
- Te gusta el ajedrez y te da curiosidad qué pasa cuando un modelo de lenguaje intenta jugarlo.

No necesitas ser un experto en ajedrez: explico lo justo (qué es un FEN, qué es SAN, cómo funciona un reloj de torneo) cuando hace falta. Sí conviene que sepas Python básico y que hayas oído hablar de *fine-tuning*. El frontend está en Angular y TypeScript, pero los capítulos que lo tratan se entienden sin haber escrito nunca una línea de Angular.

Cómo leer este libro

El libro sigue el orden en que pasaron las cosas.

- **Capítulos 1 a 7: la construcción.** La idea, el laboratorio, el dataset, el entrenamiento de cada modelo, el paso a Ollama y el árbitro. Aquí todo parece funcionar.
- **Capítulo 8: la auditoría.** El corazón del libro. Por qué Qwen siempre ganaba.
- **Capítulos 9 a 11: el duelo justo.** Cómo se arregla, la arena profesional con reloj y los resultados de verdad.
- **Capítulos 12 y 13: lo que te llevas.** Lecciones para cualquiera que afine modelos, y cómo publicarlo todo sin gastar dinero.

Si tienes prisa, lee el capítulo 1 y salta al 8. Si vas a reproducir el proyecto, léelo en orden y ten a mano los apéndices, que recogen los comandos con el directorio desde el que se ejecuta cada uno, los hiperparámetros exactos y las partidas clave en PGN.

Los apéndices D a J tienen **el código completo del proyecto**, archivo por archivo, con los números de línea y una guía de lectura al principio de cada uno: el pipeline de datos, los dos entrenamientos, la exportación, el árbitro, la arena por consola y los diagnósticos, la API y el reloj con la lógica de torneos. Si un capítulo te deja con ganas de ver cómo está hecho algo, el código está ahí, tal como se ejecutó.

Las cifras salen de la bitácora del proyecto (`logs/bitacora.md`) y de los archivos de diagnóstico. Cuando una cifra aparece, es porque se midió.

Índice

Prefacio iv
Para quién es este libro v
Cómo leer este libro vi
Capítulo 1 — La idea 1
Capítulo 2 — El laboratorio 4
Capítulo 3 — El dataset 7
Capítulo 4 — Afinando a Gemma 2 (2B) 10
Capítulo 5 — Afinando a Qwen 2.5 (3B) 13
Capítulo 6 — Del adaptador a Ollama 15
Capítulo 7 — El árbitro 18
Capítulo 8 — “Qwen siempre gana” 21
Capítulo 9 — Un duelo justo 24
Capítulo 10 — La arena profesional 27
Capítulo 11 — La verdad del tablero 31
Capítulo 12 — Lecciones para quien afina modelos 35
Capítulo 13 — Publicarlo sin gastar 38
Conclusión 42
Apéndice A — Comandos, con su directorio 43
Apéndice B — Hiperparámetros 45
Apéndice C — Partidas clave 47
Apéndice D — El pipeline de datos 49
Apéndice E — Los dos entrenamientos 64
Apéndice F — Exportación a GGUF y Modelfiles 77
Apéndice G — El árbitro 81
Apéndice H — La arena por consola y los diagnósticos 109

Apéndice I — La API del backend 116
Apéndice J — El reloj y la lógica de torneos 128
Sobre el autor 138

CAPÍTULO 1

La idea

Hay preguntas que solo se contestan poniendo a dos cosas a pelear. ¿Qué modelo pequeño aprende mejor una tarea concreta? Puedes leer *benchmarks*, comparar tablas, discutir en foros. O puedes hacer lo que haría cualquiera en un barrio: ponerlos frente a frente y ver quién gana.

La tarea que elegí fue el ajedrez.

Por qué ajedrez

El ajedrez tiene tres propiedades que lo hacen perfecto para medir un modelo de lenguaje:

1. **Las reglas son exactas.** Una jugada es legal o no lo es. No hay "más o menos". Eso significa que cada respuesta del modelo se puede comprobar con un programa, sin opiniones.
2. **El resultado es indiscutible.** Una partida termina en victoria, derrota o tablas. No hace falta un juez humano que diga si la respuesta "suena bien".
3. **Hay datos de sobra.** Lichess, el servidor de ajedrez libre más grande del mundo, publica millones de partidas, posiciones evaluadas por motores y problemas tácticos con su solución.

Y una cuarta, que no es técnica pero importa: el ajedrez se entiende. Cualquiera que vea un tablero sabe si un modelo hizo una tontería. Eso hace que los fallos sean visibles, y los fallos visibles son los que enseñan.

Los dos contendientes

Elegí dos modelos abiertos que cupieran en mi tarjeta, una **GTX 1660 Ti de 6 GB**:

- **Qwen 2.5 de 3.000 millones de parámetros (3B)**, versión *Instruct*, de Alibaba.
- **Gemma 2 de 2.000 millones de parámetros (2B)**, versión *it*, de Google.

Los dos son modelos de chat de propósito general. Ninguno fue entrenado para jugar ajedrez. Saben qué es el ajedrez, como saben qué es la fotosíntesis, pero si les das una posición y les pides la mejor jugada, contestan con párrafos llenos de confianza y jugadas que a veces ni existen.

La idea era enseñarles una sola cosa: dada una posición, elegir la mejor jugada y decirla en un formato fijo. Con el mismo dataset, con la misma técnica (QLoRA), en la misma máquina. Y luego, a jugar.

Qué le pedimos a un modelo

Para que la comparación sea limpia, el modelo nunca juega "de memoria". En cada turno recibe tres cosas:

```
Position FEN: r1bqkbnr/pppp1ppp/2n5/4p3/4P3/5N2/PPPP1PPP/RNBQKB1R w KQkq -
↪  2 3
Legal moves (SAN): Bb5, Bc4, Nc3, d4, ...
Select the best move.
```

- El **FEN**, que es la forma estándar de escribir una posición de ajedrez en una línea de texto.
- La lista de **jugadas legales** en notación SAN (la que usan los libros: `Nf3`, `Bxc6`, `O-O`).
- La instrucción.

Y debe contestar exactamente así:

```
Best move: Bb5 (UCI: f1b5)
```

La jugada en SAN y, entre paréntesis, en UCI, que es la notación "de máquina": casilla de origen y casilla de destino.

Darle la lista de jugadas legales es una decisión de diseño importante. Un modelo de 3B no sabe calcular qué jugadas son legales en una posición; eso lo hace mejor un programa en microsegundos. Lo que queremos medir es otra cosa: si, entre las jugadas posibles, el modelo sabe cuál es buena.

Alrededor de los modelos: la arena

Un modelo que contesta "Best move: Bb5" no es todavía un jugador de ajedrez. Hace falta alguien que lleve el tablero, que compruebe la jugada, que pase el turno, que decida cuándo se acaba la partida. A ese alguien lo llamé **el árbitro**: un programa en Python, con la librería `python-chess`, que habla con los modelos a través de Ollama.

Encima del árbitro puse una página web sencilla para ver las partidas. Más adelante (capítulo

10) esa página se convirtió en una aplicación de torneos con reloj, al estilo de chess.com.

Guarda la palabra *árbitro*. Va a ser la protagonista inesperada de este libro.

La sorpresa

Entrené a los dos. Los puse a jugar. Qwen ganó. Los volví a poner. Qwen ganó otra vez. Partida tras partida, Qwen.

Lo raro no era que ganara uno. Lo raro era quién. Según lo que yo creía, Gemma había entrenado 625 pasos y Qwen solo 80. Era como si el que menos había estudiado sacara siempre la mejor nota.

Hay dos formas de reaccionar a algo así. La primera es inventarse una explicación bonita: "Qwen es una arquitectura superior para el razonamiento formal". Lo escribí, de hecho, en la primera versión de la bitácora. La segunda es sospechar. Este libro cuenta lo que pasó cuando, por fin, sospeché.

Lo que te llevas de aquí

- Elige tareas donde la respuesta se pueda **verificar con un programa**. Te ahorrarán discusiones y te darán números honestos.
- Separa lo que debe hacer el modelo de lo que puede hacer un programa. Las jugadas legales las calcula `python-chess`; el modelo solo elige.
- Un resultado que contradice tus expectativas no es un éxito ni un fracaso: es una pista.

Lo que viene: en el capítulo 2 montamos el laboratorio, una GPU de 6 GB con Windows y todas sus trampas.

CAPÍTULO 2

El laboratorio

Todo este proyecto vive en una sola carpeta de un disco aparte: `E:\vertex-chess-arena`. Parece un detalle de orden, pero es la primera decisión técnica del libro.

Aislar el laboratorio

Afinar modelos genera basura en cantidades industriales: modelos descargados de varios gigas, cachés de datasets, archivos temporales, compilaciones. Por defecto, las librerías de Hugging Face y PyTorch lo guardan todo en la carpeta del usuario, en el disco `C:`. Con dos modelos y tres datasets, el disco del sistema se llena antes de que te des cuenta.

La solución es redirigir todas las cachés al disco del proyecto, al principio de cada script y antes de importar nada:

```
BASE_DIR = Path("E:/vertex-chess-arena").resolve()
CACHE_DIR = BASE_DIR / ".cache"
os.environ["HF_HOME"] = str(CACHE_DIR / "huggingface")
os.environ["TORCH_HOME"] = str(CACHE_DIR / "torch")
os.environ["TMP"] = str(CACHE_DIR / "tmp")
os.environ["TEMP"] = str(CACHE_DIR / "tmp")
```

Con eso, el proyecto completo (modelos, datos, cachés, adaptadores) cabe y se borra en una sola carpeta. Y un entorno virtual de Python (`venv`) dentro de la misma carpeta guarda las versiones exactas que funcionaron.

La GPU: una Turing de 6 GB

La tarjeta es una **NVIDIA GeForce GTX 1660 Ti** de portátil, con **6 GB** de memoria y arquitectura **Turing**. Tres datos de esa ficha condicionan todo lo demás:

1. **6 GB de memoria.** Un modelo de 3B en precisión completa ocupa unos 12 GB. No cabe ni para mirarlo. Por eso usamos **QLoRA**: el modelo base se carga comprimido a 4 bits y solo se entrenan unas

matrices pequeñas que se le añaden (el *adaptador* LoRA). Qwen 3B queda en unos 2 GB; Gemma 2B, en algo menos.

2. **Turing no soporta `bf16`.** El formato numérico que usan casi todos los tutoriales modernos para entrenar no existe en esta tarjeta. Hay que usar `fp16`.
3. **Esta 1660 Ti no tiene *tensor cores*.** Son los circuitos especializados que aceleran las multiplicaciones de matrices en las GPU RTX. Sin ellos, el `fp16` funciona, pero despacio. Este dato no lo vi venir, y explica una de las cifras más dolorosas del libro: el entrenamiento final de Qwen tardó **unos 79 segundos por paso**.

Las trampas de Gemma con 6 GB

Gemma 2 tiene un vocabulario enorme: unos **256.000 tokens**. Cada vez que el modelo predice la siguiente palabra, produce una puntuación para cada uno de esos 256.000 tokens. Con secuencias de cientos de tokens, esa matriz de puntuaciones, por sí sola, se come buena parte de la memoria.

El primer intento de entrenar a Gemma usaba Unsloth, un framework que acelera el *fine-tuning* fusionando operaciones. Al llegar al **paso 9** se cayó con este error:

```
Unsloth: No or negligible GPU memory available for fused cross entropy
```

La "entropía cruzada fusionada" es justo la optimización que calcula la pérdida sobre ese vocabulario gigante. En 6 GB no había sitio. Los intentos siguientes chocaron con el asignador de memoria de PyTorch y con dependencias que pedían un compilador de C que la máquina no tenía.

La solución, documentada en la bitácora del 17 de octubre, tuvo cuatro partes:

- Desactivar la pérdida fusionada (`UNSLOTH_USE_UNSLOTH_FUSED_CE=0`) y, al final, entrenar a Gemma sin Unsloth, con las librerías estándar: `transformers`, `peft` y `bitsandbytes`.
- Cargar el modelo en `torch.float16` con atención `eager` (la implementación más simple y la que Gemma 2 recomienda por cómo limita sus valores internos).
- Limitar la longitud a **384 tokens**, que cubren de sobra una posición con su lista de jugadas y la respuesta.
- Ajustar el asignador de memoria:

```
os.environ["PYTORCH_CUDA_ALLOC_CONF"] =
↪  "expandable_segments:True,max_split_size_mb:32"
```

Con eso, Gemma entrenó entero. El capítulo 4 cuenta cómo.

Inferencia: Ollama

Para jugar, los modelos no corren en Python. Se convierten a **GGUF** (el formato de llama.cpp), se comprimen a 4 bits (`q4_k_m`) y se sirven con **Ollama**, que los expone en `http://127.0.0.1:11434`. El árbitro les manda cada posición por HTTP y lee la respuesta.

Esto tiene una consecuencia que conviene tener presente desde ya: **Ollama y el entrenamiento comparten la misma GPU.** Si juegas una partida mientras entrenas, los dos se pelean por los 6 GB. Lo medimos sin querer: el entrenamiento de Qwen pasó de 79 a unos 145 segundos por paso mientras se jugaban partidas en la arena.

Lo que te llevas de aquí

- Aísla el laboratorio en una carpeta y redirige todas las cachés allí antes de importar nada.
- Lee la ficha de tu GPU antes de copiar un tutorial: memoria, soporte de `bf16` y *tensor cores* cambian qué puedes hacer y cuánto va a tardar.
- Los vocabularios grandes cuestan memoria en el cálculo de la pérdida. Con 6 GB, las optimizaciones "fusionadas" pueden ser el problema y no la solución.
- No entrenes y sirvas modelos a la vez en la misma GPU si quieres tiempos predecibles.

Lo que viene: en el capítulo 3 construimos el dataset: 5.000 posiciones y un detalle que pasó desapercibido.

CAPÍTULO 3

El dataset

Un modelo aprende lo que le enseñas, no lo que crees que le enseñas. Por eso el dataset merece un capítulo propio.

La receta: 5.000 posiciones de tres fuentes

El archivo final, `train_chess_elite.jsonl`, tiene **5.000 posiciones**, cada una con su mejor jugada. Salen de tres conjuntos públicos en Hugging Face:

Fuente	Posiciones	Qué aporta
`Lichess/chess-position-evaluations`	2.000 (40 %)	Posiciones evaluadas por motor, con ventaja clara
`Lichess/chess-puzzles`	2.000 (40 %)	Golpes tácticos y mates directos
`agi-noobs/chess-sft-20k`	1.000 (20 %)	Posiciones curadas para *fine-tuning*

Los datasets se leen **en streaming**: el script va recorriendo filas sin descargar los archivos completos, que en el caso de Lichess ocupan muchísimo más de lo que cabe en un portátil.

Evaluaciones de Lichess

De cada posición evaluada por el motor nos quedamos con la primera jugada de su línea principal, pero solo si la posición es **decisiva**: o hay un mate a la vista, o la ventaja es de al menos 150 centipeones (un peón y medio). Además se descartan posiciones con menos de 7 piezas (finales casi resueltos) y las de más allá de la jugada 40.

Problemas de Lichess

Los *puzzles* de Lichess tienen una trampa de formato que conviene conocer: la posición guardada es **anterior** al error del rival. La primera

jugada de la solución es la del rival que se equivoca, y la segunda es la que lo castiga. Así que el script aplica primero la jugada del rival y luego toma la siguiente como respuesta correcta.

Se reservó una cuota del 25 % (500 posiciones) para **mates directos**, es decir, jugadas cuyo SAN termina en `#`. El resto son golpes tácticos de problemas con rating de 1.900 o más. La idea era darles a los modelos "instinto de mate". Veremos en el capítulo 11 que no alcanzó.

Sin duplicados

Antes de añadir una posición, el script comprueba que su FEN no esté ya en el conjunto. Una posición repetida es un examen que el modelo ya vio; repetirla solo infla los aciertos.

El formato de cada ejemplo

Cada línea del archivo es una conversación de tres mensajes, lista para entrenar un modelo de chat:

```
{"role": "system", "content": "You are an elite chess grandmaster engine
↪  created by Vertex Coders. ..."},
{"role": "user", "content": "Position FEN: ...\nLegal moves (SAN):
↪  ...\nSelect the best move."},
{"role": "assistant", "content": "Best move: Rexe7+ (UCI: e1e7)"}
```

Antes de escribirla, `python-chess` comprueba que la posición es válida y que la jugada es legal en ella. Si no, la fila se descarta.

La auditoría

Un segundo script, `01b_audit_chess_dataset.py`, relee el archivo y verifica las cuotas (40/40/20), la legalidad de cada jugada, que la respuesta tenga el formato exacto, y el reparto por fases de la partida y por bando. Sobre el archivo final:

- **2.588** posiciones con turno de blancas y **2.412** de negras.
- **1.457** jugadas correctas dan jaque o mate; **570** son mate directo.
- **1.546** son capturas.

El detalle que pasó desapercibido

Al preparar este capítulo recalculé una cosa que la auditoría no miraba. El script que construye el prompt corta la lista de jugadas legales a las **primeras 45**:

```
legal_str = ", ".join(legal_sans[:45])
```

La intención era razonable: no hacer el prompt demasiado largo. Pero en el medio juego hay posiciones con más de 45 jugadas legales. Resultado medido sobre el archivo:

- **498 posiciones** (casi el 10 %) llevan la lista recortada.
- En **9 de ellas**, la jugada correcta **no aparece en la lista** que ve el modelo.

Nueve de cinco mil no cambian el resultado del experimento. Pero la lección vale oro: en esos nueve ejemplos le estamos enseñando al modelo a contestar una jugada que no está entre las opciones que le dimos. Y cuando más adelante la arena le mande la lista **completa**, el modelo verá prompts un poco distintos de los que vio al entrenar. Ninguna auditoría detecta lo que no se le pide comprobar.

Lo que te llevas de aquí

- Lee los datos en streaming si la fuente es enorme; no necesitas descargarla entera.
- Conoce el formato de cada fuente. En los puzzles de Lichess, la primera jugada es del rival.
- Elimina duplicados por la clave que importa (aquí, el FEN), no por la línea completa.
- Valida cada ejemplo con un programa antes de guardarlo.
- Audita también lo que **no** se te ocurrió auditar: cualquier recorte, filtro o truncamiento silencioso.

Lo que viene: en el capítulo 4, la odisea de afinar a Gemma en 6 GB.

CAPÍTULO 4

Afinando a Gemma 2 (2B)

Gemma fue el modelo difícil. No porque aprendiera peor, sino porque llegar a entrenarla en una tarjeta de 6 GB costó varios intentos. Este capítulo cuenta la receta que funcionó y, sobre todo, por qué cada pieza está ahí.

QLoRA en una frase

Cargamos el modelo base **comprimido a 4 bits** y lo dejamos congelado. Al lado de algunas de sus capas añadimos dos matrices pequeñas, `A` y `B`, y **solo entrenamos esas**. El resultado del entrenamiento no es un modelo nuevo, es un **adaptador** de pocos megas que se "enchufa" al modelo base.

Dos números controlan el adaptador:

- **`r` (rango):** el tamaño de las matrices. Usamos `r = 8`. Más rango, más capacidad y más memoria.
- **`alpha`:** cuánto pesa el adaptador sobre el modelo base. Usamos `alpha = 16`, el doble del rango, que es la convención habitual.

La receta

Parámetro	Valor	Por qué
Modelo base	Gemma 2 2B *it*, 4 bits	Cabe en 6 GB con margen
LoRA	`r=8`, `alpha=16`	Pequeño y estable
Tasa de aprendizaje	`5e-5`, calentamiento de 15 pasos, coseno	Conservadora: no queremos que olvide hablar
Lote efectivo	8	Lote real pequeño + acumulación de gradientes

Parámetro	Valor	Por qué
Longitud máxima	384 tokens	Posición + jugadas + respuesta
Precisión	`fp16`	Turing no tiene `bf16`
Atención	`eager`	La recomendada para Gemma 2
Pasos	625	5.000 ejemplos ÷ 8 = **una época**
Gradient checkpointing	activado	Recalcula activaciones en vez de guardarlas: menos memoria
Guardado intermedio	ninguno (`save_strategy="no"`)	Ver "Reanudar sin miedo"

Entrenar solo la respuesta

Esta es la decisión más importante del capítulo, y la que más adelante explicará parte de la diferencia entre los dos modelos.

Cada ejemplo tiene un prompt largo (sistema + FEN + lista de jugadas) y una respuesta cortísima (`Best move: Nf3 (UCI: g1f3)`). Si calculas la pérdida sobre **todo** el texto, el modelo pasa la mayor parte del entrenamiento aprendiendo a copiar listas de jugadas legales, que ya le damos hechas. La señal útil, la jugada, se ahoga.

La solución es **enmascarar** el prompt: a esos tokens se les pone la etiqueta `-100`, que la función de pérdida ignora. Así el modelo solo recibe nota por la respuesta:

```
prompt_ids = tokenizer.encode(prompt_text, add_special_tokens=False)
answer_ids = tokenizer.encode(answer_text, add_special_tokens=False)
input_ids = prompt_ids + answer_ids
labels    = [-100] * len(prompt_ids) + answer_ids
```

Gemma se entrenó así desde el principio. El primer Qwen, no. Guarda este dato para el capítulo 5.

Once horas

Los 625 pasos tardaron **unas 11 horas** en la GTX 1660 Ti: alrededor de un minuto por paso. Sin *tensor cores*, en `fp16`, con un lote real de una sola secuencia. Es lento, pero cabe en una noche. Como referencia, Qwen V3, con un 50 % más de parámetros, tardaría 13 h 42 min con la misma receta (capítulo 9).

Reanudar sin miedo (la lección que aprendí después)

Un entrenamiento de horas en un portátil se va a interrumpir tarde o temprano: un corte de luz, una actualización de Windows, tú mismo cerrando la ventana. Gemma entrenó sus 11 horas **sin un solo punto de guardado** (`save_strategy="no"`). Salió bien, pero fue suerte: un corte en la hora diez habría costado las diez horas.

Qwen V3 ya se entrenó con la lección aprendida: guarda un *checkpoint* cada 125 pasos y, al arrancar, busca el último y continúa desde ahí (apéndice E). Perder como mucho 125 pasos es aceptable; perderlo todo, no.

Lo que te llevas de aquí

- QLoRA te deja entrenar modelos de 2–3B en 6 GB: base congelada a 4 bits y un adaptador pequeño.
- Si la respuesta es corta y el prompt largo, **enmascara el prompt**. Si no, entrenas al modelo a copiar.
- Una época completa (aquí, 625 pasos) es la unidad honesta de comparación.
- Guarda *checkpoints* y programa la reanudación desde el primer día.

Lo que viene: en el capítulo 5 afinamos a Qwen… o eso creíamos: la versión de 80 pasos que casi no aprendió nada.

CAPÍTULO 5

Afinando a Qwen 2.5 (3B)

Qwen fue el modelo fácil. Cabía, arrancaba, entrenaba rápido. Precisamente por eso nadie lo miró con lupa, y ahí empezó el problema.

Tres Qwen en la misma carpeta

Al auditar el proyecto (capítulo 8) aparecieron **tres** versiones de Qwen afinado. Conviene presentarlas aquí, porque la confusión entre ellas explica media historia:

Versión	Dónde vive	Receta	¿Llegó a la arena?
Legacy	`models/qwen_chess_adapter` (17-oct, 03:15)	80 pasos, `r=8`, pérdida sobre todo el texto	**Sí.** Era el que jugaba
V2	`models/adapters/qwen_chess_lora` (17-oct, 13:25)	250 pasos, `r=16`, `alpha=32`, `lr 1.5e-4`	No. Nunca se exportó
V3	`models/qwen_chess_adapter_v3` (20-oct)	Receta espejo de Gemma, 625 pasos	Sí, desde el capítulo 9

El script de exportación buscaba el adaptador en `models/qwen_chess_adapter`. El script de entrenamiento V2 guardaba en `models/adapters/qwen_chess_lora`. Dos rutas distintas, y ningún error: el exportador encontró un adaptador válido (el viejo) y lo convirtió sin quejarse. **Durante todo el experimento, el Qwen de la arena fue el de 80 pasos.**

Cómo entrenaba la versión vieja

Los scripts V1 y V2 usaban `SFTTrainer` de la librería TRL con la opción `dataset_text_field="text"`: se le pasa la conversación entera ya formateada como un solo texto y el entrenador calcula la pérdida **sobre todo**. Sistema, FEN, lista de jugadas y respuesta.

Recuerda el capítulo 4. Cada ejemplo tiene unos 270 tokens de prompt y unos 15 de respuesta. Si la pérdida cuenta todo, más del 90 % del esfuerzo se va en aprender a reproducir prompts. La curva de pérdida baja (la V2 pasó de 2,33 a 0,55 en 250 pasos) y parece que el modelo aprende. Lo que aprende es a copiar FEN.

Y 250 pasos con lote efectivo de 8 son 2.000 ejemplos: **0,4 épocas**. Ni siquiera vio el dataset completo una vez. El legacy, con 80 pasos, vio 640.

Qué sabía hacer el legacy

En el sondeo del capítulo 9 le dimos al legacy 40 posiciones del propio dataset de entrenamiento. Resultado: **0 de 40** respuestas con el formato `Best move:`. Siempre contestaba con prosa del estilo *"The strongest tactical move in this position is..."*. No había aprendido ni la forma de la respuesta.

Y sin embargo, ganaba. El capítulo 8 explica cómo.

Lo que te llevas de aquí

- Cuando un script entrena y otro exporta, **una sola constante** debe definir la ruta del adaptador. Dos rutas escritas a mano son dos versiones de la verdad.
- Un exportador que "encuentra algo" no es un exportador que encuentra lo correcto. Imprime la fecha y el `train_log` del adaptador que exportas.
- Una curva de pérdida que baja no prueba que el modelo aprenda la tarea. Pregunta **qué** tokens están en la pérdida.
- Cuenta épocas, no pasos: pasos × lote ÷ tamaño del dataset.

Lo que viene: en el capítulo 6 pasamos del adaptador a Ollama, el camino que siguieron los dos modelos para llegar al tablero.

CAPÍTULO 6

Del adaptador a Ollama

Un adaptador LoRA no se puede servir solo: necesita su modelo base al lado. Para jugar cientos de partidas lo práctico es fusionar los dos, comprimirlos y servirlos con Ollama.

De LoRA a GGUF

Unsloth hace la fusión y la conversión en una llamada:

```
model.save_pretrained_gguf(
    str(temp_export_dir),
    tokenizer,
    quantization_method="q4_k_m",
)
```

`q4_k_m` es una cuantización de 4 bits que mantiene en más precisión las capas más sensibles. Es el punto medio habitual entre tamaño y calidad. Resultado en disco:

Modelo	GGUF
Qwen 2.5 3B (legacy y V3)	1,93 GB cada uno
Gemma 2 2B	1,71 GB

Curiosidad útil: el GGUF del legacy y el del V3 pesan **exactamente** lo mismo, al byte. La cuantización fija el tamaño; el contenido de los pesos no. No uses el tamaño para saber qué modelo tienes.

El Modelfile

Ollama necesita un `Modelfile` que diga qué GGUF cargar y cómo hablarle. El de Qwen legacy era así de corto:

```
FROM ./vertex-chess-qwen-q4_k_m.gguf
PARAMETER temperature 0.1
PARAMETER repeat_penalty 1.15
PARAMETER num_ctx 512
```

Sin `TEMPLATE` y sin `stop`. Ollama intenta deducir la plantilla de chat de los metadatos del GGUF, y muchas veces acierta. Pero "muchas veces" no es una base para un experimento. El de la V3 la deja escrita:

```
FROM ./vertex-chess-qwen-v3-q4_k_m.gguf
TEMPLATE """{{- range .Messages }}<|im_start|>{{ .Role }}
{{ .Content }}<|im_end|>
{{ end }}<|im_start|>assistant
"""
PARAMETER stop "<|im_end|>"
PARAMETER stop "<|endoftext|>"
PARAMETER temperature 0.1
PARAMETER repeat_penalty 1.15
PARAMETER num_ctx 1024
```

Tres cambios:

1. **La plantilla explícita** reproduce exactamente el formato ChatML con el que Qwen se entrenó: cada mensaje entre `<|im_start|>` y `<|im_end|>`, y el turno del asistente abierto al final.
2. **Los `stop`** cortan la generación en cuanto el modelo cierra su turno. Sin ellos, un modelo pequeño puede seguir escribiendo un segundo turno inventado.
3. **`num_ctx 1024`.** Con la lista completa de jugadas legales, algunos prompts del medio juego pasan de 512 tokens. Si el contexto se queda corto, Ollama recorta el prompt y el modelo deja de ver parte de lo que le mandamos, sin ningún aviso en la respuesta.

Se registra con:

```
# Desde E:\vertex-chess-arena\models\gguf
ollama create vertexqwen-v3 -f Modelfile.qwen-v3
```

Probar antes de jugar

Antes de poner un modelo en la arena, pregúntale a mano por una posición del dataset y mira la respuesta cruda. Treinta segundos. Si el legacy hubiera pasado por esa prueba el 17 de octubre, habríamos visto la prosa en lugar de `Best move:` y este libro sería otro.

Lo que te llevas de aquí

- Exporta a GGUF `q4_k_m` para servir modelos de 2–3B en 6 GB.

- Escribe la plantilla de chat y los `stop` en el Modelfile. No dependas de la detección.
- Calcula el contexto con el prompt más largo, no con el típico.
- El tamaño del archivo no identifica la versión del modelo.
- Una pregunta manual antes del primer partido vale más que cien partidas.

Lo que viene: en el capítulo 7 conocemos al árbitro, el programa que sostenía a los modelos… y que acabó jugando por ellos.

CAPÍTULO 7

El árbitro

Un modelo de 3B que elige jugadas por instinto pierde contra cualquiera que sepa contar material. La primera semana de partidas lo dejó claro: regalaba piezas, daba jaques sin sentido y, en finales ganados, paseaba la dama en círculos hasta tablas. La respuesta fue ponerle una niñera. La llamé **el árbitro**.

Qué hace el árbitro

El árbitro es una clase de Python, `ChessArbiter`, de casi mil líneas. Tiene dos trabajos que conviene separar desde ya, porque toda la historia del libro depende de esa separación:

1. **Llevar la partida:** comprobar que la jugada es legal, pasar el turno, detectar mate, tablas, repetición. Es trabajo de notario. Lo hace `python-chess` y no tiene discusión.
2. **Elegir candidatos:** antes de preguntarle al modelo, reducir la lista de jugadas a unas pocas "razonables". Esto ya no es notaría. Esto es **jugar**.

Cómo elige los candidatos

El método `_get_candidate_moves` sigue cuatro pasos:

1. **Si hay mate en una, solo ofrece esa.** El modelo no tiene nada que decidir.
2. **Busca con alfa-beta a profundidad 2**: mira cada jugada propia y la mejor respuesta del rival, y puntúa la posición resultante.
3. **Extiende con quiescencia**: en las posiciones con capturas o jaques pendientes sigue mirando hasta 4 medias jugadas más, para no evaluar una posición en mitad de un cambio de piezas.
4. **Se queda con hasta 4 jugadas** cuya puntuación esté a menos de 180 centipeones de la mejor.

La función de evaluación suma material, actividad de piezas y seguridad del rey, y añade reglas aprendidas a golpes en las partidas de prueba:

- **Filtro de intercambios (SEE):** penaliza dar un alfil o un caballo por un peón defendido.
- **Anti-jaques eternos:** a partir del segundo jaque seguido sin captura, cada jaque resta 280 puntos.
- **Mop-up (rematar finales ganados):** con ventaja de 350 o más y el rival casi sin material, premia encerrar al rey en una caja con la dama y acercar el rey propio, y mira el mate en 2.
- **Simplificar cuando vas ganando:** bonificación por cambiar piezas mayores.

El modelo dentro de una jaula

Piensa en lo que significa ese cuarto paso. El modelo ya no elige entre 30 o 40 jugadas legales. Elige entre **cuatro**, todas aprobadas por una búsqueda que no regala material y que nunca se pierde un mate en una. Para un jugador de ajedrez es como jugar con un entrenador que te tapa todas las jugadas malas y te deja escoger entre las buenas.

Y hay un detalle que ningún prompt dice: en la versión original, el prompt seguía llamando a esas cuatro jugadas `Legal moves (SAN)`. El modelo creía que tenía todo el tablero delante y en realidad le daban un menú.

Lo que el árbitro hacía cuando el modelo fallaba

Si el modelo contestaba algo que no se podía leer, el árbitro necesitaba una jugada de todas formas. La versión original lo resolvía así:

```
# La respuesta no se pudo validar: devolvemos un movimiento legal de
↪  respaldo,
# pero NO fingimos que Gemma acertó.
return candidate_moves[0], False
```

`candidate_moves[0]` es la primera de la lista. Y la lista viene ordenada por la búsqueda. Es decir: **cuando el modelo fallaba, jugaba la mejor jugada del motor.**

El comentario es honesto: marca la jugada como no acertada. Pero en el tablero no queda ninguna marca. La jugada del motor cuenta igual para ganar.

Guarda esta línea. Es la protagonista del capítulo 8.

Lo que te llevas de aquí

- Separa en tu código lo que es **verificar** (reglas, legalidad) de lo que es **decidir** (filtrar, ordenar, elegir). Lo primero no cambia el resultado; lo segundo sí.
- Un filtro de candidatos es un jugador. Si lo pones delante de tu modelo, mides a los dos.
- Nombra las cosas por lo que son. Una lista filtrada no es `Legal moves`.
- Revisa qué hace tu sistema **cuando el modelo falla.** Ahí suelen esconderse las ayudas.

Lo que viene: en el capítulo 8, la auditoría. Por qué Qwen siempre ganaba.

CAPÍTULO 8

“Qwen siempre gana”

La primera versión de la bitácora tenía una conclusión elegante: *“Qwen demuestra mayor competencia nativa en el procesamiento de cadenas FEN densas”*. Suena a paper. Era falso. Este capítulo cuenta cómo se desmonta una conclusión bonita: leyendo el código y midiendo.

El duelo con lupa

Primero, un duelo de dos partidas con colores cambiados y un contador por jugada. Para cada jugada se anotó si salió **limpia** (el modelo nombró una opción válida), si fue **respaldo** (el árbitro tuvo que elegir) y si coincidió con la **primera** jugada de la lista del motor (la “top-1”).

	Limpias	Respaldo	Top-1 del motor
Partida 1, Qwen (blancas)	31	**14**	25 de 45
Partida 1, Gemma (negras)	44	**0**	11 de 44
Partida 2, Gemma (blancas)	77	1	19 de 78
Partida 2, Qwen (negras)	62	**16**	25 de 78

Qwen ganó las dos. Y la tabla dice por qué: en la partida 1, **Gemma nunca falló y Qwen falló 14 veces**. Cada fallo de Qwen era una jugada del motor. Qwen jugó la top-1 del motor en más de la mitad de sus jugadas; Gemma, en una de cada cuatro.

El que más fallaba era el que más ayuda recibía.

Los siete problemas

La revisión de código encontró siete cosas. Ninguna era grave sola. Juntas fabricaban el resultado.

1. **El respaldo premiaba fallar.** `candidate_moves[0]` era la mejor jugada del motor (capítulo 7).

2. **El Qwen de la arena era el de 80 pasos.** El exportador leía otra carpeta (capítulo 5). En 40 posiciones del propio dataset, ese modelo dio **0 de 40** respuestas en formato.
3. **El parser era indulgente.** Si no encontraba `Best move:`, recorría la prosa buscando *cualquier cosa* con forma de jugada y la aceptaba si estaba entre las opciones. Una frase como *"the bishop could go to f7"* podía convertirse en una jugada "limpia". Y usaba `board.parse_san()`, que acepta `Bxf7` aunque en f7 no haya nada que capturar. Por eso el legacy, que nunca escribía el formato, tenía 31 jugadas "limpias".
4. **El prompt de la arena no era el del entrenamiento.** Otro mensaje de sistema, una línea extra `Side to move:` y, en lugar de las jugadas legales, los cuatro candidatos del árbitro con la etiqueta `Legal moves`.
5. **Código muerto.** `_evaluate_move`, con toda la lógica de rematar finales (la caja de la dama, el mate en 2), estaba escrita pero **no se llamaba desde ningún sitio**.
6. **Colores fijos.** El script de partidas ponía siempre al mismo modelo con blancas.
7. **Repeticiones sin reclamar** y temperatura de 0,05: las partidas tendían a repetirse jugada por jugada.

Por qué nadie lo vio

Porque cada pieza tenía una buena razón. El respaldo con la mejor jugada "evita que la partida se rompa". El parser indulgente "rescata respuestas casi correctas". El filtro de candidatos "evita que el modelo regale piezas". La temperatura baja "da respuestas estables". Cada decisión, sola, mejoraba la experiencia de ver partidas. Juntas convirtieron el experimento en una medición del árbitro.

Y había una pista que lo gritaba desde el principio: **el modelo que menos había entrenado ganaba**. En vez de sospechar, la bitácora inventó una teoría para explicarlo.

Lo que te llevas de aquí

- Cuando el resultado contradice el esfuerzo de entrenamiento, **sospecha de la arena antes que de los modelos**.
- Cuenta por separado las jugadas del modelo y las del sistema. Si no las cuentas, no existen.
- Un parser indulgente convierte fallos en aciertos. Acepta solo el formato exacto.
- El prompt de evaluación debe ser **idéntico** al de entrenamiento.

- Busca código muerto: una función que nadie llama es una promesa que no se cumple.
- Desconfía de tus propias conclusiones elegantes. Escríbelas, y luego intenta romperlas.

Lo que viene: en el capítulo 9 rehacemos el duelo para que sea justo.

CAPÍTULO 9

Un duelo justo

Sabiendo lo que estaba mal, el plan era sencillo de decir: que los dos modelos lleguen a la arena en igualdad de condiciones, y que la arena no juegue por ninguno.

Paso 1: el entrenamiento espejo

Qwen V3 se entrenó con **la misma receta que Gemma**, línea por línea (capítulo 4): QLoRA `r=8`, `alpha=16`, `lr 5e-5` con calentamiento de 15 pasos y coseno, lote efectivo 8, 384 tokens, `fp16`, atención `eager`, **625 pasos = una época**, y la pérdida solo sobre la respuesta.

Para enmascarar el prompt en Qwen hubo que pelear con un detalle de la librería: con `tokenize=True`, `apply_chat_template` devolvía un objeto `BatchEncoding` en lugar de una lista de ids. La solución robusta es pedir el texto y codificarlo aparte:

```
prompt_text = tokenizer.apply_chat_template(
    messages[:-1], tokenize=False, add_generation_prompt=True)
prompt_ids = tokenizer.encode(prompt_text, add_special_tokens=False)
answer_ids = tokenizer.encode(answer + "<|im_end|>",
↪  add_special_tokens=False)
labels = [-100] * len(prompt_ids) + answer_ids
```

Fíjate en el `<|im_end|>` al final de la respuesta: el modelo también aprende a **cerrar su turno**, que es lo que luego detiene la generación en Ollama.

Cifras del entrenamiento (de `train_log.json`):

- 625 pasos en **13 h 42 min**: unos 79 s por paso. Subió a unos 145 s por paso en los ratos en que se jugaban partidas con Ollama en la misma GPU.
- Pérdida: de **4,23** en el paso 1 a **0,23** en el paso 625. Media de todo el entrenamiento, 0,36; media de los últimos 50 pasos, 0,20.

- Los pasos 1, 2 y 4 dieron gradiente `NaN` y se saltaron. Es normal en `fp16`: el escalador de pérdida empieza alto y lo va bajando hasta encontrar un valor que no desborde.

Paso 2: un árbitro honesto

El árbitro V3 cambia cinco cosas:

1. **Respaldo aleatorio.** Si el modelo falla, la jugada se elige **al azar** entre sus opciones. Fallar ya no se premia.
2. **Parser estricto.** Solo cuenta `Best move: <SAN exacto>, UCI: <uci exacto>` o una respuesta que sea exactamente una jugada. Nada de rescatar prosa.
3. **El prompt del dataset**, carácter por carácter.
4. **Dos modos**, y el modo forma parte del resultado:
   - **`pure`**: el modelo ve **todas** las jugadas legales. El árbitro solo hace de notario.
   - **`arbiter`**: el modelo elige entre los candidatos filtrados, como antes.
5. **Código vivo y anti-bucles.** `_evaluate_move` por fin ordena los candidatos en finales ganados, y se descartan las jugadas que repiten una posición reciente.

   Además, las partidas alternan colores y reclaman la triple repetición.

Paso 3: medir al modelo sin tablero

Antes de jugar, un sondeo: las mismas 40 posiciones del dataset para los tres modelos.

Modelo	Formato `Best move:`	Acierta la jugada del dataset
Qwen legacy	0 / 40	0 / 40
Qwen V3	**39 / 40**	4 / 40
Gemma	**40 / 40**	**9 / 40**

El reentrenamiento funcionó: Qwen pasó de no saber contestar a contestar casi siempre. Pero Gemma acierta más del doble. Con datos de entrenamiento, ojo: estas posiciones ya las vieron.

Un detalle curioso del sondeo: a veces Qwen V3 da bien el SAN y mal el UCI, como en `Best move: Bb7 (UCI: d8b7)`, que no corresponde a la posición. El parser V3 lee primero el SAN, así que no le afecta. Pero es

otra prueba de que el modelo aprendió la **forma** de la respuesta mejor que su contenido.

Paso 4: jugar

Modo puro, 10 partidas: Gemma 5,5 – Qwen V3 4,5. Jugadas válidas: Gemma 99,2 %, Qwen 95,5 %. Solo una partida decisiva; las otras nueve, tablas por triple repetición (capítulo 11).

Modo árbitro, 4 partidas: Qwen V3 2,5 – Gemma 1,5. Jugadas válidas: Qwen 94,1 %, Gemma 99,6 %. Y un dato que lo dice todo: en las tres partidas decisivas **ganaron las blancas**. Con el árbitro, lo que más pesa es quién mueve primero, no qué modelo hay detrás.

Sin árbitro gana Gemma, el que mejor contesta. Con árbitro, el resultado es una moneda al aire. La diferencia entre los modelos solo se ve cuando quitas la ayuda.

Lo que te llevas de aquí

- Para comparar dos modelos, **la misma receta** de entrenamiento. Si no, comparas recetas.
- Con prompts largos y respuestas cortas, enmascara el prompt y entrena también el token de fin de turno.
- El respaldo de un sistema debe ser **neutral**. El azar es neutral; el motor no.
- Mide con y sin ayuda, y publica las dos cifras.
- Un sondeo fuera del tablero (formato y acierto por posición) separa "sabe contestar" de "sabe jugar".

Lo que viene: en el capítulo 10, la arena profesional: relojes, torneos y todo lo que hace falta para que otros puedan jugar.

CAPÍTULO 10

La arena profesional

La primera interfaz era una página HTML con un tablero y un botón. Servía para ver partidas, no para un torneo. Para que otras personas pudieran jugar, y para que los resultados tuvieran la seriedad de un torneo de verdad, hacía falta una aplicación con reloj, calendario de rondas, clasificación y partidas descargables.

La pila

Pieza	Elección	Por qué
Frontend	**Angular 21.2**, componentes *standalone*, sin zone.js, con *signals*	Moderno y predecible
Estilos	Tailwind 4	Rápido para un diseño oscuro y *responsive*
Tablero	`cm-chessboard` (MIT)	Ligero, accesible, sin dependencias pesadas
Piezas	Cburnett (CC BY-SA 3.0)	Las clásicas, con licencia que permite uso comercial
Reglas en el navegador	`chess.js`	Validar jugadas humanas y generar PGN
Backend	FastAPI, API V3	Habla con Ollama y guarda partidas y torneos

Dos decisiones de esa tabla vienen con historia:

- **¿Por qué Angular 21 y no 22?** Angular 22 pide una versión de Node más nueva que la que había en la máquina (24.13). Antes de elegir

versión, se comprobó qué Node y qué npm había instalados. Actualizar Node por una versión mayor del framework no aportaba nada al proyecto.
- **¿Por qué Cburnett y no otras piezas más bonitas?** Algunas colecciones populares tienen licencias **no comerciales**. Si el libro y la app se venden o se publican, eso importa.

El backend: una API pequeña y honesta

- `POST /api/ai-move` pide una jugada a un modelo en el modo indicado (`pure`, `arbiter` o `engine`) y devuelve, además de la jugada, si salió limpia o fue respaldo y cuánto tardó. Si Ollama no responde, devuelve **503** con un error claro; nunca inventa una jugada.
- `POST /api/games` guarda cada partida terminada en un JSONL y la añade al PGN histórico.
- `GET/PUT /api/tournaments/{id}` guarda el estado del torneo después de cada partida.

Esta última decisión salvó la Copa del capítulo 11: cuando Ollama se cayó en la ronda 6, no se perdió nada.

El reloj: la lección de chess.com

Primera versión: reloj **Fischer**, que suma un incremento fijo después de cada jugada. Es el de casi todos los torneos online. Con humanos funciona. Con una IA que contesta en un segundo, no: con 3+2, el modelo gasta un segundo y gana dos. **El reloj subía.**

Mi primera reacción fue explicar que eso es correcto según el reglamento. La respuesta de Denis fue mejor: *"como en los campeonatos normales: el reloj baja, y si llega a cero pierdes"*. Tenía razón en lo que importa: la experiencia que se espera de un reloj de ajedrez es que baje.

La solución fue el reloj **Bronstein**: después de cada jugada se devuelve lo que pensaste, **hasta** el valor del incremento. Si piensas 0,6 s con +1, recuperas 0,6 s. Nunca acabas con más tiempo del que tenías al empezar la jugada:

```
rem[side] += Math.min(thinkMs, this.control.incMs);
```

El ritmo por defecto pasó a 3+0. Y cuando el reloj llega a cero aparece un cartel sobre el tablero, como en chess.com: quién gana, por qué, y los botones de partida nueva y revancha. Con una excepción del regla-

mento FIDE (artículo 6.9): si al que le queda tiempo no tiene material para dar mate, la caída de bandera es **tablas**, no derrota.

Dos fallos que no eran del código

- **"El reloj no camina, se queda en 3:00."** No era Angular. Era el **traductor automático de Chrome**: al traducir la página al español, reemplazaba los nodos de texto y Angular seguía actualizando nodos que ya no estaban en pantalla. Solución: `translate="no"` en el HTML y la meta `notranslate`.
- **"No se pudo crear el torneo."** El torneo sí se creaba. Lo que fallaba era la navegación: tras una actualización de la app, el navegador pedía un fragmento de JavaScript que ya no existía. Solución: separar *crear* de *navegar*, y recargar la página si falla la carga de un fragmento.

Torneos

- **Formatos:** match a N partidas con colores alternos, todos contra todos y doble vuelta.
- **Emparejamientos:** tablas de **Berger**, el método clásico para todos contra todos, con un ajuste para equilibrar los colores de cada jugador.
- **Desempate: Sonneborn-Berger**, la suma de los puntos de los rivales a los que ganaste más la mitad de los de aquellos con los que hiciste tablas. Premia los resultados contra los fuertes.
- **Tabla cruzada** y **PGN del torneo** con los tiempos de reloj de cada jugada (`%clk`, `%emt`) y la cabecera `ClockMode "Bronstein"`.

Móvil

En un teléfono de 390 px, la pantalla de partida medía 557 px de ancho. Los culpables fueron tres clásicos de CSS Grid y Flexbox: hijos sin `min-width: 0`, un reloj demasiado grande y columnas de tabla que no hacían falta en pantalla pequeña. Tras el arreglo, la app cabe entera en un móvil, que era el objetivo: que cualquiera pudiera jugar desde el teléfono.

Lo que te llevas de aquí

- Comprueba las versiones instaladas antes de elegir las del framework.
- Revisa la licencia de cada recurso gráfico si vas a publicar.

- Un reloj correcto según el reglamento puede ser incorrecto para el usuario. Bronstein resuelve el caso de jugadores que responden en un segundo.
- Si algo "no se actualiza" en pantalla, sospecha también de las extensiones del navegador.
- Guarda el estado del torneo después de cada partida.
- Prueba en un móvil real (o en su ancho) desde el principio.

Lo que viene: en el capítulo 11, la verdad del tablero.

CAPÍTULO 11

La verdad del tablero

Primero, el duelo puro

Con el árbitro honesto y Qwen reentrenado, el primer examen fue un match de 10 partidas en modo puro, colores alternos: **Gemma 5,5 – Qwen V3 4,5**. Pero la cifra engaña. Solo hubo **una partida decisiva** (Gemma dio mate con negras); las otras nueve fueron tablas por triple repetición, y las partidas 1, 5 y 9 fueron idénticas jugada por jugada. Gemma contestó en formato válido el 99,2 % de las veces; Qwen V3, el 95,5 %.

Dos modelos que repiten la misma partida no están compitiendo: están recitando. Faltaba ver qué pasaba en un torneo completo.

La Copa Vertex Verano 2026

Después de todos los arreglos quedaba una prueba que ningún gráfico sustituye: un torneo de verdad, con reloj, todos contra todos, y con el árbitro a la vista en vez de escondido.

El formato

- **7 jugadores**, todos contra todos a una vuelta: **21 partidas** en 7 rondas.
- **Ritmo 2+1 Bronstein:** 2 minutos por bando; después de cada jugada se devuelve lo pensado, hasta 1 segundo. El reloj nunca sube.
- **Límite de 150 jugadas** (300 medias jugadas) y tablas por triple repetición.

Los jugadores no eran siete modelos, sino tres modelos en dos modos cada uno, más el motor:

Jugador	Qué es
Qwen V3 puro	El Qwen reentrenado (cap. 9) eligiendo entre **todas** las jugadas legales

Jugador	Qué es
Qwen V3 + árbitro	El mismo modelo, eligiendo entre los **candidatos** que filtra el árbitro
Gemma puro	Gemma 2B entre todas las jugadas legales
Gemma + árbitro	Gemma 2B entre los candidatos del árbitro
Qwen legacy puro	El adaptador viejo de 80 pasos (cap. 5)
Qwen legacy + árbitro	El adaptador viejo con candidatos del árbitro
Motor del árbitro	La búsqueda alfa-beta del árbitro jugando sola, sin modelo

Recuerda la regla honesta del capítulo 9: si un modelo contesta algo inválido, la jugada se elige **al azar** entre sus opciones. Sin ayuda del motor.

La clasificación

#	Jugador	Puntos	Jugadas válidas
1	Motor del árbitro	**6 / 6**	—
2	Qwen V3 + árbitro	4	95,3 %
2	Gemma + árbitro	4	99,5 %
2	Qwen legacy + árbitro	4	**0,5 %**
5	Gemma puro	1	97,8 %
5	Qwen V3 puro	1	97,8 %
5	Qwen legacy puro	1	3,2 %

El torneo duró 41 minutos y medio. Hubo 18 mates y 3 tablas por triple repetición. Las blancas ganaron 11 partidas y las negras 7. Nadie perdió por tiempo.

A mitad de la ronda 6, Ollama se cayó (la conexión se cortó en medio de una jugada de Qwen). Se levantó a mano, se pulsó *Reintentar* y el torneo siguió sin perder nada: cada partida terminada ya estaba guardada en el backend.

Tres lecturas

1. Árbitro contra puro: 9 a 0

Hubo nueve partidas entre un jugador con árbitro y uno sin árbitro. **Las ganó todas el que tenía árbitro.** No importó qué modelo había detrás de cada lado: Gemma con árbitro le ganó a Gemma pura, Qwen V3 con árbitro le ganó a Qwen V3 puro, y el legacy con árbitro le ganó a todos los puros.

2. El jugador que no jugó

El dato más importante del libro está en una fila de esa tabla. **Qwen legacy + árbitro** hizo 193 jugadas y solo **una** salió de su respuesta. Las otras 192 fueron azar entre los candidatos del árbitro. Y aun así sacó **4 puntos**, los mismos que los dos modelos que sí aprendieron el formato. Le dio mate a Gemma con árbitro en 79 medias jugadas.

Dicho en claro: con el árbitro filtrando candidatos, **elegir al azar rinde igual que elegir con un modelo afinado**. Los puntos los pone el filtro. Es exactamente lo que pasaba al principio del experimento, cuando "Qwen siempre ganaba".

3. Sin árbitro, nadie sabe ganar

Las tres partidas entre jugadores puros terminaron en **tablas por triple repetición**. Gemma pura y Qwen V3 puro contestan en el formato correcto casi siempre (97,8 % los dos), pero no tienen plan: mueven una torre de ida y vuelta mientras el rival hace lo mismo. Saber **decir** una jugada legal no es saber **jugar**.

Y entre los tres con árbitro salió un ciclo perfecto, como piedra, papel o tijera: el legacy le ganó a Gemma, Gemma a Qwen V3 y Qwen V3 al legacy. Es lo que esperas cuando las diferencias entre jugadores son ruido.

Lo que dice y lo que no dice

Este torneo **no** dice que Qwen y Gemma sean iguales. En el sondeo de los capítulo 9, sobre las mismas 40 posiciones, Gemma acertó la mejor jugada 9 veces y Qwen V3 4. Lo que el torneo dice es más incómodo: **una arena con árbitro no puede medir esa diferencia**, porque el árbitro la tapa. Para comparar modelos hay que mirar el modo puro y las métricas por posición, no la clasificación con ayuda.

Tampoco es una muestra grande: una partida por emparejamiento. Para conclusiones finas haría falta repetir con colores cambiados y

muchas más partidas. Pero el 9 a 0 y la fila del legacy no necesitan estadística fina para entenderse.

Lo que te llevas de aquí

- Si tu sistema tiene un "ayudante" (un filtro, un *reranker*, un *fallback*), mide el sistema **con y sin** él, y añade un jugador que no haga nada para ver cuánto aporta solo.
- Un jugador al azar dentro de tu arena es la mejor línea base que existe. Si empata con tu modelo, el que juega es la arena.
- Formato correcto no es competencia. Mide las dos cosas por separado.
- Guarda cada partida al terminarla; los cortes van a pasar.

Lo que viene: en el capítulo 12, lo que aprendí y lo que haría distinto.

CAPÍTULO 12

Lecciones para quien afina modelos

Empecé queriendo saber qué modelo pequeño aprende mejor ajedrez. Terminé aprendiendo algo más útil: **casi siempre que medimos un modelo, medimos un sistema**. El modelo es una pieza; alrededor hay un prompt, un parser, un respaldo, un filtro, una plantilla, un exportador. Cualquiera de esas piezas puede ganar la partida por él.

Este capítulo destila lo aprendido en reglas que puedes aplicar a cualquier proyecto de *fine-tuning*, juegue o no al ajedrez.

1. Mide al modelo, no al sistema

Pon siempre, junto a tu sistema completo, **la misma medición sin ayudas**. En este libro, el modo `pure` frente al modo `arbiter`. La Copa del capítulo 11 lo dejó claro: con árbitro contra sin árbitro, 9 a 0, fuera cual fuera el modelo.

2. Añade un jugador que no haga nada

El jugador más valioso del torneo fue el que no jugaba: **Qwen legacy + árbitro**, 1 jugada propia de 193, empatado con los dos modelos entrenados. Un "modelo" que responde al azar dentro de tu sistema es la línea base más honesta que existe. Si tu modelo no le gana, tu modelo no aporta.

Y la anécdota que lo resume: en la ronda 7 ese mismo jugador ganó en **cinco jugadas** (`1.h3 e5 2.g3 c5 3.c3 f5 4.f3 h5 5.g4 Qh4#`). Las cinco fueron respaldo. La última no la eligió nadie: el árbitro, al ver un mate en una, ofrece solo esa jugada.

3. Los fallos son datos

Cuenta por separado lo que hace el modelo y lo que hace el sistema cuando el modelo falla. Sin el contador de "respaldo" y "top-1 del motor" del capítulo 8, la ayuda era invisible.

4. El respaldo debe ser neutral

Si tu sistema tiene un *fallback*, que no sea mejor que el modelo. En ajedrez, el azar. En un clasificador, la clase más frecuente. En un generador, un "no sé". Nunca la respuesta de otro sistema más fuerte.

5. Parser estricto

Acepta solo el formato exacto con el que entrenaste. Un parser que "rescata" respuestas convierte errores en aciertos y te hace creer que el modelo aprendió el formato.

6. Entrena y evalúa con el mismo prompt

Carácter por carácter. Y si cambias algo del prompt en la evaluación (como los candidatos del árbitro), llámalo por su nombre y mídelo aparte.

7. Enmascara el prompt

Con prompts largos y respuestas cortas, la pérdida solo sobre la respuesta (`-100` en el resto). Y entrena también el token de fin de turno.

8. Compara recetas iguales

Mismos datos, mismos hiperparámetros, mismas épocas. Si un modelo entrena 80 pasos y el otro 625, no comparas modelos: comparas esfuerzos.

9. Una sola fuente de verdad para las rutas

El entrenador y el exportador deben leer la ruta del adaptador del mismo sitio. Imprime la fecha y el log del adaptador que exportas.

10. Audita lo que no se te ocurrió auditar

El recorte silencioso a 45 jugadas del dataset (capítulo 3), el código muerto del árbitro (capítulo 8), el traductor del navegador que congelaba el reloj (capítulo 10). Los errores que importan suelen estar donde nadie mira.

11. Formato no es competencia

Gemma pura y Qwen V3 puro contestan en formato válido el 97,8 % de las veces y no saben ganar: sus partidas entre ellos acaban en triple repetición. Mide por separado "sabe contestar" y "sabe resolver".

12. Sospecha de tus conclusiones elegantes

"Qwen demuestra mayor competencia nativa en el procesamiento de cadenas FEN densas". La escribí yo. Si un resultado te sorprende y la explicación te halaga, es el momento de leer el código otra vez.

Lo que haría distinto

- **Más datos y menos atajos.** 5.000 posiciones bastan para aprender el formato, no para aprender a jugar. El siguiente paso serían decenas de miles, con la lista de jugadas completa y sin recortes.
- **Evaluar fuera del entrenamiento.** El sondeo usó posiciones del dataset. Un conjunto aparte, nunca visto, diría cuánto generalizan.
- **Muestras más grandes.** Una partida por emparejamiento sirve para ver tendencias gruesas, no diferencias finas. Con colores cambiados y muchas más partidas, las cifras tendrían intervalos de confianza.
- **Registrar todo desde el día uno**: cada adaptador con su `train_log`, cada exportación con la ruta y la fecha de su origen.

Lo que te llevas de aquí

- Un modelo afinado vive dentro de un sistema. Mide los dos por separado.
- Las ayudas invisibles son el error más común y el más caro.
- Lo que no cuentas, no existe.

Lo que viene: en el capítulo 13, cómo publicar todo esto sin gastar un dólar más.

CAPÍTULO 13

Publicarlo sin gastar

Un experimento que solo corre en mi portátil no lo puede comprobar nadie. El último paso es publicarlo: los modelos en Hugging Face, el backend en un Space con GPU compartida y la app en Netlify, para que cualquiera pueda jugar desde el móvil. Con una condición: **no añadir gastos nuevos**. Ya pago Hugging Face PRO y Netlify; lo que no quepa ahí, no se despliega.

La arquitectura

```
Móvil / navegador
      |
      ▼
Netlify  ──  la app Angular (archivos estáticos)
      |  HTTPS
      ▼
Hugging Face Space (ZeroGPU)  ──  árbitro + modelos
      ▲
      |  descarga al arrancar
Hugging Face Hub  ──  adaptadores LoRA de Qwen V3 y Gemma
```

Los modelos en el Hub

Este paso ya está hecho. Los dos modelos están publicados como GGUF `q4_k_m`, listos para Ollama:

- **`Denisijcu/vertexqwen`**: Qwen V3 y, para quien quiera reproducir el libro, el legacy.
- **`Denisijcu/vertexgemma`**: Gemma 2 2B.

La primera versión de las fichas tenía tres errores que este libro habría señalado en cualquier otro proyecto: una licencia equivocada (Apache 2.0 para un derivado de Qwen 2.5 3B, que no lo es), un número de pasos inventado y promesas de "nivel gran maestro". Las fichas corregidas dicen qué es cada archivo, el prompt exacto, la receta del capítulo 9, las cifras del sondeo y del torneo, **las limitaciones** (aprende

el formato, no juega bien sin ayuda) y la licencia correcta de cada uno, con su archivo de aviso.

Para el Space hará falta además subir los **adaptadores LoRA** (unos 60 MB el de Qwen, unos 42 MB el de Gemma): ZeroGPU trabaja con PyTorch, no con GGUF.

El backend en ZeroGPU

ZeroGPU reparte GPUs de un grupo compartido: el Space no tiene GPU fija, la pide cuando una función la necesita y la suelta al terminar. Tres datos de la documentación cambian el diseño:

1. **Solo funciona con el SDK de Gradio.** El backend no puede ser el FastAPI tal cual. Hay que envolverlo en una app Gradio y exponer las funciones como API.
2. **Solo PyTorch.** Ollama no corre ahí. Los modelos se cargan con `transformers` + `peft` (base + adaptador), en la GPU, al arrancar el Space.
3. **La función que usa la GPU se marca con `@spaces.GPU`:**

```
import spaces

@spaces.GPU(duration=20)
def ai_move(model_id: str, fen: str, mode: str) -> dict:
    ...  # arbitro.get_options + generate + parse
```

El árbitro (búsqueda alfa-beta, parser, reglas) no necesita GPU y corre fuera de esa función. El modo `engine` no gasta GPU.

La cuota

La cuota de ZeroGPU es **diaria y la gasta quien usa el Space**, no quien lo publica:

Visitante	Cuota diaria
Sin cuenta	2 minutos
Cuenta gratis	5 minutos
Cuenta PRO	40 minutos

Cada jugada de un modelo de 2–3B tarda en torno a un segundo de GPU, más el tiempo de pedir la GPU. Una partida de 50 jugadas del

modelo cabe en la cuota de un visitante sin cuenta, pero no muchas más. Por eso la app debe:

- Ofrecer primero **humano contra modelo**, que gasta la mitad que modelo contra modelo.
- Avisar cuando la cuota se acaba, y ofrecer seguir contra el motor (sin GPU).
- Dejar los torneos completos de IA contra IA como **repeticiones** de partidas ya jugadas (el historial y los PGN), no en directo.

La app en Netlify

La app Angular se compila (`npm run build`) y se publica como archivos estáticos. Hay que cambiar dos cosas:

- La URL de la API apunta al Space en lugar de a `localhost:8000`.
- Los torneos y el historial dejan de guardarse en el disco del backend. Un Space gratuito pierde los archivos al reiniciarse, así que los torneos de cada visitante se guardan en su navegador y los torneos "oficiales" (como la Copa del capítulo 11) se publican como datos estáticos dentro de la app.

Las licencias

Antes de publicar, la parte que no se puede saltar:

- **Qwen 2.5 3B Instruct** está bajo la **Qwen Research License**: uso **no comercial** (investigación o evaluación). Un demo gratuito y público encaja; cobrar por el modelo o meterlo en un producto de pago requiere pedir licencia a Alibaba Cloud. Al distribuir el adaptador hay que incluir copia de la licencia, el aviso *"Qwen is licensed under the Qwen RESEARCH LICENSE AGREEMENT, Copyright (c) Alibaba Cloud. All Rights Reserved."* y mostrar **"Built with Qwen"** en la documentación.
- **Gemma 2 2B** está bajo los **Gemma Terms of Use**, que permiten uso comercial respetando su política de uso prohibido. Al redistribuir hay que incluir sus términos y el aviso correspondiente.
- **Piezas Cburnett** (CC BY-SA 3.0) y **cm-chessboard** (MIT): ya están acreditadas en el pie de la app.

El libro se puede vender: describe el experimento, no distribuye los pesos.

La lista de comprobación

1. Subir los adaptadores LoRA junto a los GGUF ya publicados.
2. Crear el Space con Gradio, `transformers`, `peft` y el árbitro; probar `ai_move` con las 40 posiciones del sondeo y comparar con las cifras del capítulo 9.
3. Cambiar la URL de la API en la app, compilar y publicar en Netlify.
4. Probar desde un móvil, sin cuenta de Hugging Face, una partida completa.
5. Añadir el enlace de la app a las fichas de los modelos y a la descripción del video.

Lo que te llevas de aquí

- Lee las restricciones de la plataforma **antes** de diseñar el despliegue: ZeroGPU impone Gradio y PyTorch.
- Una GPU compartida con cuota por visitante obliga a diseñar la experiencia alrededor del gasto.
- Sube adaptadores, no modelos completos, cuando el modelo base ya está en el Hub.
- Las licencias de los modelos base viajan con tus adaptadores. Léelas enteras.

Conclusión

Afiné dos modelos para jugar ajedrez y descubrí que el árbitro jugaba por ellos. Lo dice el título y lo confirmó la Copa: con árbitro, un modelo que eligió una jugada de 193 sacó los mismos puntos que dos modelos entrenados durante horas.

Pero el experimento no fracasó. Respondió a su pregunta, solo que no a la que yo creía estar haciendo. ¿Qué modelo pequeño aprende mejor ajedrez? Con la misma receta y sin ayuda, Gemma 2 2B contesta mejor y acierta más que Qwen 2.5 3B. Ninguno de los dos juega bien solo. Y para saberlo hizo falta desmontar, pieza por pieza, todo lo que yo había puesto alrededor para que las partidas "se vieran bien".

Si te quedas con una sola idea, que sea esta: **cuando tu modelo gane, pregunta quién más estaba jugando.**

— Denis Sánchez Leyva
Miami, 2026

APÉNDICE A

Comandos, con su directorio

> *Todos los comandos son de PowerShell en Windows y usan el entorno virtual del proyecto. Antes de cada bloque se indica **desde qué carpeta** se ejecuta.*

Entrenar

Desde `E:\vertex-chess-arena`:

```
venv\Scripts\python.exe scripts\01_build_chess_dataset.py
venv\Scripts\python.exe scripts\01b_audit_chess_dataset.py
venv\Scripts\python.exe scripts\02_train_qwen_chess.py
venv\Scripts\python.exe scripts\03_train_gemma_chess.py
```

El de Qwen V3 guarda un *checkpoint* cada 125 pasos: si se interrumpe, el mismo comando lo reanuda. El de Gemma no guarda puntos intermedios (capítulo 4).

Exportar y registrar en Ollama

Desde `E:\vertex-chess-arena`:

```
venv\Scripts\python.exe scripts\05_export_both_gguf.py
```

Desde `E:\vertex-chess-arena\models\gguf`:

```
ollama create vertexqwen-v3 -f Modelfile.qwen-v3
ollama create vertexgemma -f Modelfile.gemma
ollama list
```

Si Ollama no responde (en el navegador, `http://127.0.0.1:11434` debe decir *"Ollama is running"*), ciérralo desde la bandeja del sistema y, desde cualquier carpeta:

```
ollama serve
```

Tests y diagnóstico

Desde `E:\vertex-chess-arena\arena`:

```
..\venv\Scripts\python.exe -m unittest -v test_arbiter_v3.py
..\venv\Scripts\python.exe -m pytest tests -q
..\venv\Scripts\python.exe ..\diagnostics\probe_bias.py 40 vertexqwen-v3
↪  vertexgemma vertexqwen
```

Partidas por consola

Desde `E:\vertex-chess-arena\arena`:

```
..\venv\Scripts\python.exe match_runner.py --games 10 --mode pure
..\venv\Scripts\python.exe match_runner.py --games 4 --mode arbiter
```

La app de torneos

Desde `E:\vertex-chess-arena\frontend`:

```
npm ci
npm run build
npm test
```

Desde `E:\vertex-chess-arena\arena`:

```
..\venv\Scripts\python.exe -m uvicorn api_server:app --host 127.0.0.1
↪  --port 8000
```

Abre `http://127.0.0.1:8000`. La página antigua sigue en `/legacy`.

APÉNDICE B

Hiperparámetros

Recetas finales (las del duelo justo)

Parámetro	Gemma 2 2B	Qwen 2.5 3B V3
Modelo base	`unsloth/gemma-2-2b-it-bnb-4bit`	`unsloth/Qwen2.5-3B-Instruct-bnb-4bit`
LoRA `r` / `alpha` / *dropout*	8 / 16 / 0,0	8 / 16 / 0,0
Módulos LoRA	`q,k,v,o,gate,up,down_proj`	`q,k,v,o,gate,up,down_proj`
Tasa de aprendizaje	5e-5	5e-5
Calentamiento / programa	15 pasos / coseno	15 pasos / coseno
Lote por dispositivo × acumulación	1 × 8	1 × 8
Épocas / pasos	1 / 625	1 / 625
Longitud máxima	384	384
Precisión / atención	`fp16` / `eager`	`fp16` / `eager`
Optimizador	`adamw_torch`	`adamw_torch`
Semilla	42	42
Pérdida	solo respuesta	solo respuesta + `<|im_end|>`
Adaptador	~42 MB	~60 MB

Duración en la GTX 1660 Ti: Gemma, unas 11 h; Qwen V3, 13 h 42 min (pérdida 4,23 a 0,23, media 0,36).

Las versiones anteriores de Qwen

	Legacy (el que jugó)	V2 (nunca exportado)
Carpeta	`models\qwen_chess_adapter`	`models\adapters\qwen_chess_lora`
LoRA `r` / `alpha`	8 / 16	16 / 32
Pasos	80	250 (0,4 épocas)
Tasa de aprendizaje	—	1,5e-4
Pérdida	sobre todo el texto	sobre todo el texto (2,33 a 0,55)

Inferencia (Ollama)

Parámetro	Valor
Cuantización	`q4_k_m`
Contexto	1024
Temperatura (Modelfile)	0,1
`repeat_penalty`	1,15
`stop` (Qwen)	`<|im_end|>`, `<|endoftext|>`

APÉNDICE C

Partidas clave

> *Sin comentarios de reloj. Los PGN completos, con `%clk` y `%emt`, están en logs/matches_history.pgn y en el PGN del torneo 4bwm13pspa.*

1. La partida que inició la sospecha

Duelo con el árbitro original, antes de la auditoría. Qwen legacy (blancas) contra Gemma (negras). Qwen: 31 jugadas "limpias", 14 de respaldo, 25 coincidencias con la mejor jugada del motor. Gemma: 44 limpias, 0 de respaldo, 11 coincidencias.

```
1. e4 e5 2. Bc4 Nc6 3. f3 g5 4. d3 h5 5. Bd5 b5 6. c4 Bb4+ 7. Bd2 Nge7
8. Bxb4 Nxd5 9. exd5 Nxb4 10. Qb3 bxc4 11. Qxb4 cxd3 12. Qc3 d6 13. Qxd3
↪  a5
14. Qb5+ Ke7 15. Qc6 Ra6 16. Qc3 Bf5 17. Qc4 Ra7 18. Nc3 g4 19. fxg4 hxg4
20. Ne4 Rb7 21. Ng3 Bc8 22. Rc1 f6 23. Ne4 Bd7 24. Qc3 Bf5 25. Ng3 Bh7
26. Qxa5 e4 27. Rc6 Re8 28. Qc3 Rh8 29. Rc4 c5 30. Nxe4 f5 31. Qg7+ Ke8
32. Qxh8+ Ke7 33. Qxh7+ Ke8 34. Nxd6+ Qxd6 35. Qxb7 g3 36. Qc6+ Qxc6
37. dxc6 gxh2 38. Nh3 Ke7 39. Rxc5 Ke8 40. c7 Ke7 41. Nf4 Kf6 42. Nd5+ Kg5
43. c8=Q Kg6 44. Qg8+ Kh5 45. Rxh2# 1-0
```

2. El jugador que no eligió ninguna jugada

Copa Vertex Verano 2026, ronda 4. Qwen legacy + árbitro (blancas) contra Gemma + árbitro (negras). Blancas: **0 jugadas propias de 40**, todas al azar entre los candidatos del árbitro. Negras: 38 de 39.

```
1. d4 f6 2. c4 e6 3. e4 Bb4+ 4. Ke2 d6 5. f3 c6 6. g4 e5 7. a3 exd4 8.
↪  axb4 h5
9. gxh5 d3+ 10. Ke1 d2+ 11. Kxd2 Rxh5 12. Nc3 Re5 13. h4 Be6 14. f4 Bxc4
15. fxe5 dxe5+ 16. Ke1 Qxd1+ 17. Nxd1 Bxf1 18. Kxf1 b6 19. Be3 Nd7 20. Rc1
↪  Ne7
21. Rc3 a6 22. b3 b5 23. h5 Kf7 24. Rd3 Ke6 25. Rc3 Rh8 26. Rh4 g6 27. h6
↪  Rf8
28. Nf3 Rh8 29. h7 Nf8 30. Rh1 Nxh7 31. Rh6 Kd6 32. Bc5+ Kd7 33. Ne3 Ke6
```

```
34. Bxe7 Kxe7 35. Rxc6 Kf7 36. Rh2 Kg7 37. Ra2 Ng5 38. Rc7+ Kh6 39. Rh2+
↪  Nh3
40. Rxh3# 1-0
```

3. Mate en cinco sin elegir nada

Copa Vertex Verano 2026, ronda 7. El mismo modelo en sus dos versiones: Qwen legacy puro (blancas) contra Qwen legacy + árbitro (negras). Negras: 0 jugadas propias de 5. La última, `Qh4#`, era la única opción: con mate en una, el árbitro no ofrece otra cosa.

```
1. h3 e5 2. g3 c5 3. c3 f5 4. f3 h5 5. g4 Qh4# 0-1
```

APÉNDICE D

El pipeline de datos

Estos dos scripts producen y verifican el dataset del capítulo 3. Se ejecutan desde `E:\vertex-chess-arena` con el entorno virtual del proyecto (apéndice A).

Cómo leer `01_build_chess_dataset.py`. Las primeras líneas redirigen todas las cachés de Hugging Face al disco del proyecto *antes* de importar `datasets` (capítulo 2). Después hay una función por fuente:

- `format_record` convierte una posición y su jugada en la conversación de tres mensajes. Aquí está la línea `legal_sans[:45]`, el recorte silencioso que descubrimos en el capítulo 3.
- `extract_lichess_evals` lee en *streaming* las evaluaciones de Lichess y se queda solo con posiciones decisivas.
- `extract_lichess_puzzles` aplica primero la jugada del rival (el formato de los *puzzles* guarda la posición anterior al error) y reserva la cuota de mates directos.
- `extract_sft_20k` completa con el conjunto curado.
- `main` junta las tres fuentes, baraja con semilla 42 y escribe el JSONL.

Todas comparten el conjunto `global_seen`, que evita repetir un FEN aunque venga de dos fuentes distintas.

Cómo leer `01b_audit_chess_dataset.py`. Relee el archivo terminado y comprueba cuotas, legalidad, formato de la respuesta y reparto por bando y fase. Es la auditoría que *sí* existía; el capítulo 3 cuenta la que faltaba.

scripts/01_build_chess_dataset.py

```
"""
VERTEX CHESS ARENA - Motor de Ingesta Híbrida V2 (Deduplicación Canónica + Instinto de Mate)
Garantiza exactamente:
  - 2,000 (40.0%) lichess_evals   (0% duplicados canónicos, ventaja clara)
```

```
  - 2,000 (40.0%) lichess_puzzles (Incluye cuota dedicada de Mates Directos '#' + Táctica Élite)
  - 1,000 (20.0%) sft_20k         (Posiciones maestras de agi-noobs/chess-sft-20k)
"""
import os
import json
import random
from pathlib import Path

BASE_DIR = Path("E:/vertex-chess-arena").resolve()
CACHE_DIR = BASE_DIR / ".cache" / "huggingface"
CACHE_DIR.mkdir(parents=True, exist_ok=True)

os.environ["HF_HOME"] = str(CACHE_DIR)
os.environ["HF_DATASETS_CACHE"] = str(CACHE_DIR / "datasets")
os.environ["HF_HUB_DISABLE_XET"] = "1"

import chess
from datasets import load_dataset

OUTPUT_DIR = BASE_DIR / "dataset" / "processed"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_FILE = OUTPUT_DIR / "train_chess_elite.jsonl"

QUOTA_EVALS = 2000
QUOTA_PUZZLES = 2000
QUOTA_SFT = 1000

SYSTEM_PROMPT = (
    "You are an elite chess grandmaster engine created by Vertex Coders. "
    "Analyze the FEN board state and the list of legal moves, then output "
    "the single strongest tactical move in both SAN and UCI formats."
)


def format_record(board: chess.Board, target_move: chess.Move, source_tag: str) -> dict | None:
    """Valida legalidad en python-chess y devuelve el registro normalizado."""
```

```
    if not board.is_valid() or target_move not in board.legal_moves:
        return None

    san_move = board.san(target_move)
    uci_move = target_move.uci()

    legal_sans = [board.san(m) for m in board.legal_moves]
    legal_str = ", ".join(legal_sans[:45])

    user_prompt = (
        f"Position FEN: {board.fen()}\n"
        f"Legal moves (SAN): {legal_str}\n"
        f"Select the best move."
    )
    assistant_reply = f"Best move: {san_move} (UCI: {uci_move})"

    return {
        "source": source_tag,
        "fen": board.fen(),
        "san": san_move,
        "uci": uci_move,
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": user_prompt},
            {"role": "assistant", "content": assistant_reply},
        ]
    }


def extract_lichess_evals(quota: int, global_seen: set) -> list[dict]:
    print(f"\n>>> [1/3] Extrayendo {quota} posiciones únicas de
    ↪  Lichess/chess-position-evaluations...")
    records = []
    try:
        ds = load_dataset("Lichess/chess-position-evaluations",
↪   split="train", streaming=True)
        for row in ds:
            fen = row.get("fen")
            line = row.get("line")
            cp = row.get("cp")
            mate = row.get("mate")

```

```
            if not fen or not line:
                continue

            is_decisive = (mate is not None and mate != 0) or (cp is not
↪  None and abs(cp) >= 150)
            if not is_decisive:
                continue

            moves_uci = line.split()
            if not moves_uci:
                continue

            try:
                board = chess.Board(fen)
                if len(board.piece_map()) < 7 or board.fullmove_number >
                ↪  40:
                    continue

                canonical_fen = board.fen()
                if canonical_fen in global_seen:
                    continue

                move = chess.Move.from_uci(moves_uci[0])
                rec = format_record(board, move, "lichess_evals")
                if rec:
                    records.append(rec)
                    global_seen.add(canonical_fen)
                    if len(records) % 500 == 0:
                        print(f"    -> Evaluaciones únicas recolectadas:
                        ↪  {len(records)}/{quota}")
            except Exception:
                continue

            if len(records) >= quota:
                break
    except Exception as e:
        print(f"    [!] Aviso en Evaluaciones: {e}")

    return records
```

```python
def extract_lichess_puzzles(quota: int, global_seen: set, tag_override: str = "lichess_puzzles") -> list[dict]:
    print(f"\n>>> [2/3] Extrayendo {quota} golpes tácticos y mates de Lichess/chess-puzzles...")
    records = []
    mate_target = quota // 4  # Asegurar al menos 25% (500) de Jaque Mate directo (#)
    mates_collected = 0
    tactics_target = quota - mate_target
    tactics_collected = 0

    try:
        ds = load_dataset("Lichess/chess-puzzles", split="train", streaming=True)
        for row in ds:
            rating = int(row.get("Rating") or 0)
            fen = row.get("FEN")
            moves_str = row.get("Moves")
            if not fen or not moves_str:
                continue

            uci_list = moves_str.split()
            if len(uci_list) < 2:
                continue

            try:
                board = chess.Board(fen)
                opponent_blunder = chess.Move.from_uci(uci_list[0])
                if opponent_blunder not in board.legal_moves:
                    continue
                board.push(opponent_blunder)

                canonical_fen = board.fen()
                if canonical_fen in global_seen:
                    continue

                winning_move = chess.Move.from_uci(uci_list[1])
                if winning_move not in board.legal_moves:
                    continue

                san_candidate = board.san(winning_move)
                is_direct_mate = "#" in san_candidate
```

```
157
158                 if is_direct_mate and mates_collected < mate_target:
159                     rec = format_record(board, winning_move,
    ↪  tag_override)
160                     if rec:
161                         records.append(rec)
162                         global_seen.add(canonical_fen)
163                         mates_collected += 1
164                         if len(records) % 500 == 0:
165                             print(f"    -> Puzzles recolectados:
                                ↪  {len(records)}/{quota} (Mates '#':
                                ↪  {mates_collected})")
166                 elif (not is_direct_mate) and rating >= 1900 and
                    ↪  tactics_collected < tactics_target:
167                     rec = format_record(board, winning_move,
    ↪  tag_override)
168                     if rec:
169                         records.append(rec)
170                         global_seen.add(canonical_fen)
171                         tactics_collected += 1
172                         if len(records) % 500 == 0:
173                             print(f"    -> Puzzles recolectados:
                                ↪  {len(records)}/{quota} (Mates '#':
                                ↪  {mates_collected})")
174             except Exception:
175                 continue
176
177             if len(records) >= quota:
178                 break
179     except Exception as e:
180         print(f"    [!] Aviso en Puzzles: {e}")
181
182     return records
183
184
185 def extract_sft_20k(quota: int, global_seen: set) -> list[dict]:
186     print(f"\n>>> [3/3] Extrayendo {quota} posiciones curadas de
    ↪  agi-noobs/chess-sft-20k...")
187     records = []
188     try:
189         ds = load_dataset("agi-noobs/chess-sft-20k", split="train",
    ↪  streaming=True)
```

```
        for row in ds:
            fen = row.get("fen") or row.get("FEN")
            move_str = (
                row.get("best_move_uci")
                or row.get("best_move_san")
                or row.get("pv_uci")
                or row.get("best_move")
                or row.get("uci_move")
                or row.get("move")
            )

            if not fen or not move_str:
                continue

            try:
                board = chess.Board(fen)
                canonical_fen = board.fen()
                if canonical_fen in global_seen:
                    continue

                first_token = str(move_str).strip().split()[0]
                try:
                    move = chess.Move.from_uci(first_token)
                except ValueError:
                    move = board.parse_san(first_token)

                rec = format_record(board, move, "sft_20k")
                if rec:
                    records.append(rec)
                    global_seen.add(canonical_fen)
                    if len(records) % 250 == 0:
                        print(f"    -> SFT recolectados:
                        ↪  {len(records)}/{quota}")
            except Exception:
                continue

            if len(records) >= quota:
                break
    except Exception as e:
        print(f"    [!] Aviso en SFT-20k: {e}")

    return records
```

```python


def main():
    print("=" * 70)
    print(">>> INICIANDO VERTEX GRANDMASTER BLEND V2 (0% DUPLICADOS CANÓNICOS)")
    print("=" * 70)

    global_seen_fens = set()

    eval_records = extract_lichess_evals(QUOTA_EVALS, global_seen_fens)
    puzzle_records = extract_lichess_puzzles(QUOTA_PUZZLES, global_seen_fens)
    sft_records = extract_sft_20k(QUOTA_SFT, global_seen_fens)

    all_records = eval_records + puzzle_records + sft_records
    random.seed(42)
    random.shuffle(all_records)

    with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
        for r in all_records:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")

    print("\n" + "=" * 70)
    print(f">>> ¡DATASET HÍBRIDO V2 COMPLETADO CON ÉXITO!")
    print(f"    - Aperturas/Estrategia (Lichess Evals): {len(eval_records)}")
    print(f"    - Táctica Letal (Lichess Puzzles): {len(puzzle_records)}")
    print(f"    - Refuerzo SFT (Chess-SFT-20k): {len(sft_records)}")
    print(f"    - TOTAL ÚNICO MEZCLADO: {len(all_records)} posiciones")
    print("=" * 70)


if __name__ == "__main__":
    main()
```

scripts/01b_audit_chess_dataset.py

```
"""
VERTEX CHESS ARENA - Auditoría Forense y Saneamiento AST de Dataset
Verifica cuotas (40/40/20), legalidad estricta en python-chess, fases de juego y perfil táctico.
"""
import json
import re
from pathlib import Path
from collections import Counter
import chess

BASE_DIR = Path("E:/vertex-chess-arena").resolve()
DATASET_FILE = BASE_DIR / "dataset" / "processed" / "train_chess_elite.jsonl"

# Cuotas esperadas según la tabla de arquitectura
EXPECTED_QUOTAS = {
    "lichess_evals":   (2000, 40.0),
    "lichess_puzzles": (2000, 40.0),
    "sft_20k":         (1000, 20.0),
}

FEN_EXTRACT_RE = re.compile(r"Position FEN:\s*([^\n]+)")
REPLY_EXTRACT_RE = re.compile(r"^Best move:\s*(\S+)\s*\(UCI:\s*([a-h][1-8][a-h][1-8][qrbn]?)\)$")


def audit_and_sanitize():
    print("=" * 78)
    print(">>> VERTEX CODERS - AUDITORÍA FORENSE Y SANEAMIENTO DEL DATASET DE AJEDREZ")
    print("=" * 78)

    if not DATASET_FILE.exists():
        print(f"[ERROR] No existe el archivo: {DATASET_FILE}")
        print("Ejecuta primero: python E:\\vertex-chess-arena\\scripts\\01_build_chess_dataset.py")
        return

    raw_lines = DATASET_FILE.read_text(encoding="utf-8").splitlines()
    print(f">>> Leyendo archivo: {DATASET_FILE}")
    print(f">>> Líneas totales en bruto: {len(raw_lines)}\n")
```

```
38
39      valid_records = []
40      seen_fens = set()
41
42      # Contadores de auditoría
43      errors = Counter()
44      source_counts = Counter()
45      phase_counts = Counter()
46      turn_counts = Counter()
47      tactical_profile = Counter()
48      seq_char_lengths = []
49
50      for idx, line in enumerate(raw_lines, start=1):
51          if not line.strip():
52              errors["empty_line"] += 1
53              continue
54
55          try:
56              obj = json.loads(line)
57          except Exception:
58              errors["invalid_json"] += 1
59              continue
60
61          msgs = obj.get("messages", [])
62          if len(msgs) != 3:
63              errors["bad_message_structure"] += 1
64              continue
65
66          user_content = msgs[1].get("content", "")
67          assistant_content = msgs[2].get("content", "").strip()
68
69          # 1. Extraer FEN desde el prompt real que verá el LLM
70          fen_m = FEN_EXTRACT_RE.search(user_content)
71          if not fen_m:
72              errors["missing_fen_in_prompt"] += 1
73              continue
74          fen = fen_m.group(1).strip()
75
76          # 2. Verificar duplicados
77          if fen in seen_fens:
78              errors["duplicate_fen"] += 1
79              continue
```

```
80
81          # 3. Validar sintaxis de respuesta del asistente
82          reply_m = REPLY_EXTRACT_RE.match(assistant_content)
83          if not reply_m:
84              errors["bad_assistant_format"] += 1
85              continue
86          san_move, uci_move = reply_m.group(1), reply_m.group(2)
87
88          # 4. Validación AST profunda con python-chess
89          try:
90              board = chess.Board(fen)
91              if not board.is_valid():
92                  errors["invalid_board_state"] += 1
93                  continue
94
95              move_from_uci = chess.Move.from_uci(uci_move)
96              if move_from_uci not in board.legal_moves:
97                  errors["illegal_uci_move"] += 1
98                  continue
99
100             expected_san = board.san(move_from_uci)
101             if expected_san != san_move:
102                 errors["san_uci_mismatch"] += 1
103                 continue
104         except Exception:
105             errors["chess_engine_exception"] += 1
106             continue
107
108         # Registro 100% válido y verificado
109         seen_fens.add(fen)
110         valid_records.append(obj)
111
112         # Métricas de distribución
113         src = obj.get("source", "untagged_legacy")
114         source_counts[src] += 1
115
116         # Bando al turno
117         turn_counts["White (w)" if board.turn == chess.WHITE else "Black
    ↪  (b)"] += 1
118
119         # Fase de la partida según fullmove_number y cantidad de piezas
120         piece_count = len(board.piece_map())
```

```
        if board.fullmove_number <= 12:
            phase_counts["1. Apertura (Jugadas 1-12)"] += 1
        elif piece_count <= 10 or board.fullmove_number > 35:
            phase_counts["3. Final (<=10 piezas o Jugada >35)"] += 1
        else:
            phase_counts["2. Medio Juego (Jugadas 13-35)"] += 1

        # Perfil táctico de la jugada objetivo
        if "#" in san_move:
            tactical_profile["Jaque Mate Directo (#)"] += 1
        elif "+" in san_move:
            tactical_profile["Jaque Táctico (+)"] += 1
        if "x" in san_move:
            tactical_profile["Captura de Material (x)"] += 1
        if "=" in san_move:
            tactical_profile["Coronación de Peón (=)"] += 1
        if "x" not in san_move and "+" not in san_move and "#" not in
        ↪  san_move:
            tactical_profile["Desarrollo / Posicional Silencioso"] += 1

        total_chars = sum(len(m.get("content", "")) for m in msgs)
        seq_char_lengths.append(total_chars)

    total_valid = len(valid_records)
    if total_valid == 0:
        print("[CRÍTICO] 0 registros válidos encontrados. Revisa el
        ↪  generador.")
        return

    #
    ↪  ========================================================================
    # REPORTE DE CUMPLIMIENTO DE LA TABLA (40% / 40% / 20%)
    #
    ↪  ========================================================================
    print("1. CUMPLIMIENTO DE LA TABLA DE BLENDING (CUOTAS POR FUENTE)")
    print("-" * 78)
    print(f"{'Fuente de Datos':<25} | {'Meta':<14} | {'Real':<14} |
    ↪  {'Estado'}")
    print("-" * 78)
    for src_key, (target_cnt, target_pct) in EXPECTED_QUOTAS.items():
        actual_cnt = source_counts.get(src_key, 0)
        actual_pct = (actual_cnt / total_valid) * 100
```

```
        diff_pct = abs(actual_pct - target_pct)
        status = "OK (CUMPLIDO)" if diff_pct <= 2.5 and actual_cnt > 0
    ↪  else "DESVIADO"
        print(
            f"{src_key:<25} | {target_cnt:4d} ({target_pct:4.1f}%) | "
            f"{actual_cnt:4d} ({actual_pct:4.1f}%) | {status}"
        )
    if source_counts.get("untagged_legacy", 0) > 0:
        cnt = source_counts["untagged_legacy"]
        print(f"{'untagged_legacy (Viejo)':<25} | {'0 (0.0%)':<14} |
        ↪  {cnt:4d} ({(cnt/total_valid)*100:4.1f}%) | EJECUTA EL NUEVO
        ↪  01")
    print("-" * 78)

    #
    ↪  ============================================================================
    # REPORTE DE AGRESIVIDAD TÁCTICA Y FASES DEL JUEGO
    #
    ↪  ============================================================================
    print("\n2. PERFIL TÁCTICO Y ANTI-BUCLES (QUÉ APRENDERÁN QWEN Y
    ↪  GEMMA)")
    print("-" * 78)
    for k, v in sorted(tactical_profile.items()):
        print(f"  - {k:<36}: {v:4d} ({(v/total_valid)*100:5.1f}%)")

    print("\n3. DISTRIBUCIÓN POR FASE DE PARTIDA Y TURNO")
    print("-" * 78)
    for k, v in sorted(phase_counts.items()):
        print(f"  - {k:<36}: {v:4d} ({(v/total_valid)*100:5.1f}%)")
    for k, v in sorted(turn_counts.items()):
        print(f"  - Turno {k:<30}: {v:4d} ({(v/total_valid)*100:5.1f}%)")

    #
    ↪  ============================================================================
    # AUDITORÍA DE LONGITUD PARA VRAM (MAX_SEQ_LENGTH = 384)
    #
    ↪  ============================================================================
    max_chars = max(seq_char_lengths)
    avg_chars = sum(seq_char_lengths) / len(seq_char_lengths)
    est_max_tokens = int(max_chars / 3.2)
    print("\n4. AUDITORÍA DE SEGURIDAD VRAM (GTX 1660 Ti - 6 GB)")
    print("-" * 78)
```

```
    print(f"  - Longitud promedio de secuencia : {avg_chars:.0f} chars (~{int(avg_chars/3.2)} tokens)")
    print(f"  - Longitud máxima de secuencia   : {max_chars} chars (~{est_max_tokens} tokens)")
    print(f"  - Límite configurado (MAX_SEQ)   : 384 tokens -> {'SEGURO (0% Truncamiento)' if est_max_tokens < 380 else 'AJUSTAR'}")

    # ==========================================================================
    # SANEAMIENTO AUTOMÁTICO
    # ==========================================================================
    print("\n5. RESULTADO DE SANEAMIENTO AST (PYTHON-CHESS)")
    print("-" * 78)
    print(f"  - Registros válidos certificados : {total_valid}")
    print(f"  - Registros corruptos/duplicados : {sum(errors.values())}")
    if errors:
        for err_type, cnt in errors.items():
            print(f"      * Purgado por [{err_type}]: {cnt}")
        # Sobrescribir el dataset solo con los registros limpios
        with open(DATASET_FILE, "w", encoding="utf-8") as f:
            for r in valid_records:
                f.write(json.dumps(r, ensure_ascii=False) + "\n")
        print(f"  >>> [SANEADO] Se sobrescribió {DATASET_FILE.name} con {total_valid} registros 100% puros.")
    else:
        print("  >>> [PERFECTO] 0 errores detectados. El 100% de las jugadas son legales y únicas.")

    # Mostrar 3 muestras reales (1 de cada fuente)
    print("\n6. MUESTRAS REALES AUDITADAS DEL DATASET")
    print("-" * 78)
    shown_sources = set()
    for r in valid_records:
        s = r.get("source", "untagged")
        if s not in shown_sources:
            shown_sources.add(s)
            print(f"  [{s.upper()}]")
            print(f"    FEN       : {r.get('fen', 'N/A')}")
            print(f"    Respuesta : {r['messages'][2]['content']}")
        if len(shown_sources) >= 3:
            break
```

```
    print("=" * 78)


if __name__ == "__main__":
    audit_and_sanitize()
```

APÉNDICE E

Los dos entrenamientos

Este es el corazón técnico del libro: dos scripts que entrenan dos modelos de familias distintas **con la misma receta**, línea por línea, en una GPU de 6 GB. Compararlos uno al lado del otro es la mejor forma de ver qué es receta (idéntico en los dos) y qué es adaptación a cada arquitectura (lo único que cambia).

Lo que es idéntico. Las constantes del principio: `LORA_R = 8`, `LORA_ALPHA = 16`, `LEARNING_RATE = 5e-5`, `WARMUP_STEPS = 15`, lote 1 × 8, `MAX_SEQ_LENGTH = 384`, una época, semilla 42, `fp16`, atención `eager`, *gradient checkpointing* y los siete módulos LoRA.

Lo que cambia, y por qué.

- **La plantilla de chat.** Gemma no tiene rol de sistema: `to_chess_turns` pega el texto de sistema al principio del mensaje del usuario y llama `model` al turno del asistente. Qwen sí tiene rol de sistema y usa ChatML.
- **El enmascarado.** En Gemma, `encode_turns` codifica turno a turno y pone `-100` en todo lo que no es la respuesta del modelo. En Qwen, `encode_example` pide el texto del prompt con `apply_chat_template(..., tokenize=False, add_generation_prompt=True)`, lo codifica aparte (capítulo 9) y añade `<|im_end|>` a la respuesta para que el modelo aprenda a cerrar su turno.
- **El modelo base**, cada uno en su versión de 4 bits.
- **Los puntos de guardado.** Gemma entrenó sin ninguno (`save_strategy="no"`). Qwen V3 guarda cada 125 pasos y reanuda solo desde el último (`resume_from_checkpoint`), la lección del capítulo 4.

Los dos usan `DataCollatorForSeq2Seq`, que rellena las secuencias de un lote y respeta las etiquetas `-100`. Gemma tardó unas 11 horas; Qwen V3, 13 h 42 min.

scripts/03_train_gemma_chess.py

```
"""
Entrenamiento ultra-seguro de Gemma 2 (2B) para ajedrez en la unidad E:
con gestión correcta de CachingAllocator para la GTX 1660 Ti.
"""
from __future__ import annotations

import os
import gc

# ============================================================================
# 0. AISLAMIENTO ESTRICTO EN LA UNIDAD E: + CONFIGURACIÓN CUDA VRAM
# ============================================================================
os.environ["PYTORCH_CUDA_ALLOC_CONF"] =
↪  "expandable_segments:True,max_split_size_mb:32"
os.environ["HF_HUB_DISABLE_XET"] = "1"
os.environ["HF_HUB_ENABLE_HF_TRANSFER"] = "0"

import json
import random
from pathlib import Path
from typing import Any
from collections import Counter

BASE_DIR = Path("E:/vertex-chess-arena").resolve()
CACHE_DIR = BASE_DIR / ".cache"
TMP_DIR = CACHE_DIR / "tmp"
HF_CACHE_DIR = CACHE_DIR / "huggingface"
TORCH_CACHE_DIR = CACHE_DIR / "torch"
TRITON_CACHE_DIR = CACHE_DIR / "triton"

for d in [TMP_DIR, HF_CACHE_DIR, TORCH_CACHE_DIR, TRITON_CACHE_DIR]:
    d.mkdir(parents=True, exist_ok=True)

os.environ["HF_HOME"] = str(HF_CACHE_DIR)
os.environ["HF_DATASETS_CACHE"] = str(HF_CACHE_DIR / "datasets")
os.environ["TORCH_HOME"] = str(TORCH_CACHE_DIR)
os.environ["TRITON_CACHE_DIR"] = str(TRITON_CACHE_DIR)
os.environ["TMP"] = str(TMP_DIR)
os.environ["TEMP"] = str(TMP_DIR)

import torch
from transformers import (
```

```
    AutoTokenizer,
    AutoModelForCausalLM,
    DataCollatorForSeq2Seq,
    Trainer,
    TrainingArguments
)
from peft import LoraConfig, get_peft_model
from datasets import Dataset

#
↪  ================================================================================
# CONFIGURACIÓN ESTRICTA PARA 6 GB VRAM
#
↪  ================================================================================
BASE_MODEL = "unsloth/gemma-2-2b-it-bnb-4bit"
DATASET_PATH = BASE_DIR / "dataset" / "processed" /
↪  "train_chess_elite.jsonl"
OUTPUT_DIR = BASE_DIR / "outputs_gemma"
FINAL_DIR = BASE_DIR / "models" / "gemma_chess_adapter"

MAX_SEQ_LENGTH = 384
SEED = 42
NUM_EPOCHS = 1
#MAX_STEPS = 80
MAX_STEPS = -1
PER_DEVICE_BATCH_SIZE = 1
GRADIENT_ACCUMULATION_STEPS = 8
LEARNING_RATE = 5e-5
WARMUP_STEPS = 15

LORA_R = 8
LORA_ALPHA = 16
LORA_DROPOUT = 0.0
LORA_TARGET_MODULES = ["q_proj", "k_proj", "v_proj", "o_proj",
↪  "gate_proj", "up_proj", "down_proj"]


def encode_turns(turns: list[tuple[str, str]], tok) -> tuple[list[int],
↪  list[int]]:
    enc = lambda s: tok.encode(s, add_special_tokens=False)
    eot = enc("<end_of_turn>")
    nl = enc("\n")
```

```
79
80      ids = [tok.bos_token_id]
81      labels = [-100]
82      for role, content in turns:
83          head = enc(f"<start_of_turn>{role}\n")
84          body = enc(content)
85          ids += head + body + eot + nl
86          if role == "model":
87              labels += [-100] * len(head) + body + eot + [-100] * len(nl)
88          else:
89              labels += [-100] * (len(head) + len(body) + len(eot) +
    ↪  len(nl))
90      return ids, labels
91
92
93  def to_chess_turns(messages: list[dict[str, str]]) -> list[tuple[str,
    ↪  str]]:
94      turns = []
95      sys_prompt = ""
96      for msg in messages:
97          if msg["role"] == "system":
98              sys_prompt = msg["content"]
99          elif msg["role"] == "user":
100             user_content = msg["content"]
101             if sys_prompt:
102                 user_content = f"{sys_prompt}\n\n{user_content}"
103                 sys_prompt = ""
104             turns.append(("user", user_content))
105         elif msg["role"] == "assistant":
106             turns.append(("model", msg["content"]))
107     return turns
108
109
110 def build_dataset(records: list[dict[str, Any]], tok) ->
    ↪  tuple[list[dict[str, Any]], Counter]:
111     stats: Counter[str] = Counter()
112     rows = []
113     for rec in records:
114         turns = to_chess_turns(rec["messages"])
115         ids, labels = encode_turns(turns, tok)
116         if len(ids) > MAX_SEQ_LENGTH:
117             stats["drop_too_long"] += 1
```

```
            continue
        if all(l == -100 for l in labels):
            stats["drop_no_labels"] += 1
            continue
        rows.append({"input_ids": ids, "attention_mask": [1] * len(ids),
↪   "labels": labels})
        stats["kept"] += 1
    return rows, stats


def main() -> None:
    random.seed(SEED)
    torch.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)

    gc.collect()
    torch.cuda.empty_cache()

    print("=" * 65)
    print(f">>> ENTRENAMIENTO SEGURO DE GEMMA 2 (2B) | Dataset:
    ↪  {DATASET_PATH}")
    print("=" * 65)

    print(">>> Cargando tokenizador oficial de Gemma 2...")
    tok = AutoTokenizer.from_pretrained(
        BASE_MODEL,
        model_max_length=MAX_SEQ_LENGTH,
        trust_remote_code=True,
        cache_dir=str(HF_CACHE_DIR),
    )
    if tok.pad_token is None:
        tok.pad_token = tok.eos_token if tok.eos_token else "<pad>"
    tok.padding_side = "right"

    records = [json.loads(l) for l in
↪   Path(DATASET_PATH).read_text(encoding="utf-8").splitlines() if
↪   l.strip()]
    rows, stats = build_dataset(records, tok)

    print(f"Registros totales: {len(records)} | Conservados:
    ↪  {stats['kept']}")

```

```
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA es obligatorio en este entorno.")

    print(">>> Liberando caché CUDA antes de cargar modelo...")
    torch.cuda.empty_cache()
    gc.collect()

    print(">>> Cargando modelo base Gemma 2 en torch.float16...")
    model = AutoModelForCausalLM.from_pretrained(
        BASE_MODEL,
        torch_dtype=torch.float16,
        attn_implementation="eager",
        device_map="auto",
        cache_dir=str(HF_CACHE_DIR),
    )
    model.config.use_cache = False

    print(">>> Aplicando adaptadores LoRA (r=8)...")
    model = get_peft_model(model, LoraConfig(
        r=LORA_R,
        lora_alpha=LORA_ALPHA,
        lora_dropout=LORA_DROPOUT,
        bias="none",
        task_type="CAUSAL_LM",
        target_modules=LORA_TARGET_MODULES,
    ))
    model.print_trainable_parameters()
    model.enable_input_require_grads()

    random.shuffle(rows)
    train_ds = Dataset.from_list(rows)

    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    training_args = TrainingArguments(
        output_dir=str(OUTPUT_DIR),
        num_train_epochs=NUM_EPOCHS,
        max_steps=MAX_STEPS,
        per_device_train_batch_size=PER_DEVICE_BATCH_SIZE,
        gradient_accumulation_steps=GRADIENT_ACCUMULATION_STEPS,
        learning_rate=LEARNING_RATE,
        warmup_steps=WARMUP_STEPS,
```

```
        lr_scheduler_type="cosine",
        logging_steps=1,
        save_strategy="no",
        bf16=False,
        fp16=True,
        gradient_checkpointing=True,
        gradient_checkpointing_kwargs={"use_reentrant": False},
        optim="adamw_torch",
        report_to="none",
        seed=SEED,
        data_seed=SEED,
        remove_unused_columns=False,
    )

    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_ds,
        data_collator=DataCollatorForSeq2Seq(
            tok,
            padding=True,
            label_pad_token_id=-100,
            pad_to_multiple_of=8
        ),
    )

    print(">>> ¡Arrancando entrenamiento en modo fp16 seguro para GTX
    ↪  1660 Ti!")
    trainer.train()

    FINAL_DIR.mkdir(parents=True, exist_ok=True)
    trainer.model.save_pretrained(str(FINAL_DIR))
    tok.save_pretrained(str(FINAL_DIR))

    print(f"\n>>> ¡Contendiente 2 (Gemma) entrenado con éxito absoluto!")
    print(f">>> Adaptador LoRA guardado en: {FINAL_DIR}")


if __name__ == "__main__":
    main()
```

scripts/02_train_qwen_chess.py

```
"""
VERTEX CHESS ARENA - Qwen 2.5 (3B) V3: entrenamiento ESPEJO del de Gemma 2 (2B).

Por qué existe esta versión (ver logs/bitacora.md, 2026-10-20):
  - El Qwen que estaba en Ollama era el adaptador viejo de 80 pasos, entrenado con el
    dataset de 3,000 filas y con pérdida sobre TODO el texto (system + FEN + lista de
    jugadas). Gemma se entrenó con 5,000 filas y pérdida solo sobre la respuesta.
  - Para que el duelo sea justo, este script copia la receta de 03_train_gemma_chess.py:
    mismo dataset, mismo Trainer + DataCollatorForSeq2Seq, misma máscara (-100) sobre el
    prompt, mismos hiperparámetros (r=8, alpha=16, lr=5e-5, warmup=15, cosine, 1 época,
    batch efectivo 8, fp16, seed 42, MAX_SEQ_LENGTH=384).
  - Lo único que cambia es el formato de chat: Qwen usa ChatML con rol system propio.

Correr desde E:\\vertex-chess-arena :
    venv\\Scripts\\python.exe scripts\\02_train_qwen_chess.py
"""
from __future__ import annotations

import os
import gc

os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True,max_split_size_mb:32"
os.environ["HF_HUB_DISABLE_XET"] = "1"
os.environ["HF_HUB_ENABLE_HF_TRANSFER"] = "0"

import json
import random
from pathlib import Path
from typing import Any
from collections import Counter

BASE_DIR = Path("E:/vertex-chess-arena").resolve()
CACHE_DIR = BASE_DIR / ".cache"
```

```
TMP_DIR = CACHE_DIR / "tmp"
HF_CACHE_DIR = CACHE_DIR / "huggingface"
TORCH_CACHE_DIR = CACHE_DIR / "torch"
TRITON_CACHE_DIR = CACHE_DIR / "triton"
for d in [TMP_DIR, HF_CACHE_DIR, TORCH_CACHE_DIR, TRITON_CACHE_DIR]:
    d.mkdir(parents=True, exist_ok=True)

os.environ["HF_HOME"] = str(HF_CACHE_DIR)
os.environ["HF_DATASETS_CACHE"] = str(HF_CACHE_DIR / "datasets")
os.environ["TORCH_HOME"] = str(TORCH_CACHE_DIR)
os.environ["TRITON_CACHE_DIR"] = str(TRITON_CACHE_DIR)
os.environ["TMP"] = str(TMP_DIR)
os.environ["TEMP"] = str(TMP_DIR)

import torch
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    DataCollatorForSeq2Seq,
    Trainer,
    TrainingArguments,
)
from peft import LoraConfig, get_peft_model
from datasets import Dataset

#
↪  ============================================================================
# CONFIGURACIÓN — IDÉNTICA A 03_train_gemma_chess.py (salvo modelo y
↪  rutas)
#
↪  ============================================================================
BASE_MODEL = "unsloth/Qwen2.5-3B-Instruct-bnb-4bit"
DATASET_PATH = BASE_DIR / "dataset" / "processed" /
↪  "train_chess_elite.jsonl"
OUTPUT_DIR = BASE_DIR / "outputs_qwen_v3"
FINAL_DIR = BASE_DIR / "models" / "qwen_chess_adapter_v3"

MAX_SEQ_LENGTH = 384
SEED = 42
NUM_EPOCHS = 1
MAX_STEPS = -1                # 1 época completa = 5000 / 8 = 625 pasos,
↪  igual que Gemma
```

```
PER_DEVICE_BATCH_SIZE = 1
GRADIENT_ACCUMULATION_STEPS = 8
LEARNING_RATE = 5e-5
WARMUP_STEPS = 15

LORA_R = 8
LORA_ALPHA = 16
LORA_DROPOUT = 0.0
LORA_TARGET_MODULES = ["q_proj", "k_proj", "v_proj", "o_proj",
↪  "gate_proj", "up_proj", "down_proj"]

END_OF_TURN = "<|im_end|>"


def encode_example(messages: list[dict[str, str]], tok) ->
↪  tuple[list[int], list[int]]:
    """prompt (system+user+'<|im_start|>assistant\\n') enmascarado; se
    ↪  entrena solo
    la respuesta + <|im_end|>, exactamente como Gemma entrena respuesta +
↪  <end_of_turn>."""
    prompt_msgs = [m for m in messages if m["role"] != "assistant"]
    answer = next(m["content"] for m in messages if m["role"] ==
↪  "assistant")

    prompt_text = tok.apply_chat_template(prompt_msgs, tokenize=False,
↪  add_generation_prompt=True)
    prompt_ids = tok.encode(prompt_text, add_special_tokens=False)
    answer_ids = tok.encode(answer, add_special_tokens=False)
    eot_ids = tok.encode(END_OF_TURN, add_special_tokens=False)
    nl_ids = tok.encode("\n", add_special_tokens=False)

    ids = list(prompt_ids) + answer_ids + eot_ids + nl_ids
    labels = [-100] * len(prompt_ids) + answer_ids + eot_ids + [-100] *
↪  len(nl_ids)
    return ids, labels


def build_dataset(records: list[dict[str, Any]], tok) ->
↪  tuple[list[dict[str, Any]], Counter]:
    stats: Counter[str] = Counter()
    rows = []
    for rec in records:
```

```
        ids, labels = encode_example(rec["messages"], tok)
        if len(ids) > MAX_SEQ_LENGTH:
            stats["drop_too_long"] += 1
            continue
        if all(l == -100 for l in labels):
            stats["drop_no_labels"] += 1
            continue
        rows.append({"input_ids": ids, "attention_mask": [1] * len(ids),
↪   "labels": labels})
        stats["kept"] += 1
    return rows, stats


def main() -> None:
    random.seed(SEED)
    torch.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)
    gc.collect()
    torch.cuda.empty_cache()

    print("=" * 65)
    print(f">>> QWEN 2.5 (3B) V3 — ESPEJO DE GEMMA | Dataset:
    ↪  {DATASET_PATH}")
    print("=" * 65)

    tok = AutoTokenizer.from_pretrained(BASE_MODEL,
↪   model_max_length=MAX_SEQ_LENGTH, cache_dir=str(HF_CACHE_DIR))
    if tok.pad_token is None:
        tok.pad_token = tok.eos_token
    tok.padding_side = "right"

    records = [json.loads(l) for l in
↪   Path(DATASET_PATH).read_text(encoding="utf-8").splitlines() if
↪   l.strip()]
    rows, stats = build_dataset(records, tok)
    print(f"Registros totales: {len(records)} | Conservados:
    ↪  {stats['kept']} | Descartados: {dict(stats)}")

    # Muestra de control: lo que ve el modelo y lo que aprende
    sample = rows[0]
    learned = [t for t, l in zip(sample["input_ids"], sample["labels"])
↪   if l != -100]
```

```
    print(">>> Ejemplo de tokens entrenados:", repr(tok.decode(learned)))

    if not torch.cuda.is_available():
        raise RuntimeError("CUDA es obligatorio en este entorno.")
    torch.cuda.empty_cache()
    gc.collect()

    print(">>> Cargando Qwen 2.5 3B (bnb-4bit) en torch.float16...")
    model = AutoModelForCausalLM.from_pretrained(
        BASE_MODEL,
        torch_dtype=torch.float16,
        attn_implementation="eager",
        device_map="auto",
        cache_dir=str(HF_CACHE_DIR),
    )
    model.config.use_cache = False

    model = get_peft_model(model, LoraConfig(
        r=LORA_R,
        lora_alpha=LORA_ALPHA,
        lora_dropout=LORA_DROPOUT,
        bias="none",
        task_type="CAUSAL_LM",
        target_modules=LORA_TARGET_MODULES,
    ))
    model.print_trainable_parameters()
    model.enable_input_require_grads()

    random.shuffle(rows)
    train_ds = Dataset.from_list(rows)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    training_args = TrainingArguments(
        output_dir=str(OUTPUT_DIR),
        num_train_epochs=NUM_EPOCHS,
        max_steps=MAX_STEPS,
        per_device_train_batch_size=PER_DEVICE_BATCH_SIZE,
        gradient_accumulation_steps=GRADIENT_ACCUMULATION_STEPS,
        learning_rate=LEARNING_RATE,
        warmup_steps=WARMUP_STEPS,
        lr_scheduler_type="cosine",
        logging_steps=1,
```

```
        save_strategy="steps",
        save_steps=125,              # checkpoints por si se corta (Gemma usaba "no")
        save_total_limit=2,
        bf16=False,
        fp16=True,
        gradient_checkpointing=True,
        gradient_checkpointing_kwargs={"use_reentrant": False},
        optim="adamw_torch",
        report_to="none",
        seed=SEED,
        data_seed=SEED,
        remove_unused_columns=False,
    )

    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_ds,
        data_collator=DataCollatorForSeq2Seq(tok, padding=True, label_pad_token_id=-100, pad_to_multiple_of=8),
    )

    print(">>> Arrancando entrenamiento V3 (fp16, GTX 1660 Ti)...")
    last_ckpt = None
    ckpts = sorted(OUTPUT_DIR.glob("checkpoint-*"), key=lambda p: int(p.name.split("-")[1]))
    if ckpts:
        last_ckpt = str(ckpts[-1])
        print(f">>> Reanudando desde {last_ckpt}")
    trainer.train(resume_from_checkpoint=last_ckpt)

    FINAL_DIR.mkdir(parents=True, exist_ok=True)
    trainer.model.save_pretrained(str(FINAL_DIR))
    tok.save_pretrained(str(FINAL_DIR))
    (FINAL_DIR / "train_log.json").write_text(json.dumps(trainer.state.log_history, indent=1), encoding="utf-8")
    print(f"\n>>> Qwen V3 entrenado. Adaptador en: {FINAL_DIR}")


if __name__ == "__main__":
    main()
```

APÉNDICE F

Exportación a GGUF y Modelfiles

`export_contender` fusiona el adaptador con su modelo base y cuantiza a `q4_k_m` con Unsloth. `write_clean_modelfiles` escribe los Modelfiles con la plantilla de chat **explícita** y los `stop` de cada familia (capítulo 6).

Fíjate en el bloque final de `main`: solo exporta un GGUF si todavía no existe. Así, al reexportar Qwen V3 no se tocó el GGUF de Gemma con el que ya se habían jugado partidas, y las cifras del libro siguen correspondiendo a los archivos publicados.

scripts/05_export_both_gguf.py

```python
import os
import gc
import shutil
from pathlib import Path

# ============================================================================
# 0. AISLAMIENTO ESTRICTO EN LA UNIDAD E: + FIX DE DESCARGA HF XET
# ============================================================================
BASE_DIR = Path("E:/vertex-chess-arena").resolve()
CACHE_DIR = BASE_DIR / ".cache"
HF_CACHE_DIR = CACHE_DIR / "huggingface"
TORCH_CACHE_DIR = CACHE_DIR / "torch"
TRITON_CACHE_DIR = CACHE_DIR / "triton"
TMP_DIR = CACHE_DIR / "tmp"

for d in [HF_CACHE_DIR, TORCH_CACHE_DIR, TRITON_CACHE_DIR, TMP_DIR]:
    d.mkdir(parents=True, exist_ok=True)

os.environ["HF_HOME"] = str(HF_CACHE_DIR)
os.environ["HF_DATASETS_CACHE"] = str(HF_CACHE_DIR / "datasets")
os.environ["TORCH_HOME"] = str(TORCH_CACHE_DIR)
os.environ["TRITON_CACHE_DIR"] = str(TRITON_CACHE_DIR)
os.environ["TMP"] = str(TMP_DIR)
```

```
os.environ["TEMP"] = str(TMP_DIR)
os.environ["HF_HUB_DISABLE_XET"] = "1"
os.environ["HF_HUB_ENABLE_HF_TRANSFER"] = "0"

import torch
from unsloth import FastLanguageModel

GGUF_DIR = BASE_DIR / "models" / "gguf"
GGUF_DIR.mkdir(parents=True, exist_ok=True)

def export_contender(adapter_name: str, target_gguf_name: str) -> None:
    adapter_path = BASE_DIR / "models" / adapter_name
    if not (adapter_path / "adapter_config.json").exists():
        print(f">>> [OMITIDO] No se encontró {adapter_path /
        ↪  'adapter_config.json'}")
        return

    gc.collect()
    torch.cuda.empty_cache()

    temp_export_dir = BASE_DIR / "models" / f"tmp_{adapter_name}"

    print(f"\n{'='*65}")
    print(f">>> EXPORTANDO {adapter_name} A GGUF (q4_k_m)...")
    print(f"{'='*65}")

    model, tokenizer = FastLanguageModel.from_pretrained(
        model_name=str(adapter_path),
        max_seq_length=512,
        dtype=torch.float16,
        load_in_4bit=True,
        cache_dir=str(HF_CACHE_DIR),
    )

    model.save_pretrained_gguf(
        str(temp_export_dir),
        tokenizer,
        quantization_method="q4_k_m",
    )

    found_ggufs = list(temp_export_dir.rglob("*.gguf"))
    if not found_ggufs and Path(f"{temp_export_dir}_gguf").exists():
```

```
        found_ggufs = list(Path(f"{temp_export_dir}_gguf").rglob("*.gguf"))

    if found_ggufs:
        final_gguf_path = GGUF_DIR / target_gguf_name
        shutil.move(str(found_ggufs[0]), str(final_gguf_path))
        print(f">>> [OK] Binario GGUF listo en: {final_gguf_path}")
    else:
        print(f">>> [ADVERTENCIA] Revisa la carpeta {temp_export_dir} para localizar el archivo .gguf")

    del model
    del tokenizer
    gc.collect()
    torch.cuda.empty_cache()

QWEN_TEMPLATE = """{{- range .Messages }}<|im_start|>{{ .Role }}
{{ .Content }}<|im_end|>
{{ end }}<|im_start|>assistant
"""

GEMMA_TEMPLATE = """{{- $system := "" }}
{{- range .Messages }}
{{- if eq .Role "system" }}{{ $system = .Content }}{{ continue }}
{{- else if eq .Role "user" }}<start_of_turn>user
{{- if $system }}
{{ $system }}

{{- $system = "" }}
{{- end }}
{{- else if eq .Role "assistant" }}<start_of_turn>model
{{- end }}
{{ .Content }}<end_of_turn>
{{ end }}<start_of_turn>model
"""


def _modelfile(gguf_name: str, template: str, stops: list[str]) -> str:
    """V3: TEMPLATE y stops explícitos. Antes los Modelfiles solo tenían FROM y Ollama tenía
    que adivinar la plantilla a partir del GGUF."""
    lines = [f"FROM ./{gguf_name}", f'TEMPLATE """{template}"""']
```

```
    lines += [f'PARAMETER stop "{s}"' for s in stops]
    lines += ["PARAMETER temperature 0.1", "PARAMETER repeat_penalty 1.15", "PARAMETER num_ctx 1024"]
    return "\n".join(lines) + "\n"


def write_clean_modelfiles() -> None:
    """Modelfiles UTF-8 sin BOM (evita el 400 Bad Request de Ollama)."""
    files = {
        "Modelfile.qwen-v3": _modelfile("vertex-chess-qwen-v3-q4_k_m.gguf", QWEN_TEMPLATE, ["<|im_end|>", "<|endoftext|>"]),
        "Modelfile.gemma": _modelfile("vertex-chess-gemma-q4_k_m.gguf", GEMMA_TEMPLATE, ["<end_of_turn>", "<start_of_turn>"]),
    }
    for name, content in files.items():
        with open(GGUF_DIR / name, "w", encoding="utf-8", newline="\n") as f:
            f.write(content)

    print("\n>>> Modelfiles generados. Para registrar en Ollama (desde E:\\vertex-chess-arena\\models\\gguf):")
    print("    ollama create vertexqwen-v3 -f Modelfile.qwen-v3")
    print("    (vertexgemma ya está registrado; Modelfile.gemma queda como referencia)")


if __name__ == "__main__":
    # V3: solo se exporta el Qwen nuevo. El GGUF de Gemma ya existe y no se toca, para que
    # el Gemma del duelo sea exactamente el mismo que ya jugó.
    if not (GGUF_DIR / "vertex-chess-qwen-v3-q4_k_m.gguf").exists():
        export_contender("qwen_chess_adapter_v3", "vertex-chess-qwen-v3-q4_k_m.gguf")
    if not (GGUF_DIR / "vertex-chess-gemma-q4_k_m.gguf").exists():
        export_contender("gemma_chess_adapter", "vertex-chess-gemma-q4_k_m.gguf")
    write_clean_modelfiles()
```

APÉNDICE G

El árbitro

El archivo más largo del proyecto y el protagonista del libro. Para no perderse, conviene leerlo en este orden:

1. **Las constantes del principio**: `SYSTEM_PROMPT` (el del entrenamiento), `LEGACY_SYSTEM_PROMPT` (el que usaba la arena antes de la auditoría), los modos `arbiter`/`pure` y los respaldos `random`/`engine`. El tercer modo de la app, el motor jugando solo, lo resuelve la API (apéndice I) llamando directamente a la búsqueda.
2. **`request_model_move`**, al final: es la puerta de entrada. Pide las opciones, construye el prompt, llama a Ollama, interpreta la respuesta y, si falla, aplica el respaldo.
3. **`get_options`**: en modo `pure` devuelve todas las jugadas legales; en modo `arbiter`, los candidatos.
4. **`_get_candidate_moves`**: mate en una, búsqueda a profundidad 2, hasta 4 jugadas a menos de 180 centipeones de la mejor, orden de *mop-up* en finales ganados y filtro anti-repetición (capítulo 7).
5. **`parse_llm_move`**: el parser estricto V3. Su *docstring* explica qué dejó de aceptar.
6. **`_fallback_move`**: tres líneas que resumen la auditoría del capítulo 8. Con `random`, fallar no se premia.
7. **La búsqueda**: `_search_root_moves`, `_search_minimax` (alfa-beta) y `_quiescence`.
8. **La evaluación**: `_static_material_and_board_eval`, `_evaluate_move` y las ayudas de finales (`_mop_up_endgame_bonus`, `_is_mop_up_mode`).

Todo lo que está por encima de `get_options` es *el motor*: lo que juega. Todo lo que está por debajo es *la interfaz con el modelo*: lo que mide.

arena/engine_arbiter.py

```
import re
import json
```

```
import random
import urllib.request
from collections import deque
import chess

# V3 (2026-10-20): la arena usa EXACTAMENTE el system prompt y el formato de usuario del
# dataset de entrenamiento (dataset/processed/train_chess_elite.jsonl). Antes la arena usaba
# otro system prompt, una línea extra "Side to move" y solo 4 candidatas, y eso sacaba a los
# modelos de la distribución que aprendieron.
SYSTEM_PROMPT = (
    "You are an elite chess grandmaster engine created by Vertex Coders. Analyze the FEN board "
    "state and the list of legal moves, then output the single strongest tactical move in both "
    "SAN and UCI formats."
)
LEGACY_SYSTEM_PROMPT = (
    "You are Vertex-Chess, a Grandmaster-level chess engine developed by Vertex Coders LLC. "
    "Given the current board state in FEN notation, the side to move, and the list of legal moves, "
    "select the strongest winning move. Respond strictly with the chosen move in SAN and UCI format."
)

# Modos de juego:
#   "arbiter" -> el motor (alpha-beta prof. 2 + quiescencia) filtra hasta 4 candidatas y el LLM elige.
#                Mide al sistema híbrido motor+LLM.
#   "pure"    -> el LLM recibe TODAS las jugadas legales, igual que en el entrenamiento.
#                Mide al modelo. Es el modo justo para el duelo de arquitecturas.
MODES = ("arbiter", "pure")
# Fallback cuando la respuesta no se puede validar:
#   "random" -> jugada aleatoria entre las opciones ofrecidas (fallar NO se premia)
#   "engine" -> la mejor del motor (comportamiento viejo: fallar = jugar como el motor)
```

```
FALLBACKS = ("random", "engine")

PIECE_VALUES = {
    chess.PAWN: 100,
    chess.KNIGHT: 320,
    chess.BISHOP: 330,
    chess.ROOK: 500,
    chess.QUEEN: 950,
    chess.KING: 20000,
}

CENTER_CORE = {chess.E4, chess.D4, chess.E5, chess.D5}
CENTER_EXT = {
    chess.C3, chess.D3, chess.E3, chess.F3,
    chess.C4, chess.F4,
    chess.C5, chess.F5,
    chess.C6, chess.D6, chess.E6, chess.F6,
}
RIM_SQUARES = {
    chess.A1, chess.A2, chess.A3, chess.A4, chess.A5, chess.A6, chess.A7,
↪   chess.A8,
    chess.H1, chess.H2, chess.H3, chess.H4, chess.H5, chess.H6, chess.H7,
↪   chess.H8,
}

STARTING_MINORS = {
    chess.WHITE: (chess.B1, chess.G1, chess.C1, chess.F1),
    chess.BLACK: (chess.B8, chess.G8, chess.C8, chess.F8),
}
OPTIMAL_CASTLED_KING = {
    chess.WHITE: (chess.G1, chess.C1, chess.B1),
    chess.BLACK: (chess.G8, chess.C8, chess.B8),
}
CORNER_CASTLED_KING = {
    chess.WHITE: (chess.H1, chess.A1),
    chess.BLACK: (chess.H8, chess.A8),
}


class ChessArbiter:
    def __init__(
        self,
```

```
        ollama_url: str = "http://127.0.0.1:11434",
        mode: str = "arbiter",
        fallback: str = "random",
        seed: int | None = None,
    ):
        if mode not in MODES:
            raise ValueError(f"mode debe ser uno de {MODES}")
        if fallback not in FALLBACKS:
            raise ValueError(f"fallback debe ser uno de {FALLBACKS}")
        self.ollama_url = ollama_url
        self.mode = mode
        self.fallback = fallback
        self.rng = random.Random(seed)
        self.recent_positions: deque[str] = deque(maxlen=80)
        self.last_piece_squares: dict[bool, deque[int]] = {
            chess.WHITE: deque(maxlen=4),
            chess.BLACK: deque(maxlen=4),
        }
        self.consecutive_checks: dict[bool, int] = {
            chess.WHITE: 0,
            chess.BLACK: 0,
        }

    def _pos_key(self, board: chess.Board) -> str:
        parts = board.fen().split(" ")
        return " ".join(parts[:3])

    def _chebyshev_dist(self, sq1: int, sq2: int) -> int:
        return max(
            abs(chess.square_file(sq1) - chess.square_file(sq2)),
            abs(chess.square_rank(sq1) - chess.square_rank(sq2)),
        )

    def _manhattan_dist(self, sq1: int, sq2: int) -> int:
        return (
            abs(chess.square_file(sq1) - chess.square_file(sq2))
            + abs(chess.square_rank(sq1) - chess.square_rank(sq2))
        )

    def _edge_distance_bonus(self, sq: int) -> int:
        f = chess.square_file(sq)
        r = chess.square_rank(sq)
```

```
        dist_f = min(f, 7 - f)
        dist_r = min(r, 7 - r)
        return (6 - (dist_f + dist_r)) * 60

    def _get_side_material(self, board: chess.Board, color: bool) -> int:
        return sum(
            PIECE_VALUES[pc.piece_type]
            for pc in board.piece_map().values()
            if pc.color == color and pc.piece_type != chess.KING
        )

    def _is_mop_up_mode(self, board: chess.Board, perspective: bool) ->
    ↪  bool:
        piece_map = board.piece_map()
        if len(piece_map) > 10:
            return False
        my_mat = self._get_side_material(board, perspective)
        enemy_mat = self._get_side_material(board, not perspective)
        return (my_mat - enemy_mat) >= 350 and enemy_mat <= 100

    def _king_danger_penalty(self, board: chess.Board, king_sq: int,
    ↪  defender_color: bool) -> int:
        attacker_color = not defender_color
        danger = 0
        for adj_sq in board.attacks(king_sq):
            enemy_attackers = board.attackers(attacker_color, adj_sq)
            if enemy_attackers:
                my_defenders = board.attackers(defender_color, adj_sq)
                if len(enemy_attackers) >= len(my_defenders):
                    danger += 45 * len(enemy_attackers)
        return danger

    def _piece_threat_loss(self, board: chess.Board, sq: int, color:
    ↪  bool) -> int:
        """
        Calcula cuántos puntos estamos en riesgo de perder en la casilla
↪  'sq' si una pieza nuestra
        está atacada por una pieza enemiga de menor valor (ej. Peón ataca
↪  Alfil en d6 -> pierde 230;
        Alfil ataca Torre en f8 -> pierde 170) o si está completamente
↪  indefensa.
        """
```

```
        pc = board.piece_at(sq)
        if not pc or pc.color != color or pc.piece_type == chess.KING:
            return 0
        my_val = PIECE_VALUES[pc.piece_type]
        enemy_color = not color
        attackers = board.attackers(enemy_color, sq)
        if not attackers:
            return 0

        min_attacker_val = min(
            PIECE_VALUES[att_pc.piece_type]
            for att_sq in attackers
            if (att_pc := board.piece_at(att_sq)) is not None
        )

        # Si la pieza no está defendida por nadie, está colgada entera
        if not board.is_attacked_by(color, sq):
            return my_val

        # Si está defendida, pero nos la ataca una pieza más barata (ej.
        ↪  9.c5 contra Bd6, o 17.Bxd6 contra Rf8)
        if min_attacker_val < my_val:
            return my_val - min_attacker_val

        return 0

    def _has_any_threatened_piece(self, board: chess.Board, color: bool)
    ↪  -> bool:
        for sq, pc in board.piece_map().items():
            if pc.color == color and pc.piece_type != chess.KING:
                if self._piece_threat_loss(board, sq, color) >= 150:
                    return True
        return False

    def _static_material_and_board_eval(
        self,
        board: chess.Board,
        perspective: bool,
        was_winning_before: bool = False,
        allow_mop_up_eval: bool = False,
    ) -> int:
        if board.is_checkmate():
```

```
            return -100000 if board.turn == perspective else 100000

        if board.is_stalemate() or board.is_insufficient_material():
            return -80000 if was_winning_before else 0

        piece_map = board.piece_map()
        queens_count = sum(1 for pc in piece_map.values() if
↪   pc.piece_type == chess.QUEEN)
        is_endgame = (queens_count == 0) or (len(piece_map) <= 10)

        my_mat = 0
        enemy_mat = 0
        score = 0
        enemy_pawns_squares: list[int] = []

        for sq, pc in piece_map.items():
            if pc.piece_type == chess.KING:
                continue

            val = PIECE_VALUES[pc.piece_type]
            rank = chess.square_rank(sq)
            file_idx = chess.square_file(sq)
            back_rank = 0 if pc.color == chess.WHITE else 7
            seventh_rank = 6 if pc.color == chess.WHITE else 1

            if pc.color == perspective:
                my_mat += val
            else:
                enemy_mat += val
                if pc.piece_type == chess.PAWN:
                    enemy_pawns_squares.append(sq)

            if sq in CENTER_CORE:
                val += 28
            elif sq in CENTER_EXT:
                val += 16
            elif pc.piece_type == chess.KNIGHT and sq in RIM_SQUARES:
                val -= 65
            elif pc.piece_type == chess.BISHOP and sq in RIM_SQUARES:
                val -= 30

            if not is_endgame:
```

```
                if pc.piece_type in (chess.KNIGHT, chess.BISHOP) and rank
                ↪  == back_rank:
                    val -= 45
                elif pc.piece_type == chess.QUEEN and rank == back_rank
                ↪  and sq not in (chess.D1, chess.D8, chess.E1,
                ↪  chess.E8):
                    val -= 65

            if not is_endgame and pc.piece_type in (chess.QUEEN,
            ↪  chess.KNIGHT, chess.BISHOP):
                enemy_col = not pc.color
                safe_mobility = 0
                for att_sq in board.attacks(sq):
                    occupant = board.piece_at(att_sq)
                    if occupant and occupant.color == pc.color:
                        continue
                    attacked_by_cheaper = any(
                        (ep := board.piece_at(esq)) and
↪  PIECE_VALUES[ep.piece_type] < val
                        for esq in board.attackers(enemy_col, att_sq)
                    )
                    if not attacked_by_cheaper:
                        safe_mobility += 1
                if safe_mobility == 0:
                    val -= 140
                elif safe_mobility == 1:
                    val -= 60

            if pc.piece_type == chess.ROOK:
                if rank == seventh_rank:
                    val += 80
                if file_idx in (3, 4):
                    val += 28

            if pc.piece_type in (chess.KNIGHT, chess.BISHOP, chess.ROOK,
            ↪  chess.QUEEN):
                if not board.is_attacked_by(pc.color, sq):
                    val -= 30

            if pc.piece_type == chess.PAWN and (is_endgame or
            ↪  len(piece_map) <= 14):
```

```
                advancement = rank if pc.color == chess.WHITE else (7 -
↪  rank)
                val += (advancement * advancement) * 12

            if pc.color == perspective:
                score += val
            else:
                score -= val

        my_king = board.king(perspective)
        enemy_king = board.king(not perspective)

        if not is_endgame and my_king is not None and enemy_king is not
        ↪  None:
            if my_king in OPTIMAL_CASTLED_KING[perspective]:
                score += 60
            elif my_king in CORNER_CASTLED_KING[perspective]:
                score += 20
            elif chess.square_file(my_king) in (3, 4):
                score -= 35

            if enemy_king in OPTIMAL_CASTLED_KING[not perspective]:
                score -= 60
            elif enemy_king in CORNER_CASTLED_KING[not perspective]:
                score -= 20
            elif chess.square_file(enemy_king) in (3, 4):
                score += 35

            score -= self._king_danger_penalty(board, my_king,
↪  perspective)
            score += self._king_danger_penalty(board, enemy_king, not
↪  perspective)

        if is_endgame and my_king is not None and enemy_king is not None:
            if allow_mop_up_eval:
                score += self._edge_distance_bonus(enemy_king)
                k_cheb = self._chebyshev_dist(my_king, enemy_king)
                k_manh = self._manhattan_dist(my_king, enemy_king)
                score += (8 - k_cheb) * 110 + (14 - k_manh) * 45
            else:
                for sq_k, col_k in ((my_king, perspective), (enemy_king,
                ↪  not perspective)):
```

```
                    king_bonus = 0
                    if sq_k in CENTER_CORE:
                        king_bonus = 40
                    elif sq_k in CENTER_EXT:
                        king_bonus = 20
                    elif sq_k in RIM_SQUARES:
                        king_bonus = -30
                    if col_k == perspective:
                        score += king_bonus
                    else:
                        score -= king_bonus

                if enemy_pawns_squares:
                    min_dist_to_pawn = min(self._chebyshev_dist(my_king,
↪  psq) for psq in enemy_pawns_squares)
                    score += (8 - min_dist_to_pawn) * 45

        return score

    def _worst_enemy_tactical_reply_loss(self, board: chess.Board,
    ↪  my_color: bool) -> int:
        if board.is_game_over():
            return 0

        worst_delta = 0

        for reply in board.legal_moves:
            is_cap = board.is_capture(reply)
            gives_chk = board.gives_check(reply)
            if not (is_cap or gives_chk):
                continue

            victim = board.piece_at(reply.to_square)
            attacker = board.piece_at(reply.from_square)
            victim_val = PIECE_VALUES.get(victim.piece_type, 100) if
↪  victim else 0
            attacker_val = PIECE_VALUES.get(attacker.piece_type, 100) if
↪  attacker else 100

            board.push(reply)
            if board.is_checkmate():
                board.pop()
```

```
                return -90000

            can_legally_recapture = any(
                m.to_square == reply.to_square for m in board.legal_moves
            )

            fork_loss = 0
            if gives_chk:
                for attacked_sq in board.attacks(reply.to_square):
                    target_pc = board.piece_at(attacked_sq)
                    if (
                        target_pc
                        and target_pc.color == my_color
                        and target_pc.piece_type != chess.KING
                    ):
                        if not board.is_attacked_by(my_color,
                        ↪  attacked_sq):
                            fork_loss = max(fork_loss,
↪  PIECE_VALUES[target_pc.piece_type] - 80)
                        elif attacker_val <
                        ↪  PIECE_VALUES[target_pc.piece_type]:
                            fork_loss = max(fork_loss,
↪  PIECE_VALUES[target_pc.piece_type] - attacker_val)

            board.pop()

            if is_cap:
                if can_legally_recapture:
                    net_loss = max(0, victim_val - attacker_val)
                else:
                    net_loss = victim_val
                if gives_chk and net_loss > 0:
                    net_loss += 180
            else:
                net_loss = 0

            # Factor 1.45x para que perder un Alfil por un peón
            ↪  (9...exd4?? 10.cxd6) o una Torre por un Alfil
            # (17...Rc8?? 18.Bxf8) jamás pueda ser compensado por comer
            ↪  un peón suelto en otro lado.
            total_tactical_loss = int((net_loss + fork_loss) * 1.45)
            if -total_tactical_loss < worst_delta:
```

```
                worst_delta = -total_tactical_loss

        return worst_delta

    def _is_move_checkmate(self, board: chess.Board, mv: chess.Move) ->
    ↪  bool:
        board.push(mv)
        is_mate = board.is_checkmate()
        board.pop()
        return is_mate

    def _mop_up_endgame_bonus(
        self,
        board_before: chess.Board,
        mv: chess.Move,
        board_after: chess.Board,
        my_color: bool,
    ) -> int:
        if board_after.is_stalemate():
            return -95000

        my_king_before = board_before.king(my_color)
        my_king_after = board_after.king(my_color)
        enemy_king = board_after.king(not my_color)
        if my_king_before is None or my_king_after is None or enemy_king
        ↪  is None:
            return 0

        bonus = 0
        piece = board_before.piece_at(mv.from_square)
        dist_before = self._chebyshev_dist(my_king_before, enemy_king)
        dist_after = self._chebyshev_dist(my_king_after, enemy_king)
        manh_before = self._manhattan_dist(my_king_before, enemy_king)
        manh_after = self._manhattan_dist(my_king_after, enemy_king)

        enemy_replies = list(board_after.legal_moves)
        if enemy_replies:
            all_lead_to_mate = True
            mate_in_next_count = 0
            for rep in enemy_replies:
                board_after.push(rep)
                has_mate = any(
```

```
                    board_after.gives_check(m2) and
↪ self._is_move_checkmate(board_after, m2)
                    for m2 in board_after.legal_moves
                )
                board_after.pop()
                if has_mate:
                    mate_in_next_count += 1
                else:
                    all_lead_to_mate = False

            if all_lead_to_mate:
                return 50000
            bonus += mate_in_next_count * 120

        if piece and piece.piece_type == chess.KING:
            if dist_after < dist_before:
                bonus += 320
            elif dist_after == dist_before and manh_after < manh_before:
                bonus += 180
            elif dist_after > dist_before:
                bonus -= 300

        elif piece and piece.piece_type in (chess.QUEEN, chess.ROOK,
        ↪ chess.BISHOP):
            gives_chk = board_after.is_check()
            if gives_chk and dist_after > 2:
                bonus -= 350

            if piece.piece_type == chess.QUEEN and not gives_chk:
                df = abs(chess.square_file(mv.to_square) -
↪ chess.square_file(enemy_king))
                dr = abs(chess.square_rank(mv.to_square) -
↪ chess.square_rank(enemy_king))
                if (df == 1 and dr == 2) or (df == 2 and dr == 1):
                    bonus += 240

            num_escapes = len(enemy_replies)
            if 1 <= num_escapes <= 3 and not gives_chk:
                bonus += (5 - num_escapes) * 65

        return bonus

```

```
    def _evaluate_move(self, board: chess.Board, mv: chess.Move) -> int:
        my_color = board.turn
        ply = board.fullmove_number
        piece = board.piece_at(mv.from_square)
        if not piece:
            return -99999

        my_mat_before = self._get_side_material(board, my_color)
        enemy_mat_before = self._get_side_material(board, not my_color)
        mat_diff = my_mat_before - enemy_mat_before
        was_winning_before = mat_diff >= 200

        mop_up = self._is_mop_up_mode(board, my_color)
        before_eval = self._static_material_and_board_eval(
            board, my_color, was_winning_before, allow_mop_up_eval=mop_up
        )

        # Cuánto material estaba en riesgo en mv.from_square antes de
        ↪  mover (ej. 9.c5 contra Bd6 -> 230 pts; 17.Bxd6 contra Rf8 ->
        ↪  170 pts)
        threat_loss_here = self._piece_threat_loss(board, mv.from_square,
↪   my_color)
        was_threatened_here = threat_loss_here >= 120

        victim_before = board.piece_at(mv.to_square)
        my_piece_val = PIECE_VALUES.get(piece.piece_type, 100)
        victim_before_val = PIECE_VALUES.get(victim_before.piece_type, 0)
↪   if victim_before else 0

        simplification_bonus = 0
        if victim_before and victim_before.piece_type == piece.piece_type
        ↪  and piece.piece_type in (chess.QUEEN, chess.ROOK):
            if mat_diff >= 250:
                simplification_bonus = 180
            elif mat_diff <= -250:
                simplification_bonus = -240

        promotion_bonus = 600 if mv.promotion == chess.QUEEN else 0

        back_rank = 0 if my_color == chess.WHITE else 7
        retreat_penalty = 0
        if (
```

```
            not mop_up
            and not was_threatened_here  # Si la pieza está amenazada, permitirle retroceder a la 1ra fila si hace falta
            and piece.piece_type in (chess.KNIGHT, chess.BISHOP)
            and chess.square_rank(mv.from_square) != back_rank
            and chess.square_rank(mv.to_square) == back_rank
            and not board.is_capture(mv)
        ):
            retreat_penalty = -165

        recent_squares = self.last_piece_squares[my_color]
        same_piece_penalty = 0
        if (
            mv.from_square in recent_squares
            and not board.is_capture(mv)
            and not was_threatened_here
            and not (mop_up and piece.piece_type == chess.KING)
        ):
            if board.gives_check(mv):
                if self.consecutive_checks[my_color] >= 1:
                    same_piece_penalty = -280 * self.consecutive_checks[my_color]
                else:
                    same_piece_penalty = -90
            else:
                same_piece_penalty = -220

        opening_bonus = 0
        my_king_sq = board.king(my_color)
        is_king_castled = (
            my_king_sq in OPTIMAL_CASTLED_KING[my_color] or my_king_sq in CORNER_CASTLED_KING[my_color]
            if my_king_sq is not None
            else False
        )
        undeveloped_minors = sum(
            1
            for sq in STARTING_MINORS[my_color]
            if (pc := board.piece_at(sq)) and pc.color == my_color and pc.piece_type in (chess.KNIGHT, chess.BISHOP)
        )

```

```
        if ply <= 14 and not mop_up:
            if piece.piece_type == chess.PAWN:
                if mv.to_square in CENTER_CORE:
                    opening_bonus += 85
                elif mv.to_square in (chess.C4, chess.C5, chess.E3,
                ↪  chess.D3, chess.E6, chess.D6):
                    opening_bonus += 35
                elif chess.square_file(mv.from_square) in (0, 1, 5, 6, 7)
                ↪  and not board.is_capture(mv):
                    if not is_king_castled or undeveloped_minors >= 2:
                        opening_bonus -= 220

            elif piece.piece_type in (chess.KNIGHT, chess.BISHOP):
                if mv.from_square in STARTING_MINORS[my_color]:
                    opening_bonus += 105
                if mv.to_square in CENTER_EXT or mv.to_square in
                ↪  CENTER_CORE:
                    opening_bonus += 45
                if mv.to_square in RIM_SQUARES:
                    opening_bonus -= 180

            elif piece.piece_type == chess.ROOK and not
            ↪  board.is_castling(mv) and not board.is_capture(mv):
                opening_bonus -= 300
            elif piece.piece_type == chess.KING and not
            ↪  board.is_castling(mv):
                opening_bonus -= 350
            elif piece.piece_type == chess.QUEEN and ply <= 6 and not
            ↪  board.is_capture(mv):
                opening_bonus -= 140

        if board.is_castling(mv):
            opening_bonus += 160

        board.push(mv)
        if board.is_checkmate():
            board.pop()
            return 100000

        is_rep = self._pos_key(board) in self.recent_positions
        after_eval = self._static_material_and_board_eval(
            board, my_color, was_winning_before, allow_mop_up_eval=mop_up
```

```
        )
        material_gain = after_eval - before_eval

        enemy_can_recapture_landing = any(
            r.to_square == mv.to_square for r in board.legal_moves
        )
        bad_trade_penalty = 0
        if enemy_can_recapture_landing and my_piece_val >
        ↪  victim_before_val + 50:
            bad_trade_penalty = -int((my_piece_val - victim_before_val) *
↪  1.35)

        # Bono proporcional por salvar la pieza amenazada (ej. salvar Bd6
        ↪  de 9.c5 o salvar Rf8 de 17.Bxd6)
        evacuation_bonus = 0
        if was_threatened_here and self._piece_threat_loss(board,
        ↪  mv.to_square, my_color) == 0 and bad_trade_penalty == 0:
            evacuation_bonus = int(threat_loss_here * 0.85) + 60

        # Seguridad de la Dama: penalizar poner la Dama a salto de
        ↪  caballo (dist 1,2) de un Caballo enemigo activo
        # que pueda meterle un jaque doble al Rey (corrige 49...Qe3??
        ↪  permitiendo 51.Nf5+!)
        queen_fork_danger = 0
        if piece.piece_type == chess.QUEEN and not mop_up:
            enemy_col = not my_color
            for esq, epc in board.piece_map().items():
                if epc.color == enemy_col and epc.piece_type ==
                ↪  chess.KNIGHT:
                    # Si un caballo enemigo puede saltar en 1 o 2 jugadas
                    ↪  a una casilla que ataca nuestra Dama y da jaque
                    for knight_jump in board.attacks(esq):
                        if mv.to_square in board.attacks(knight_jump):
                            queen_fork_danger -= 75
                            break

        clean_capture_bonus = 0
        if victim_before_val >= 300:
            if not enemy_can_recapture_landing:
                clean_capture_bonus = 140
            elif my_piece_val <= victim_before_val:
                clean_capture_bonus = 95
```

```
599
600         enemy_punishment = self._worst_enemy_tactical_reply_loss(board,
    ↪  my_color)
601         gives_chk = board.is_check()
602         board.pop()
603
604         mop_up_bonus = 0
605         if mop_up and enemy_punishment == 0 and bad_trade_penalty == 0:
606             b_after = board.copy()
607             b_after.push(mv)
608             mop_up_bonus = self._mop_up_endgame_bonus(board, mv, b_after,
    ↪  my_color)
609
610         total_score = (
611             material_gain
612             + enemy_punishment
613             + bad_trade_penalty
614             + clean_capture_bonus
615             + evacuation_bonus
616             + queen_fork_danger
617             + simplification_bonus
618             + promotion_bonus
619             + retreat_penalty
620             + opening_bonus
621             + same_piece_penalty
622             + mop_up_bonus
623         )
624
625         if gives_chk and is_rep:
626             total_score -= 12000
627         elif is_rep:
628             total_score -= 10000
629
630         return total_score
631
632     def _search_eval(self, board: chess.Board, perspective: bool) -> int:
633         """Evaluación ligera para búsqueda táctica; positiva favorece a
        ↪  perspective."""
634         if board.is_checkmate():
635             return -100000 if board.turn == perspective else 100000
636         if board.is_stalemate() or board.is_insufficient_material() or
        ↪  board.is_fivefold_repetition():
```

```
            return 0

        score = 0
        for sq, pc in board.piece_map().items():
            if pc.piece_type == chess.KING:
                continue
            value = PIECE_VALUES[pc.piece_type]
            rank = chess.square_rank(sq)
            file_idx = chess.square_file(sq)
            if pc.piece_type == chess.PAWN:
                advance = rank if pc.color == chess.WHITE else 7 - rank
                value += advance * 8
            if sq in CENTER_CORE:
                value += 18
            elif sq in CENTER_EXT:
                value += 8
            if pc.piece_type == chess.KNIGHT and sq in RIM_SQUARES:
                value -= 25
            sign = 1 if pc.color == perspective else -1
            score += sign * value

        # Preferimos seguridad del rey y penalizamos el jaque al bando
        ↪ evaluado.
        if board.is_check():
            score += -35 if board.turn == perspective else 35
        return score

    def _quiescence(self, board: chess.Board, alpha: int, beta: int,
                    perspective: bool, qdepth: int = 4) -> int:
        """Extiende la búsqueda en posiciones tácticas con capturas y
        ↪ promociones."""
        stand_pat = self._search_eval(board, perspective)
        maximizing = board.turn == perspective
        if board.is_game_over():
            if board.is_checkmate():
                return -100000 if board.turn == perspective else 100000
            return 0
        if qdepth <= 0:
            return stand_pat

        in_check = board.is_check()
```

```
        # En jaque no existe "stand pat": hay que buscar TODAS las evasiones legales,
        # incluidas las que no capturan ni dan jaque.
        if not in_check:
            if maximizing:
                if stand_pat >= beta:
                    return beta
                alpha = max(alpha, stand_pat)
            else:
                if stand_pat <= alpha:
                    return alpha
                beta = min(beta, stand_pat)

        tactical = (
            list(board.legal_moves) if in_check else [
                m for m in board.legal_moves
                if board.is_capture(m) or m.promotion is not None or board.gives_check(m)
            ]
        )
        if not tactical:
            return stand_pat
        tactical.sort(
            key=lambda m: (
                1 if m.promotion else 0,
                PIECE_VALUES.get(
                    board.piece_at(m.to_square).piece_type, 0
                ) if board.is_capture(m) and board.piece_at(m.to_square) else 0,
                1 if board.gives_check(m) else 0,
            ),
            reverse=True,
        )

        if maximizing:
            value = stand_pat
            for move in tactical:
                board.push(move)
                value = max(value, self._quiescence(board, alpha, beta, perspective, qdepth - 1))
                board.pop()
                alpha = max(alpha, value)
```

```
                if alpha >= beta:
                    break
            return value
        else:
            value = stand_pat
            for move in tactical:
                board.push(move)
                value = min(value, self._quiescence(board, alpha, beta,
↪  perspective, qdepth - 1))
                board.pop()
                beta = min(beta, value)
                if beta <= alpha:
                    break
            return value

    def _search_root_moves(self, board: chess.Board, depth: int = 2) ->
↪  list[tuple[int, chess.Move]]:
        """Búsqueda minimax con poda alpha-beta; devuelve las jugadas
        ↪  mejor puntuadas."""
        perspective = board.turn
        scored = []
        moves = list(board.legal_moves)

        # Primero mate en una; no permitimos que las heurísticas lo
        ↪  oculten.
        for move in moves:
            board.push(move)
            mate = board.is_checkmate()
            board.pop()
            if mate:
                return [(100000, move)]

        # Orden táctico para encontrar antes capturas, jaques y
        ↪  promociones.
        def order_key(move):
            victim = board.piece_at(move.to_square)
            attacker = board.piece_at(move.from_square)
            capture_value = PIECE_VALUES.get(victim.piece_type, 0) if
↪  victim else 0
            attacker_value = PIECE_VALUES.get(attacker.piece_type, 100)
↪  if attacker else 100
            return (
```

```
                1 if move.promotion else 0,
                1 if board.gives_check(move) else 0,
                capture_value - attacker_value // 10 if board.is_capture(move) else 0,
            )

        moves.sort(key=order_key, reverse=True)
        for move in moves:
            board.push(move)
            if board.is_checkmate():
                value = 100000
            elif board.is_game_over():
                value = 0
            else:
                value = self._search_minimax(
                    board, depth - 1, -1000000, 1000000, perspective
                )
            board.pop()
            scored.append((value, move))

        scored.sort(key=lambda item: item[0], reverse=True)
        return scored

    def _search_minimax(self, board: chess.Board, depth: int, alpha: int,
                        beta: int, perspective: bool) -> int:
        if board.is_game_over():
            if board.is_checkmate():
                return -100000 if board.turn == perspective else 100000
            return 0
        if depth <= 0:
            return self._quiescence(board, alpha, beta, perspective)

        maximizing = board.turn == perspective
        moves = list(board.legal_moves)
        moves.sort(
            key=lambda m: (
                1 if m.promotion else 0,
                1 if board.is_capture(m) else 0,
                1 if board.gives_check(m) else 0,
            ),
            reverse=True,
        )
```

```
790
791         if maximizing:
792             value = -1000000
793             for move in moves:
794                 board.push(move)
795                 child = self._search_minimax(board, depth - 1, alpha,
    ↪  beta, perspective)
796                 board.pop()
797                 value = max(value, child)
798                 alpha = max(alpha, value)
799                 if alpha >= beta:
800                     break
801             return value
802         else:
803             value = 1000000
804             for move in moves:
805                 board.push(move)
806                 child = self._search_minimax(board, depth - 1, alpha,
    ↪  beta, perspective)
807                 board.pop()
808                 value = min(value, child)
809                 beta = min(beta, value)
810                 if beta <= alpha:
811                     break
812             return value
813
814     def _get_candidate_moves(self, board: chess.Board) ->
    ↪  list[chess.Move]:
815         """Combina heurísticas existentes con una búsqueda táctica
        ↪  independiente."""
816         all_legal = list(board.legal_moves)
817         if len(all_legal) <= 1:
818             return all_legal
819
820         # Mate inmediato siempre tiene prioridad absoluta.
821         for move in all_legal:
822             board.push(move)
823             is_mate = board.is_checkmate()
824             board.pop()
825             if is_mate:
826                 return [move]
827
```

```
828         # La búsqueda evita que las bonificaciones manuales dominen
            ↪  tácticas evidentes.
829         # Profundidad 2 + quiescencia: coste moderado para una partida
            ↪  interactiva.
830         search_scores = self._search_root_moves(board, depth=2)
831         if not search_scores:
832             return all_legal[:1]
833 
834         best_search = search_scores[0][0]
835         # Conservamos hasta 4 candidatas cercanas a la mejor puntuación
            ↪  táctica.
836         # El LLM decide entre opciones razonables, pero no entre jugadas
            ↪  claramente inferiores.
837         acceptable = [move for score, move in search_scores if score >=
    ↪  best_search - 180]
838 
839         # V3: en finales ganados (mop-up) la búsqueda de profundidad 2 no
            ↪  ve el plan de mate,
840         # así que ordenamos las jugadas aceptables con _evaluate_move
            ↪  (caja de la Dama, marcha
841         # del Rey, mate en 2, anti-ahogado). Antes _evaluate_move no se
            ↪  llamaba en ningún sitio.
842         if self._is_mop_up_mode(board, board.turn):
843             acceptable.sort(key=lambda m: self._evaluate_move(board, m),
    ↪  reverse=True)
844 
845         # V3: anti-bucles. Si hay alternativas, descartamos jugadas que
            ↪  repiten una posición
846         # reciente (evita series tipo Ra7+ Ra8+ Ra7+ ... o Nf8+ Nd7+
            ↪  ...).
847         fresh = [m for m in acceptable if not self._repeats_recent(board,
    ↪  m)]
848         search_candidates = (fresh or acceptable)[:4]
849 
850         # Si la búsqueda detecta que todas las opciones pierden material,
            ↪  no recortamos
851         # a una sola jugada por heurísticas ruidosas: mantenemos varias
            ↪  defensas.
852         return search_candidates or [search_scores[0][1]]
853 
854     def _repeats_recent(self, board: chess.Board, mv: chess.Move) ->
        ↪  bool:
```

```
855         board.push(mv)
856         key = self._pos_key(board)
857         board.pop()
858         return key in self.recent_positions
859 
860     def get_options(self, board: chess.Board, mode: str | None = None) ->
    ↪  list[chess.Move]:
861         """Jugadas que se le ofrecen al LLM según el modo."""
862         mode = mode or self.mode
863         if mode == "pure":
864             return list(board.legal_moves)
865         return self._get_candidate_moves(board)
866 
867     def build_prompt(self, board: chess.Board, candidate_moves:
    ↪  list[chess.Move]) -> str:
868         """Formato idéntico al del dataset de entrenamiento."""
869         legal_san = [board.san(m) for m in candidate_moves]
870         return (
871             f"Position FEN: {board.fen()}\n"
872             f"Legal moves (SAN): {', '.join(legal_san)}\n"
873             f"Select the best move."
874         )
875 
876     @staticmethod
877     def _norm_san(san: str) -> str:
878         return san.strip().rstrip(".,;:)").replace("0", "O").replace("+",
        ↪  "").replace("#", "").replace("!", "").replace("?", "")
879 
880     def parse_llm_move(
881         self,
882         board: chess.Board,
883         raw_response: str,
884         candidate_moves: list[chess.Move],
885     ) -> tuple[chess.Move, bool]:
886         """Parser ESTRICTO (V3).
887 
888         Solo cuenta como acierto limpio si el modelo nombra una de las
    ↪  opciones ofrecidas:
889           1. "Best move: <SAN>"  con el SAN exacto (se ignoran +, #, !,
    ↪  ?), o
890           2. "UCI: <uci>" exacto, o
891           3. la respuesta completa es un SAN exacto (p. ej. "Kxe2").
```

```
        Ya no se usa board.parse_san() (aceptaba "Bxf7" sin captura) ni se escanea la prosa
        buscando cualquier cosa con forma de jugada.
        """
        clean_text = raw_response.replace("*", "").replace("`", "").strip()
        san_map = {self._norm_san(board.san(m)): m for m in candidate_moves}
        candidate_set = set(candidate_moves)

        best = re.search(r"Best\s+move\s*:\s*(\S+)", clean_text, re.IGNORECASE)
        if best:
            mv = san_map.get(self._norm_san(best.group(1)))
            if mv is not None:
                return mv, True

        uci = re.search(r"UCI\s*:\s*([a-h][1-8][a-h][1-8][qrbn]?)\b", clean_text, re.IGNORECASE)
        if uci:
            try:
                mv = chess.Move.from_uci(uci.group(1).lower())
                if mv in candidate_set:
                    return mv, True
            except ValueError:
                pass

        mv = san_map.get(self._norm_san(clean_text))
        if mv is not None:
            return mv, True

        return self._fallback_move(candidate_moves), False

    def _fallback_move(self, candidate_moves: list[chess.Move]) -> chess.Move:
        if self.fallback == "engine" or len(candidate_moves) == 1:
            return candidate_moves[0]
        return self.rng.choice(candidate_moves)

    def request_model_move(self, model_name: str, board: chess.Board, mode: str | None = None) -> dict:
```

```
        if board.fen().startswith(
        ↪  "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR"):
            self.recent_positions.clear()
            self.last_piece_squares[chess.WHITE].clear()
            self.last_piece_squares[chess.BLACK].clear()
            self.consecutive_checks[chess.WHITE] = 0
            self.consecutive_checks[chess.BLACK] = 0

        turn = board.turn
        self.recent_positions.append(self._pos_key(board))
        mode = mode or self.mode
        candidate_moves = self.get_options(board, mode)
        user_prompt = self.build_prompt(board, candidate_moves)

        if "gemma" in model_name.lower():
            messages = [{"role": "user", "content":
↪   f"{SYSTEM_PROMPT}\n\n{user_prompt}"}]
        else:
            messages = [
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": user_prompt},
            ]

        payload = {
            "model": model_name,
            "messages": messages,
            "stream": False,
            "options": {
                "temperature": 0.05,
                "top_p": 0.9,
                "repeat_penalty": 1.15,
                "num_predict": 48,
                "num_ctx": 1024,  # V3: en modo "pure" la lista completa
                ↪  de legales es más larga
            },
        }

        req = urllib.request.Request(
            f"{self.ollama_url}/api/chat",
            data=json.dumps(payload).encode("utf-8"),
            headers={"Content-Type": "application/json"},
            method="POST",
```

```
        )

        try:
            with urllib.request.urlopen(req, timeout=45) as resp:
                data = json.loads(resp.read().decode("utf-8"))
                raw_text = data.get("message", {}).get("content", "")
        except Exception as e:
            raw_text = f"ERROR_OLLAMA: {e}"

        move, is_valid_llm = self.parse_llm_move(board, raw_text,
↪  candidate_moves)
        san_played = board.san(move)

        if board.gives_check(move) and not board.is_capture(move):
            self.consecutive_checks[turn] += 1
        else:
            self.consecutive_checks[turn] = 0

        self.last_piece_squares[turn].append(move.to_square)

        board.push(move)
        self.recent_positions.append(self._pos_key(board))
        board.pop()

        return {
            "move_uci": move.uci(),
            "move_san": san_played,
            "raw_output": raw_text.strip(),
            "clean_hit": is_valid_llm,
            "move_obj": move,
            "mode": mode,
            "options": [board.san(m) for m in candidate_moves][:40],
            "picked_first_option": move == candidate_moves[0],
        }
```

APÉNDICE H

La arena por consola y los diagnósticos

Tres scripts cortos que hicieron más por este libro que cualquier otro:

- **`match_runner.py`** juega partidas entre dos modelos, **alternando colores**, reclamando la triple repetición (`claim_draw`) y guardando cada resultado en `logs/arena_results.jsonl`. El modo (`--mode pure` o `--mode arbiter`) es un argumento, no una constante escondida.
- **`probe_bias.py`** es el sondeo del capítulo 9: las mismas posiciones del dataset para cada modelo, con su prompt de entrenamiento exacto, y dos contadores (formato y acierto). Sin tablero, sin árbitro, sin respaldo.
- **`duel_both_colors.py`** es el duelo del capítulo 8: dos partidas con el árbitro original y colores cambiados. Fue el que produjo la tabla que cambió la historia.

arena/match_runner.py

```
"""
Vertex Chess Arena — Match Runner V3 (torneo justo).

- Alterna colores en cada partida (antes Qwen siempre jugaba con blancas).
- Modo "pure" por defecto: el LLM elige entre TODAS las jugadas legales (mide al modelo).
  Modo "arbiter": el motor filtra top-4 y el LLM elige (mide al sistema híbrido).
- Fallback aleatorio: una respuesta inválida ya no se convierte en la mejor jugada del motor.
- Cobra tablas por triple repetición y regla de 50 jugadas (claim_draw=True).
```

```
- Guarda PGN en logs/matches_history.pgn y un resumen por partida en logs/arena_results.jsonl.

Correr desde E:\\vertex-chess-arena\\arena :
    ..\\venv\\Scripts\\python.exe match_runner.py --games 10 --mode pure
    ..\\venv\\Scripts\\python.exe match_runner.py --games 10 --mode arbiter --qwen vertexqwen
"""
import argparse
import datetime
import json
from pathlib import Path

import chess
import chess.pgn

from engine_arbiter import ChessArbiter

BASE_DIR = Path(__file__).resolve().parents[1]
PGN_LOG = BASE_DIR / "logs" / "matches_history.pgn"
RESULTS_LOG = BASE_DIR / "logs" / "arena_results.jsonl"


def play_game(white: str, black: str, mode: str, max_moves: int, seed: int, verbose: bool) -> dict:
    arbiter = ChessArbiter(mode=mode, fallback="random", seed=seed)
    board = chess.Board()
    game = chess.pgn.Game()
    game.headers["Event"] = f"Vertex Chess Arena V3 ({mode})"
    game.headers["Site"] = "Miami, FL (Vertex Coders Lab)"
    game.headers["Date"] = datetime.datetime.now().strftime("%Y.%m.%d")
    game.headers["White"] = white
    game.headers["Black"] = black
    node = game
    stats = {white: {"clean": 0, "fallback": 0}, black: {"clean": 0, "fallback": 0}}

    while not board.is_game_over(claim_draw=True) and board.ply() < max_moves * 2:
        current = white if board.turn == chess.WHITE else black
        res = arbiter.request_model_move(current, board)
        stats[current]["clean" if res["clean_hit"] else "fallback"] += 1
```

```
        if verbose:
            tag = "OK" if res["clean_hit"] else "FALLBACK"
            print(f"  {board.fullmove_number}{'.' if board.turn else
            ↪  '...'} {res['move_san']:<8} [{current} {tag}]
            ↪  {res['raw_output'][:60]!r}")
        board.push(res["move_obj"])
        node = node.add_variation(res["move_obj"])

    outcome = board.outcome(claim_draw=True)
    result = outcome.result() if outcome else "1/2-1/2"  # límite de
↪   jugadas = tablas
    termination = outcome.termination.name if outcome else "MAX_MOVES"
    game.headers["Result"] = result
    game.headers["Termination"] = termination
    with open(PGN_LOG, "a", encoding="utf-8") as f:
        f.write(str(game) + "\n\n")
    return {"white": white, "black": black, "mode": mode, "result":
    ↪  result,
            "termination": termination, "plies": board.ply(), "stats":
            ↪  stats}


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--games", type=int, default=10, help="número de
↪   partidas (se alternan colores)")
    ap.add_argument("--mode", choices=["pure", "arbiter"],
↪   default="pure")
    ap.add_argument("--qwen", default="vertexqwen-v3")
    ap.add_argument("--gemma", default="vertexgemma")
    ap.add_argument("--max-moves", type=int, default=150)
    ap.add_argument("--verbose", action="store_true")
    args = ap.parse_args()

    score = {args.qwen: 0.0, args.gemma: 0.0}
    totals = {args.qwen: {"clean": 0, "fallback": 0}, args.gemma:
↪  {"clean": 0, "fallback": 0}}
    for g in range(args.games):
        white, black = (args.qwen, args.gemma) if g % 2 == 0 else
↪   (args.gemma, args.qwen)
        print(f">>> Partida {g + 1}/{args.games}: {white} (B) vs {black}
        ↪  (N) | modo {args.mode}")
```

```
        r = play_game(white, black, args.mode, args.max_moves, seed=1000 + g, verbose=args.verbose)
        if r["result"] == "1-0":
            score[white] += 1
        elif r["result"] == "0-1":
            score[black] += 1
        else:
            score[white] += 0.5
            score[black] += 0.5
        for m in (white, black):
            for k in ("clean", "fallback"):
                totals[m][k] += r["stats"][m][k]
        r["timestamp"] = datetime.datetime.now().isoformat(timespec="seconds")
        with open(RESULTS_LOG, "a", encoding="utf-8") as f:
            f.write(json.dumps(r) + "\n")
        print(f"    {r['result']} ({r['termination']}, {r['plies']} plies) | marcador {score}")

    print("\n" + "=" * 65)
    print(f"RESULTADO FINAL ({args.games} partidas, modo {args.mode})")
    for m in (args.qwen, args.gemma):
        t = totals[m]
        n = t["clean"] + t["fallback"]
        rate = 100 * t["clean"] / n if n else 0
        print(f"  {m:<16} puntos {score[m]:>4} | jugadas válidas {t['clean']}/{n} ({rate:.1f}%)")
    print("=" * 65)


if __name__ == "__main__":
    main()
```

diagnostics/probe_bias.py

```
"""
Diagnóstico Vertex Chess Arena — ¿quién decide realmente las jugadas?
Correr desde E:\\vertex-chess-arena\\arena :
    ..\\venv\\Scripts\\python.exe ..\\diagnostics\\probe_bias.py 40 vertexqwen-v3 vertexgemma
Requiere Ollama corriendo con vertexqwen y vertexgemma.
"""
```

```
import json, random, sys, time, re
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "arena"))
import chess
from engine_arbiter import ChessArbiter, SYSTEM_PROMPT
import urllib.request

BASE = Path(__file__).resolve().parents[1]
OUT = BASE / "diagnostics" / "probe_results.jsonl"
N = int(sys.argv[1]) if len(sys.argv) > 1 else 40
MODELS = sys.argv[2:] or ["vertexqwen", "vertexgemma"]
random.seed(7)
recs = [json.loads(l) for l in open(BASE /
↪  "dataset/processed/train_chess_elite.jsonl", encoding="utf-8")]
sample = random.sample(recs, N)
TRAIN_SYS = recs[0]["messages"][0]["content"]

def chat(model, messages):
    payload = {"model": model, "messages": messages, "stream": False,
               "options": {"temperature": 0.05, "top_p": 0.9,
               ↪  "repeat_penalty": 1.15, "num_predict": 48}}
    req = urllib.request.Request("http://127.0.0.1:11434/api/chat",
↪  data=json.dumps(payload).encode(),
                                 headers={"Content-Type":
↪  "application/json"}, method="POST")
    with urllib.request.urlopen(req, timeout=120) as r:
        return json.loads(r.read())["message"]["content"]

def msgs(model, sys_p, user):
    if "gemma" in model:
        return [{"role": "user", "content": f"{sys_p}\n\n{user}"}]
    return [{"role": "system", "content": sys_p}, {"role": "user",
    ↪  "content": user}]

arb = ChessArbiter()
OUT.write_text("", encoding="utf-8")
for i, r in enumerate(sample):
    board = chess.Board(r["fen"])
    cands = arb._get_candidate_moves(board)
    arena_prompt = arb.build_prompt(board, cands)
    train_prompt = r["messages"][1]["content"]   # prompt EXACTO del
↪  entrenamiento (todas las legales)
```

```
    row = {"i": i, "fen": r["fen"], "label": r["san"], "source": r["source"],
          "cands": [board.san(m) for m in cands]}
    for model in MODELS:
        t = time.time()
        raw_a = chat(model, msgs(model, SYSTEM_PROMPT, arena_prompt))
        mv, ok = arb.parse_llm_move(board, raw_a, cands)
        raw_t = chat(model, msgs(model, TRAIN_SYS, train_prompt))
        legal = list(board.legal_moves)
        mv_t, ok_t = arb.parse_llm_move(board, raw_t, legal)
        row[model] = {"arena_raw": raw_a[:120], "arena_ok": ok, "arena_idx": cands.index(mv) if ok else -1,
                      "train_raw": raw_t[:120], "train_ok": ok_t, "train_hit": ok_t and board.san(mv_t) == r["san"],
                      "secs": round(time.time() - t, 1)}
    with open(OUT, "a", encoding="utf-8") as f:
        f.write(json.dumps(row, ensure_ascii=False) + "\n")
    print(i, row["label"], "|", " | ".join(row[m]["train_raw"][:40] for m in MODELS), flush=True)
print("\nRESUMEN (posiciones del dataset de entrenamiento)")
rows = [json.loads(l) for l in open(OUT, encoding="utf-8")]
for m in MODELS:
    fmt = sum(r[m]["train_raw"].startswith("Best move:") for r in rows)
    hit = sum(r[m]["train_hit"] for r in rows)
    print(f"  {m:<16} formato 'Best move:' {fmt}/{len(rows)} | acierta la jugada del dataset {hit}/{len(rows)}")
print("DONE")
```

diagnostics/duel_both_colors.py

```
"""Dos partidas con colores invertidos usando el mismo árbitro que la arena.
Correr desde E:\\vertex-chess-arena\\arena :  python ..\\diagnostics\\duel_both_colors.py"""
import sys, json
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "arena"))
import chess, chess.pgn
from engine_arbiter import ChessArbiter
OUT = Path(__file__).resolve().parent / "duel_results.txt"
OUT.write_text("", encoding="utf-8")
```

```
for white, black in (("vertexqwen", "vertexgemma"), ("vertexgemma",
↪  "vertexqwen")):
    arb = ChessArbiter(); board = chess.Board(); game = chess.pgn.Game()
    game.headers.update(White=white, Black=black)
    node = game; st = {white: [0, 0, 0], black: [0, 0, 0]}  # clean,
↪  fallback, eligio_candidata_0
    while not board.is_game_over() and board.ply() < 160:
        m = white if board.turn else black
        cands = arb._get_candidate_moves(board)
        res = arb.request_model_move(m, board)
        st[m][0 if res["clean_hit"] else 1] += 1
        st[m][2] += int(res["move_obj"] == cands[0])
        board.push(res["move_obj"]); node =
↪  node.add_variation(res["move_obj"])
    game.headers["Result"] = board.result(claim_draw=True)
    txt = f"{white} (W) vs {black} (B): {board.result(claim_draw=True)}
↪  plies={board.ply()} outcome={board.outcome(claim_draw=True)}\n  stats
↪  [clean, fallback, top1]: {st}\n{game}\n\n"
    print(txt, flush=True)
    with open(OUT, "a", encoding="utf-8") as f: f.write(txt)
print("DONE")
```

APÉNDICE I

La API del backend

`api_server.py` es la API V3 con FastAPI que usa la aplicación de torneos (capítulo 10). Se configura con variables de entorno (`OLLAMA_URL`, `ALLOWED_ORIGINS`, `ARENA_DATA_DIR`, `FRONTEND_DIST`), lo que la deja lista para desplegarse fuera de la máquina de desarrollo.

Tres detalles para leerla:

- **`arbiter_for`** guarda un árbitro por partida (`game_id`), porque el árbitro recuerda las posiciones recientes para su filtro anti-repetición. Dos partidas simultáneas no se mezclan.
- **`ai_move`** devuelve, además de la jugada, si salió limpia o fue respaldo, las opciones ofrecidas y el tiempo de respuesta. Si Ollama no contesta, responde **503** con el código `LLM_UNAVAILABLE`: nunca inventa una jugada.
- **Los errores** salen siempre con la misma forma, `{"error": {"code", "message"}}`, gracias a los tres manejadores de excepciones del principio.

Al final, la ruta comodín sirve la aplicación Angular compilada, y `/legacy` la página antigua.

arena/api_server.py

```
"""
Vertex Chess Arena — API V3 (FastAPI)
Vertex Coders LLC — Denis Sanchez Leyva, Miami FL

Sirve a la app Angular (frontend/) y a la página vieja
↪  (arena/index.html).

Endpoints
  GET  /health                    estado de la API y de Ollama
  GET  /api/players               jugadores disponibles (modelos de
↪  Ollama + motor del árbitro)
```

```
  POST /api/ai-move                jugada de una IA para un FEN (modos
↪  pure | arbiter | engine)
  POST /api/human-play             compatibilidad con la página vieja
  GET  /api/games                  historial de partidas guardadas
  POST /api/games                  guardar partida terminada (PGN con
↪  relojes)
  GET  /api/tournaments            lista de torneos
  GET  /api/tournaments/{id}       un torneo
  PUT  /api/tournaments/{id}       crear / actualizar un torneo

Configuración por variables de entorno (todas opcionales):
  OLLAMA_URL        http://127.0.0.1:11434
  ALLOWED_ORIGINS   http://localhost:4200,http://127.0.0.1:4200
  ARENA_DATA_DIR    <proyecto>/logs
  FRONTEND_DIST     <proyecto>/frontend/dist/vertex-chess-arena/browser

Correr desde E:\\vertex-chess-arena\\arena :
    ..\\venv\\Scripts\\python.exe -m uvicorn api_server:app --host
↪  127.0.0.1 --port 8000
"""
from __future__ import annotations

import json
import logging
import os
import re
import threading
import time
import urllib.request
import uuid
from collections import OrderedDict
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Literal

import chess
import chess.pgn
from fastapi import FastAPI, HTTPException, Query, Request
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, JSONResponse
from pydantic import BaseModel, Field
```

```
49
50 from engine_arbiter import ChessArbiter
51
52 log = logging.getLogger("vertex.arena")
53 logging.basicConfig(level=logging.INFO, format="%(asctime)s
   ↪  %(levelname)s %(name)s: %(message)s")
54
55 ARENA_DIR = Path(__file__).resolve().parent
56 BASE_DIR = ARENA_DIR.parent
57 OLLAMA_URL = os.getenv("OLLAMA_URL",
   ↪  "http://127.0.0.1:11434").rstrip("/")
58 ALLOWED_ORIGINS = [
59     o.strip()
60     for o in os.getenv("ALLOWED_ORIGINS",
       ↪  "http://localhost:4200,http://127.0.0.1:4200").split(",")
61     if o.strip()
62 ]
63 DATA_DIR = Path(os.getenv("ARENA_DATA_DIR", str(BASE_DIR / "logs")))
64 FRONTEND_DIST = Path(os.getenv("FRONTEND_DIST", str(BASE_DIR / "frontend"
   ↪  / "dist" / "vertex-chess-arena" / "browser")))
65
66 GAMES_LOG = DATA_DIR / "arena_games.jsonl"
67 PGN_LOG = DATA_DIR / "matches_history.pgn"
68 TOURNAMENTS_DIR = DATA_DIR / "tournaments"
69
70 ENGINE_ID = "vertex-engine"
71
72 # Plantel conocido. "available" se calcula en vivo contra Ollama.
73 ROSTER: list[dict[str, str]] = [
74     {"model": "vertexqwen-v3", "name": "Qwen 2.5 · 3B · V3", "family":
   ↪  "qwen",
75      "description": "Entrenamiento espejo de Gemma: 5,000 posiciones,
        ↪  pérdida solo en la respuesta, 625 pasos."},
76     {"model": "vertexgemma", "name": "Gemma 2 · 2B", "family": "gemma",
77      "description": "QLoRA r=8 sobre 5,000 posiciones, 625 pasos."},
78     {"model": "vertexqwen", "name": "Qwen 2.5 · 3B · legacy", "family":
   ↪  "qwen",
79      "description": "Adaptador viejo de 80 pasos (dataset de 3,000, sin
        ↪  máscara). Se conserva para comparar."},
80 ]
81 ROSTER_IDS = {p["model"] for p in ROSTER}
82
```

```
app = FastAPI(
    title="Vertex Chess Arena API",
    description="Duelo de modelos afinados (Qwen vs Gemma) con relojes y torneos — Vertex Coders LLC",
    version="3.0.0",
)
app.add_middleware(
    CORSMiddleware,
    allow_origins=ALLOWED_ORIGINS,
    allow_methods=["GET", "POST", "PUT"],
    allow_headers=["Content-Type"],
)


# ---------------------------------------------------------------------------
# Errores con formato único {"error": {"code", "message"}}
# ---------------------------------------------------------------------------
class ApiError(HTTPException):
    def __init__(self, status: int, code: str, message: str):
        super().__init__(status_code=status, detail={"code": code, "message": message})


@app.exception_handler(HTTPException)
async def http_error_handler(_: Request, exc: HTTPException):
    detail = exc.detail if isinstance(exc.detail, dict) else {"code": "HTTP_ERROR", "message": str(exc.detail)}
    return JSONResponse(status_code=exc.status_code, content={"error": detail})


@app.exception_handler(RequestValidationError)
async def validation_error_handler(_: Request, exc: RequestValidationError):
    first = exc.errors()[0] if exc.errors() else {}
    where = ".".join(str(p) for p in first.get("loc", []) if p != "body")
    return JSONResponse(
        status_code=422,
        content={"error": {"code": "INVALID_REQUEST", "message": f"Campo inválido: {where} — {first.get('msg', '')}"}},
```

```
    )


@app.exception_handler(Exception)
async def unhandled_error_handler(_: Request, exc: Exception):
    log.exception("Error no controlado: %s", exc)
    return JSONResponse(status_code=500, content={"error": {"code":
    ↪  "INTERNAL_ERROR", "message": "Error interno del servidor."}})


#
↪  ----------------------------------------------------------------------------
# Ollama
#
↪  ----------------------------------------------------------------------------
def ollama_models(timeout: float = 2.0) -> set[str] | None:
    """Nombres de modelos instalados en Ollama, o None si Ollama no
    ↪  responde."""
    try:
        with urllib.request.urlopen(f"{OLLAMA_URL}/api/tags",
        ↪  timeout=timeout) as resp:
            data = json.loads(resp.read().decode("utf-8"))
        names = set()
        for m in data.get("models", []):
            name = m.get("name", "")
            names.add(name)
            names.add(name.removesuffix(":latest"))
        return names
    except Exception:
        return None


#
↪  ----------------------------------------------------------------------------
# Un árbitro por partida (el árbitro guarda historial para el
↪  anti-repetición)
#
↪  ----------------------------------------------------------------------------
_ARBITERS: "OrderedDict[str, ChessArbiter]" = OrderedDict()
_ARBITERS_LOCK = threading.Lock()
_MAX_ARBITERS = 128

```

```python

def arbiter_for(game_id: str | None) -> ChessArbiter:
    key = game_id or "__shared__"
    with _ARBITERS_LOCK:
        arb = _ARBITERS.get(key)
        if arb is None:
            arb = ChessArbiter(ollama_url=OLLAMA_URL, fallback="random")
            _ARBITERS[key] = arb
        _ARBITERS.move_to_end(key)
        while len(_ARBITERS) > _MAX_ARBITERS:
            _ARBITERS.popitem(last=False)
        return arb


#
↪ ----------------------------------------------------------------------------
# Modelos de request / response
#
↪ ----------------------------------------------------------------------------
GAME_ID_PATTERN = r"^[A-Za-z0-9_-]{1,80}$"
TOURNAMENT_ID_PATTERN = r"^[a-z0-9-]{6,64}$"


class AIMoveRequest(BaseModel):
    fen: str = Field(..., max_length=120, description="Tablero en FEN")
    model_name: str = Field("vertexqwen-v3", max_length=60)
    mode: Literal["pure", "arbiter", "engine"] = "pure"
    game_id: str | None = Field(None, pattern=GAME_ID_PATTERN)


class HumanMoveRequest(BaseModel):
    fen: str = Field(..., max_length=120)
    move_san_or_uci: str = Field(..., max_length=12)
    ai_opponent: str = Field("vertexqwen-v3", max_length=60)
    mode: Literal["pure", "arbiter", "engine"] = "pure"


class SavedGame(BaseModel):
    pgn: str = Field(..., max_length=200_000)
    white: str = Field(..., max_length=80)
    black: str = Field(..., max_length=80)
    result: Literal["1-0", "0-1", "1/2-1/2", "*"]
```

```
    termination: str = Field(..., max_length=60)
    time_control: str = Field(..., max_length=20)
    plies: int = Field(..., ge=0, le=2000)
    tournament_id: str | None = Field(None,
↪   pattern=TOURNAMENT_ID_PATTERN)
    round: int | None = Field(None, ge=1, le=500)
    stats: dict[str, Any] = Field(default_factory=dict)


def _parse_board(fen: str) -> chess.Board:
    try:
        board = chess.Board(fen)
    except ValueError:
        raise ApiError(400, "INVALID_FEN", "El FEN no es válido.")
    if not board.is_valid():
        raise ApiError(400, "INVALID_FEN", "La posición no es legal.")
    return board


#
↪  ----------------------------------------------------------------------------
# Rutas
#
↪  ----------------------------------------------------------------------------
@app.get("/health")
def health():
    models = ollama_models()
    return {"status": "ok", "ollama": "up" if models is not None else
    ↪   "down", "version": app.version}


@app.get("/api/players")
def players():
    installed = ollama_models()
    out = []
    for p in ROSTER:
        out.append({**p, "kind": "llm", "available": bool(installed and
↪   p["model"] in installed)})
    out.append({
        "model": ENGINE_ID, "name": "Motor del árbitro", "family":
        ↪   "engine", "kind": "engine",
```

```
        "description": "Alpha-beta prof. 2 + quiescencia, sin LLM. Línea base para comparar.",
        "available": True,
    })
    return {"ollama": "up" if installed is not None else "down", "players": out}


@app.post("/api/ai-move")
def ai_move(req: AIMoveRequest):
    board = _parse_board(req.fen)
    if board.is_game_over():
        raise ApiError(409, "GAME_OVER", "La partida ya terminó en esa posición.")

    engine_only = req.mode == "engine" or req.model_name == ENGINE_ID
    if not engine_only and req.model_name not in ROSTER_IDS:
        raise ApiError(400, "UNKNOWN_MODEL", f"Modelo desconocido: {req.model_name}")

    arbiter = arbiter_for(req.game_id)
    t0 = time.perf_counter()
    if engine_only:
        move = arbiter.get_options(board, "arbiter")[0]
        san = board.san(move)
        arbiter.recent_positions.append(arbiter._pos_key(board))
        board.push(move)
        arbiter.recent_positions.append(arbiter._pos_key(board))
        board.pop()
        res = {"move_obj": move, "move_san": san, "move_uci": move.uci(), "clean_hit": True,
               "raw_output": "motor", "mode": "engine", "options": [san]}
    else:
        res = arbiter.request_model_move(req.model_name, board, mode=req.mode)
        if res["raw_output"].startswith("ERROR_OLLAMA"):
            log.warning("Ollama no respondió para %s: %s", req.model_name, res["raw_output"])
            raise ApiError(503, "LLM_UNAVAILABLE", "Ollama no respondió. ¿Está corriendo y el modelo creado?")
    think_ms = int((time.perf_counter() - t0) * 1000)

```

```
    board.push(res["move_obj"])
    return {
        "model": req.model_name,
        "mode": res["mode"],
        "played_san": res["move_san"],
        "played_uci": res["move_uci"],
        "clean_hit": res["clean_hit"],
        "raw_output": res["raw_output"][:300],
        "options_count": len(res.get("options", [])),
        "think_ms": think_ms,
        "new_fen": board.fen(),
        "game_over": board.is_game_over(),
        "result": board.result() if board.is_game_over() else None,
    }


@app.post("/api/human-play")
def human_play(req: HumanMoveRequest):
    """Compatibilidad con arena/index.html (página vieja)."""
    board = _parse_board(req.fen)
    move = None
    try:
        move = board.parse_san(req.move_san_or_uci)
    except ValueError:
        try:
            cand = chess.Move.from_uci(req.move_san_or_uci.lower())
            move = cand if cand in board.legal_moves else None
        except ValueError:
            move = None
    if move is None:
        raise ApiError(400, "ILLEGAL_MOVE", f"Movimiento ilegal:
        ↪ {req.move_san_or_uci}")
    human_san = board.san(move)
    board.push(move)
    if board.is_game_over():
        return {"human_move": human_san, "ai_move_san": None, "new_fen":
        ↪ board.fen(),
                "game_over": True, "result": board.result()}
    ai = ai_move(AIMoveRequest(fen=board.fen(),
↪   model_name=req.ai_opponent, mode=req.mode))
    return {"human_move": human_san, "ai_model": req.ai_opponent,
    ↪ "ai_move_san": ai["played_san"],
```

```
            "ai_move_uci": ai["played_uci"], "ai_clean_hit":
            ↪  ai["clean_hit"], "new_fen": ai["new_fen"],
            "game_over": ai["game_over"], "result": ai["result"]}


_FILE_LOCK = threading.Lock()


@app.post("/api/games", status_code=201)
def save_game(game: SavedGame):
    if chess.pgn.read_game(__import__("io").StringIO(game.pgn)) is None:
        raise ApiError(400, "INVALID_PGN", "El PGN no se pudo leer.")
    record = game.model_dump()
    record["id"] = uuid.uuid4().hex[:12]
    record["saved_at"] =
↪   datetime.now(timezone.utc).isoformat(timespec="seconds")
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    with _FILE_LOCK:
        with open(GAMES_LOG, "a", encoding="utf-8") as f:
            f.write(json.dumps(record, ensure_ascii=False) + "\n")
        with open(PGN_LOG, "a", encoding="utf-8") as f:
            f.write(game.pgn.strip() + "\n\n")
    return {"id": record["id"], "saved_at": record["saved_at"]}


@app.get("/api/games")
def list_games(limit: int = Query(50, ge=1, le=500), tournament_id: str |
↪   None = None):
    if not GAMES_LOG.exists():
        return []
    rows = []
    with open(GAMES_LOG, encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            try:
                rec = json.loads(line)
            except json.JSONDecodeError:
                continue
            if tournament_id and rec.get("tournament_id") !=
            ↪  tournament_id:
```

```
                continue
            rows.append(rec)
    return list(reversed(rows))[:limit]


def _tournament_path(tid: str) -> Path:
    if not re.fullmatch(TOURNAMENT_ID_PATTERN, tid):
        raise ApiError(400, "INVALID_ID", "Identificador de torneo
        ↪  inválido.")
    return TOURNAMENTS_DIR / f"{tid}.json"


@app.get("/api/tournaments")
def list_tournaments():
    if not TOURNAMENTS_DIR.exists():
        return []
    out = []
    for p in TOURNAMENTS_DIR.glob("*.json"):
        try:
            t = json.loads(p.read_text(encoding="utf-8"))
        except (json.JSONDecodeError, OSError):
            continue
        out.append({k: t.get(k) for k in ("id", "name", "format",
↪  "status", "createdAt", "timeControl")} |
                   {"players": len(t.get("participants", []))})
    return sorted(out, key=lambda t: t.get("createdAt") or "",
    ↪  reverse=True)


@app.get("/api/tournaments/{tid}")
def get_tournament(tid: str):
    path = _tournament_path(tid)
    if not path.exists():
        raise ApiError(404, "NOT_FOUND", "Torneo no encontrado.")
    return json.loads(path.read_text(encoding="utf-8"))


@app.put("/api/tournaments/{tid}")
async def put_tournament(tid: str, request: Request):
    path = _tournament_path(tid)
    raw = await request.body()
    if len(raw) > 2_000_000:
```

```
375         raise ApiError(413, "TOO_LARGE", "El torneo es demasiado
    ↪  grande.")
376     try:
377         data = json.loads(raw)
378     except json.JSONDecodeError:
379         raise ApiError(400, "INVALID_JSON", "JSON inválido.")
380     if not isinstance(data, dict) or data.get("id") != tid:
381         raise ApiError(400, "INVALID_TOURNAMENT", "El cuerpo debe ser un
    ↪  torneo con el mismo id de la URL.")
382     TOURNAMENTS_DIR.mkdir(parents=True, exist_ok=True)
383     tmp = path.with_suffix(".tmp")
384     with _FILE_LOCK:
385         tmp.write_text(json.dumps(data, ensure_ascii=False, indent=1),
    ↪  encoding="utf-8")
386         tmp.replace(path)
387     return {"ok": True}
388
389
390 #
    ↪  ------------------------------------------------------------------------------
391 # Frontend: app Angular compilada si existe; si no, la página vieja.
392 #
    ↪  ------------------------------------------------------------------------------
393 @app.get("/legacy")
394 def legacy_ui():
395     return FileResponse(ARENA_DIR / "index.html")
396
397
398 @app.get("/{full_path:path}")
399 def spa(full_path: str):
400     if full_path.startswith("api/"):
401         raise ApiError(404, "NOT_FOUND", "Ruta no encontrada.")
402     if FRONTEND_DIST.exists():
403         target = (FRONTEND_DIST / full_path).resolve()
404         if full_path and target.is_file() and FRONTEND_DIST.resolve() in
    ↪  target.parents:
405             return FileResponse(target)
406         return FileResponse(FRONTEND_DIST / "index.html")
407     return FileResponse(ARENA_DIR / "index.html")
```

APÉNDICE J

El reloj y la lógica de torneos

Dos archivos TypeScript de la aplicación Angular, sin dependencias de interfaz: se pueden probar y reutilizar por separado.

`chess-clock.ts` es el reloj Bronstein del capítulo 10. El estado vive en *signals* de Angular. La línea clave está en `press`: al terminar una jugada se devuelve lo pensado, **hasta** el incremento, así que el reloj nunca sube. `tick` detecta la caída de bandera.

`tournament-logic.ts` contiene todo el reglamento de los torneos:

- `bergerRounds`: el calendario de todos contra todos con el método del círculo de Berger, con un "descanso" ficticio si el número de jugadores es impar.
- `buildPairings`: convierte las rondas en partidas y reparte los colores de forma voraz para que cada jugador tenga blancas y negras equilibradas.
- `standings`: puntos, partidas, victorias, tablas, derrotas, desempate **Sonneborn-Berger**, porcentaje de jugadas válidas, tiempo medio y caídas de bandera.
- `crosstable`: la tabla cruzada.

frontend/src/app/core/chess-clock.ts

```
import { computed, signal } from '@angular/core';
import { Side, TimeControl } from './models';

/**
 * Reloj de ajedrez con demora Bronstein (la de los torneos de la US
 ↪  Chess).
 *
 * Ritmo "3+2" = 3 minutos por jugador y hasta 2 s de devolución por
 ↪  jugada: al terminar
 * una jugada se le devuelve al jugador lo que pensó, con un tope de 2 s.
 ↪  Por eso el reloj
```

```
 * NUNCA sube por encima de lo que tenía al empezar su turno, y si llega a 0:00 pierde.
 * (Con Fischer, el sistema de la FIDE, se suman siempre los 2 s y el reloj puede subir si
 * se juega rápido; con modelos que responden en ~1.5 s eso confundía.)
 *
 * - El tiempo se mide con un reloj monotónico (performance.now) y se refresca cada 100 ms;
 *   el valor exacto se calcula siempre a partir de la marca de inicio, así el redondeo del
 *   intervalo nunca acumula error.
 * - press(side): el jugador `side` terminó su jugada a se le descuenta lo que pensó, se le
 *   devuelve hasta `incMs` de ese tiempo y empieza a correr el reloj del rival.
 * - pause()/resume() congelan el reloj (pausas visuales del modo IA vs IA, errores de red).
 */
export class ChessClock {
  private readonly remaining = signal<Record<Side, number>>({ w: 0, b: 0 });
  private readonly runningSide = signal<Side | null>(null);
  private readonly now = signal(0);
  private startedAt = 0;
  private timer: ReturnType<typeof setInterval> | null = null;
  private flagged = false;

  static readonly MODE = 'Bronstein';

  /** Tiempo restante en vivo de cada lado. */
  readonly time = computed<Record<Side, number>>(() => {
    const rem = this.remaining();
    const side = this.runningSide();
    if (!side) return rem;
    const elapsed = this.now() - this.startedAt;
    return { ...rem, [side]: Math.max(0, rem[side] - elapsed) } as Record<Side, number>;
  });

  readonly active = this.runningSide.asReadonly();

  constructor(
```

```
    readonly control: TimeControl,
    private readonly onFlag: (side: Side) => void,
    private readonly clockFn: () => number = () => performance.now(),
    private readonly tickMs = 100,
  ) {
    this.remaining.set({ w: control.baseMs, b: control.baseMs });
  }

  /** Arranca el reloj de `side` (inicio de partida o reanudación). */
  start(side: Side): void {
    if (this.flagged) return;
    this.startedAt = this.clockFn();
    this.now.set(this.startedAt);
    this.runningSide.set(side);
    this.ensureTimer();
  }

  /**
   * `side` completó su jugada. Devuelve el tiempo que pensó y el reloj
↪  que le queda
   * (ya con la devolución Bronstein: min(lo pensado, demora)).
   */
  press(side: Side): { thinkMs: number; remainingMs: number } {
    const t = this.clockFn();
    const rem = { ...this.remaining() };
    let thinkMs = 0;
    if (this.runningSide() === side) {
      thinkMs = t - this.startedAt;
      rem[side] = Math.max(0, rem[side] - thinkMs);
    }
    rem[side] += Math.min(thinkMs, this.control.incMs);
    this.remaining.set(rem);
    const other: Side = side === 'w' ? 'b' : 'w';
    this.startedAt = t;
    this.now.set(t);
    this.runningSide.set(other);
    this.ensureTimer();
    return { thinkMs: Math.round(thinkMs), remainingMs: rem[side] };
  }

  /** Congela el reloj y devuelve el lado que estaba corriendo. */
  pause(): Side | null {
```

```
    const side = this.runningSide();
    if (side) {
      const rem = { ...this.remaining() };
      rem[side] = Math.max(0, rem[side] - (this.clockFn() -
↪   this.startedAt));
      this.remaining.set(rem);
    }
    this.runningSide.set(null);
    this.clearTimer();
    return side;
  }

  stop(): void {
    this.pause();
  }

  /** Para pruebas y para forzar la comprobación de bandera. */
  tick(): void {
    const side = this.runningSide();
    if (!side) return;
    const t = this.clockFn();
    this.now.set(t);
    if (!this.flagged && this.remaining()[side] - (t - this.startedAt) <=
    ↪  0) {
      this.flagged = true;
      this.pause();
      const rem = { ...this.remaining() };
      rem[side] = 0;
      this.remaining.set(rem);
      this.onFlag(side);
    }
  }

  get hasFlagged(): boolean {
    return this.flagged;
  }

  destroy(): void {
    this.clearTimer();
    this.runningSide.set(null);
  }

```

```
  private ensureTimer(): void {
    if (this.timer) return;
    this.timer = setInterval(() => this.tick(), this.tickMs);
  }

  private clearTimer(): void {
    if (this.timer) {
      clearInterval(this.timer);
      this.timer = null;
    }
  }
}
```

frontend/src/app/core/tournament-logic.ts

```
import { Contender, GameResult, Pairing, Side, Tournament,
↪  TournamentFormat } from './models';
import { newId } from './live-game';

/**
 * Emparejamientos round robin con las tablas de Berger (método del
↪  círculo).
 * Con n impar se añade un "descanso" (null) y quien lo toca no juega esa
↪  ronda.
 * Los colores se asignan para que cada jugador termine con ±1 blancas
↪  respecto a negras
 * (gana blancas quien lleva menos; a igualdad, quien jugó con negras la
↪  ronda anterior).
 */
export function bergerRounds(ids: string[]): Array<Array<[string,
↪  string]>> {
  const players: Array<string | null> = [...ids];
  if (players.length % 2 === 1) players.push(null);
  const n = players.length;
  const rounds: Array<Array<[string, string]>> = [];
  const rot = players.slice(1);
  // Balance de colores: diff = blancas - negras; last = color de la
  ↪  ronda anterior.
  const diff = new Map<string, number>(ids.map((id) => [id, 0]));
  const last = new Map<string, 'w' | 'b' | null>(ids.map((id) => [id,
  ↪  null]));
  for (let r = 0; r < n - 1; r++) {
```

```
    const circle = [players[0], ...rot];
    const games: Array<[string, string]> = [];
    for (let i = 0; i < n / 2; i++) {
      const a = circle[i];
      const b = circle[n - 1 - i];
      if (a === null || b === null) continue;
      const da = diff.get(a)!;
      const db = diff.get(b)!;
      let aWhite: boolean;
      if (da !== db) aWhite = da < db;
      else if (last.get(a) !== last.get(b)) aWhite = last.get(a) === 'b'
      ↪  || last.get(b) === 'w';
      else aWhite = (r + i) % 2 === 0;
      const [w, bl] = aWhite ? [a, b] : [b, a];
      diff.set(w, diff.get(w)! + 1);
      diff.set(bl, diff.get(bl)! - 1);
      last.set(w, 'w');
      last.set(bl, 'b');
      games.push([w, bl]);
    }
    rounds.push(games);
    rot.unshift(rot.pop()!);
  }
  return rounds;
}

export function buildPairings(format: TournamentFormat, participants:
↪  Contender[], matchGames = 2): Pairing[] {
  const ids = participants.map((p) => p.id);
  const pairings: Pairing[] = [];
  if (format === 'match') {
    const [a, b] = ids;
    for (let g = 0; g < matchGames; g++) {
      pairings.push({ id: newId(8), round: g + 1, white: g % 2 === 0 ? a
↪  : b, black: g % 2 === 0 ? b : a });
    }
    return pairings;
  }
  const single = bergerRounds(ids);
  single.forEach((games, r) => games.forEach(([w, b]) => pairings.push({
↪  id: newId(8), round: r + 1, white: w, black: b })));
  if (format === 'double-round-robin') {
```

```
    const offset = single.length;
    single.forEach((games, r) =>
      games.forEach(([w, b]) => pairings.push({ id: newId(8), round:
↪  offset + r + 1, white: b, black: w })),
    );
  }
  return pairings;
}

export function scoreFor(result: GameResult, side: Side): number {
  if (result === '1/2-1/2') return 0.5;
  return (result === '1-0') === (side === 'w') ? 1 : 0;
}

export interface StandingRow {
  id: string;
  contender: Contender;
  rank: number;
  points: number;
  played: number;
  wins: number;
  draws: number;
  losses: number;
  sonnebornBerger: number;
  whiteGames: number;
  validMovePct: number | null;
  avgThinkMs: number | null;
  timeLosses: number;
}

/**
 * Clasificación: puntos a Sonneborn-Berger a victorias a enfrentamiento
↪  directo implícito
 * (no se calcula) a nombre. SB = suma de puntos de los rivales vencidos +
↪  mitad de los empatados.
 */
export function standings(t: Tournament): StandingRow[] {
  const rows = new Map<string, StandingRow>();
  const telemetry = new Map<string, { clean: number; fallback: number;
  ↪  thinkMs: number; moves: number }>();
  for (const c of t.participants) {
    rows.set(c.id, {
```

```
      id: c.id, contender: c, rank: 0, points: 0, played: 0, wins: 0, draws: 0, losses: 0,
      sonnebornBerger: 0, whiteGames: 0, validMovePct: null, avgThinkMs: null, timeLosses: 0,
    });
    telemetry.set(c.id, { clean: 0, fallback: 0, thinkMs: 0, moves: 0 });
  }
  const finished = t.pairings.filter((p) => p.result && p.termination !== 'aborted');
  for (const p of finished) {
    const res = p.result!;
    for (const side of ['w', 'b'] as Side[]) {
      const id = side === 'w' ? p.white : p.black;
      const row = rows.get(id);
      if (!row) continue;
      const pts = scoreFor(res, side);
      row.points += pts;
      row.played++;
      if (side === 'w') row.whiteGames++;
      if (pts === 1) row.wins++;
      else if (pts === 0.5) row.draws++;
      else row.losses++;
      if (p.stats?.timeouts === side) row.timeLosses++;
      const st = p.stats?.[side];
      const tel = telemetry.get(id)!;
      if (st) {
        tel.clean += st.clean;
        tel.fallback += st.fallback;
        tel.thinkMs += st.thinkMs;
        tel.moves += st.moves;
      }
    }
  }
  for (const p of finished) {
    const res = p.result!;
    const w = rows.get(p.white);
    const b = rows.get(p.black);
    if (!w || !b) continue;
    w.sonnebornBerger += scoreFor(res, 'w') * b.points;
    b.sonnebornBerger += scoreFor(res, 'b') * w.points;
  }
  for (const [id, row] of rows) {
```

```
    const tel = telemetry.get(id)!;
    const judged = tel.clean + tel.fallback;
    // El motor no usa LLM: el % de respuestas válidas no aplica.
    row.validMovePct = judged && row.contender.mode !== 'engine' ? (100 * tel.clean) / judged : null;
    row.avgThinkMs = tel.moves ? tel.thinkMs / tel.moves : null;
  }
  const sorted = [...rows.values()].sort(
    (a, b) =>
      b.points - a.points ||
      b.sonnebornBerger - a.sonnebornBerger ||
      b.wins - a.wins ||
      a.contender.label.localeCompare(b.contender.label),
  );
  sorted.forEach((r, i) => {
    const prev = sorted[i - 1];
    r.rank = prev && prev.points === r.points && prev.sonnebornBerger === r.sonnebornBerger && prev.wins === r.wins ? prev.rank : i + 1;
  });
  return sorted;
}

/** Tabla cruzada: celdas con los resultados de la fila contra la columna ("1", "½", "0"). */
export function crosstable(t: Tournament): Map<string, Map<string, string[]>> {
  const table = new Map<string, Map<string, string[]>>();
  for (const a of t.participants) {
    const row = new Map<string, string[]>();
    for (const b of t.participants) row.set(b.id, []);
    table.set(a.id, row);
  }
  const symbol = (x: number) => (x === 1 ? '1' : x === 0.5 ? '½' : '0');
  for (const p of t.pairings) {
    if (!p.result || p.termination === 'aborted') continue;
    table.get(p.white)?.get(p.black)?.push(symbol(scoreFor(p.result, 'w')));
    table.get(p.black)?.get(p.white)?.push(symbol(scoreFor(p.result, 'b')));
  }
  return table;
}
```

```

export function progress(t: Tournament): { done: number; total: number }
↪ {
  return { done: t.pairings.filter((p) => !!p.result).length, total:
  ↪  t.pairings.length };
}

export const FORMAT_LABEL: Record<TournamentFormat, string> = {
  'round-robin': 'Todos contra todos',
  'double-round-robin': 'Todos contra todos (ida y vuelta)',
  match: 'Match',
};
```

Sobre el autor

Denis Sánchez Leyva es fundador y desarrollador principal de **Vertex Coders LLC**, en Miami, Florida. Construye productos con inteligencia artificial para web, móvil y ciberseguridad, y afina modelos de lenguaje propios en su propio hardware: una GTX 1660 Ti de 6 GB que, como cuenta este libro, da para mucho más de lo que parece.

Programa en las horas libres que le deja su trabajo nocturno, y documenta cada proyecto con una bitácora, un video y un libro. Ha publicado una docena de libros en Amazon, en español e inglés, sobre agentes de IA, el protocolo MCP, interfaces con Jetpack Compose y programación cuántica, entre otros temas.

Jaque al modelo es el relato honesto de uno de esos proyectos, con sus errores incluidos.

Para seguir el proyecto

- Código: `https://github.com/Denisijcu/vertex-chess-arena`
- Modelo Qwen: `https://huggingface.co/Denisijcu/vertexqwen`
- Modelo Gemma: `https://huggingface.co/Denisijcu/vertexgemma`
- Video: `https://youtu.be/2_t0ijTg1_M`